

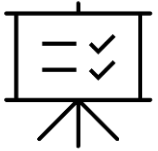
Advanced Algorithms & Data Structures

AIM OF THE SESSION



To familiarize students with the basics of data structures, and algorithmic analysis to evaluate time complexities.

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

- Understand the concept of Data Structures.
- Understand the concept of arrays.
- Analyse the how the access array elements..

LEARNING OUTCOMES

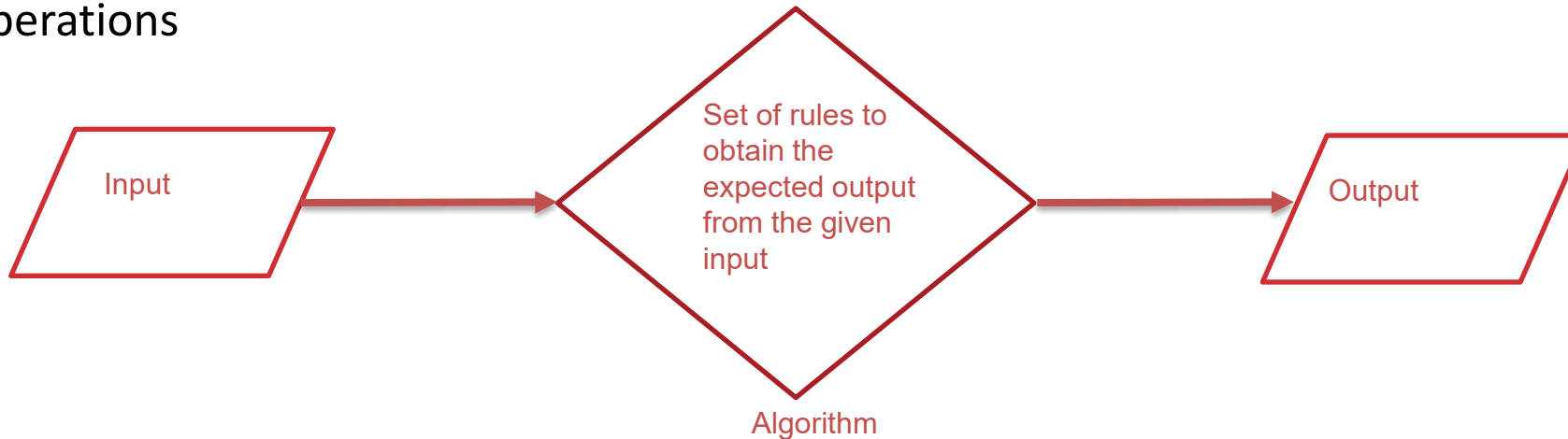


At the end of this session, you should be able to:

- Evaluate the time complexities of the different Algorithmic solutions for various types of Problems.

ALGORITHM

- A set of finite rules or instructions to be followed in calculations or other problem-solving operations
(or)
- A procedure for solving a mathematical problem in a finite number of steps that frequently involves recursive operations



DATA STRUCTURE

- Data Structures are used to store and organize the data in memory in an efficient manner
- It is a way of arranging data on a computer to be accessed and updated efficiently.
- Data Structure is used not only for organizing the data but also for processing, retrieving, and storing data.

COMPLEXITY OF AN ALGORITHM

- Complexity in algorithms refers to the amount of resources (such as time or memory/Space) required to solve a problem or perform a task.
- The most common measure of complexity is time complexity, which refers to the amount of time an algorithm takes to produce a result as a function of the size of the input.
- Memory/Space complexity refers to the amount of memory used by an algorithm.

TIME COMPLEXITY

- Time taken by the algorithm to solve the problem.
- It is measured by calculating the iteration of loops, the number of comparisons etc.
- Time complexity is a function describing the amount of time an algorithm takes in terms of the amount of input to the algorithm.

Array in Data Structure

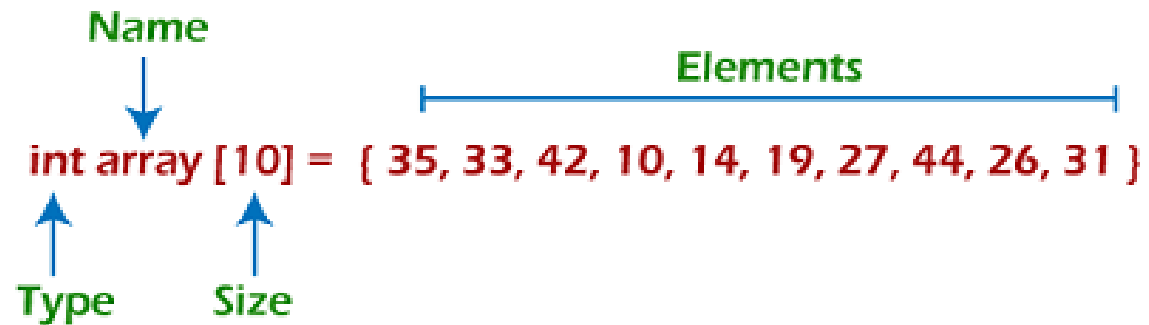
Arrays are defined as the collection of similar types of data items stored at contiguous memory locations. It is one of the simplest data structures where each data element can be randomly accessed by using its index number.

Properties of array:

- Each element in an array is of the same data type and carries the same size that is 4 bytes.
- Elements in the array are stored at contiguous memory locations from which the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

Representation of an array

We can represent an array in various ways in different programming languages. As an illustration, let's see the declaration of array in C language –

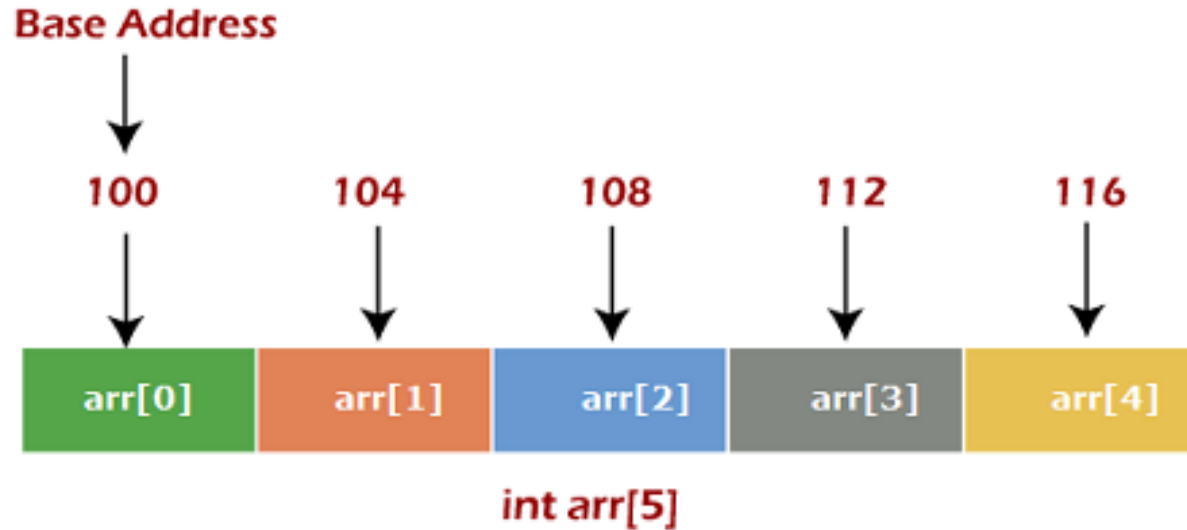


As per the above illustration, there are some of the following important points -

- Index starts with 0.
- The array's length is 10, which means we can store 10 elements.
- Each element in the array can be accessed via its index.

- Why are arrays required?
- Arrays are useful because -
- Sorting and searching a value in an array is easier.
- Arrays are best to process multiple values quickly and easily.

Memory allocation of an array

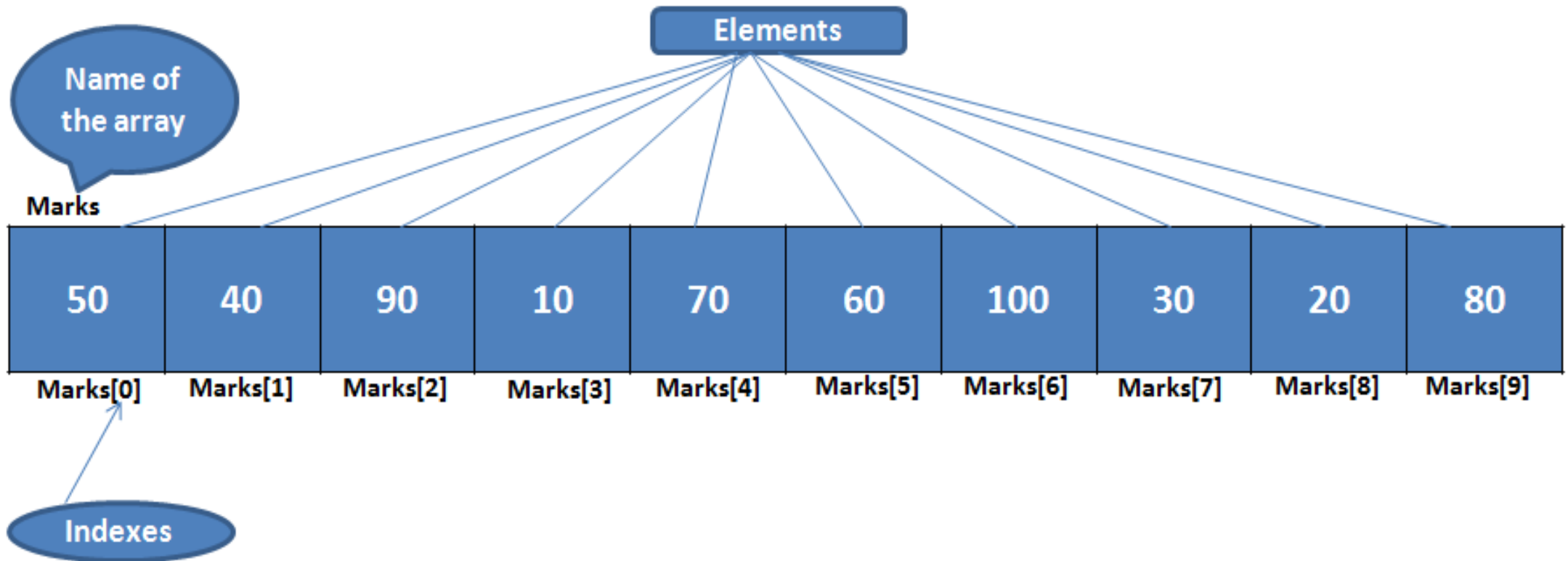


Arrays

- An array is a fundamental data structure that enables the storing and manipulation of potentially huge quantities of data.
- An array can be defined as a data structure consisting of an ordered set of data values of the homogeneous type.
- An array is a collection of individual data elements that is
 - a. Ordered
 - b. Fixed in size
 - c. Homogeneous

- The elements of the array are stored in consecutive memory locations and are referenced by an integer index also known as subscript. The index begins at a zero and is always written inside square brackets.
- Following are the two types of arrays:
 1. Single dimensional arrays
 2. Multi dimensional arrays
- Single dimensional arrays are represented as vectors containing a row of values.
- Multi dimensional arrays can be viewed as tables containing several rows and columns of data.

- Example: *int marks[10];*



Declaration:

- **Syntax:** *data-type ArrayName[SIZE];*
- Ex: 1. int ages[10];
 2. float heights[20];
 3. char name[20];
- The **data-type** can be any basic data type or an user-defined data type.
- **Array Name** must be a valid identifier.
- The **SIZE** of an array represents the maximum number of elements in the array.

Initialization:

- **Syntax:** *Data-type array-name[size] = { value list } ;*
- The **value list** is a comma-separated list of constants.
- Ex:

```
void main()
{
    int age[5] = {30,25,45,35,60};
    printf("%d" , age[0]);
}
```

Accessing Elements:

- **Syntax:** *ArrayName[index]*
- To access array elements in an array, specify the array name, followed by square braces enclosing an integer, which is called the array index.
- The array index indicates the particular element of the array which we want to access.
- For example if we want to access the fifth element of an array "A" then write:

printf("%d", A[4]);

Operations on Arrays:

- There are number of operations that can be performed on arrays. These operations include:
 - ☐ Traversing an array
 - ☐ Inserting an element in an array
 - ☐ Deleting an element form an array
 - ☐ Merging two arrays
 - ☐ Rotation elements of an array
 - ☐ Reversing array elements
 - ☐ Searching an element in an array
 - ☐ Sorting an array in ascending order

Traversing an Array:

- Traversing means to access all the elements of the array, starting from first element up to the last element in the array on-by-one.
- Algorithm:
 1. Let LB be the lower bound and the UP be the upper bound of linear array A.
 2. Set $i = \text{Lower Bound LB}$
 3. Repeat for $i = \text{LB to UB}$
Display $A[i]$
[End of the loop]
 4. Exit

Inserting an element in an array:

- For inserting the element at required position, elements must be moved one position right to new locations.
- Algorithm:
 1. Set $i = n - 1$
 2. Repeat steps while $i \geq \text{pos}$
 - Set $A[i+1] = A[i]$ [shift the elements by 1 position right]
 - Set $i = i - 1$[End of loop]
 3. Set $A[\text{pos}] = \text{ele}$
 4. Set $n = n + 1$
 5. End

Deleting an element from an array:

- For deleting the element at required position, elements must be moved one position left from the next position of the deleting element.
- Algorithm:
 1. Set $i = \text{pos}$
 2. Repeat steps while $i \leq n-1$
 - Set $A[i] = A[i+1]$ [shift the elements by 1 position left]
 - Set $i = i + 1$[End of loop]
 3. Set $n = n - 1$
 4. End

Merging Two Sorted Arrays:

Given two sorted arrays of size m and n respectively. Also given an empty third array of size $(m+n)$. write code to merge the two arrays and store the result in the third array.

Input:

$a = \{1, 4, 5, 9\}$

$b = \{0, 2, 3, 7, 15, 29\}$

Output:

$c = \{0, 1, 2, 3, 4, 5, 7, 9, 15, 29\}$

Algorithm:

- Algorithm to merge two arrays $A[0..m-1]$ and $B[0..n-1]$ into an array $C[0..m+n-1]$ is as following:
 1. Introduce read-indices **i**, **j** to traverse arrays A and B, accordingly. Introduce write-index **k** to store position of the first free cell in the resulting array. By default **i** = **j** = **k** = 0.
 2. At each step: if both indices are in range (**i** < m and **j** < n), choose minimum of ($A[\mathbf{i}]$, $B[\mathbf{j}]$) and write it to $C[\mathbf{k}]$. Otherwise go to step 4.

3. Increase **k** and index of the array, algorithm located minimal value at, by one.
Repeat step 2.
4. Copy the rest values from the array, which index is still in range, to the resulting array.

Rotate an array by d elements left:

- Write a C program to left rotate an array by d position.

Example:

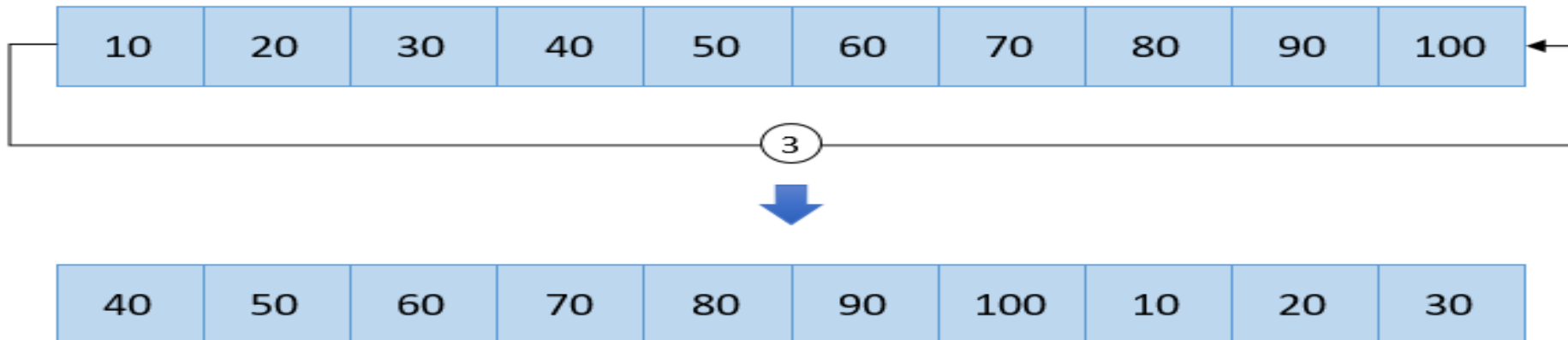
Input

Input 10 elements in array: 1 2 3 4 5 6 7 8 9 10

Input number of times to rotate: 3

Output

Array after left rotation 3 times: 4 5 6 7 8 9 10 1 2 3



Method: 1

1. Let **A** be an array to be rotated left by **d** positions.
2. Print **A** before rotation.
3. Define one temporary variable **temp** which will hold the first element of array **A**.
4. First copy the first element of array **A** into **temp** and then shift all elements one position left. After shifting, copy back **temp** into Array **A** as last element. Repeat this step **d** times.
5. Print the rotated array **A**.

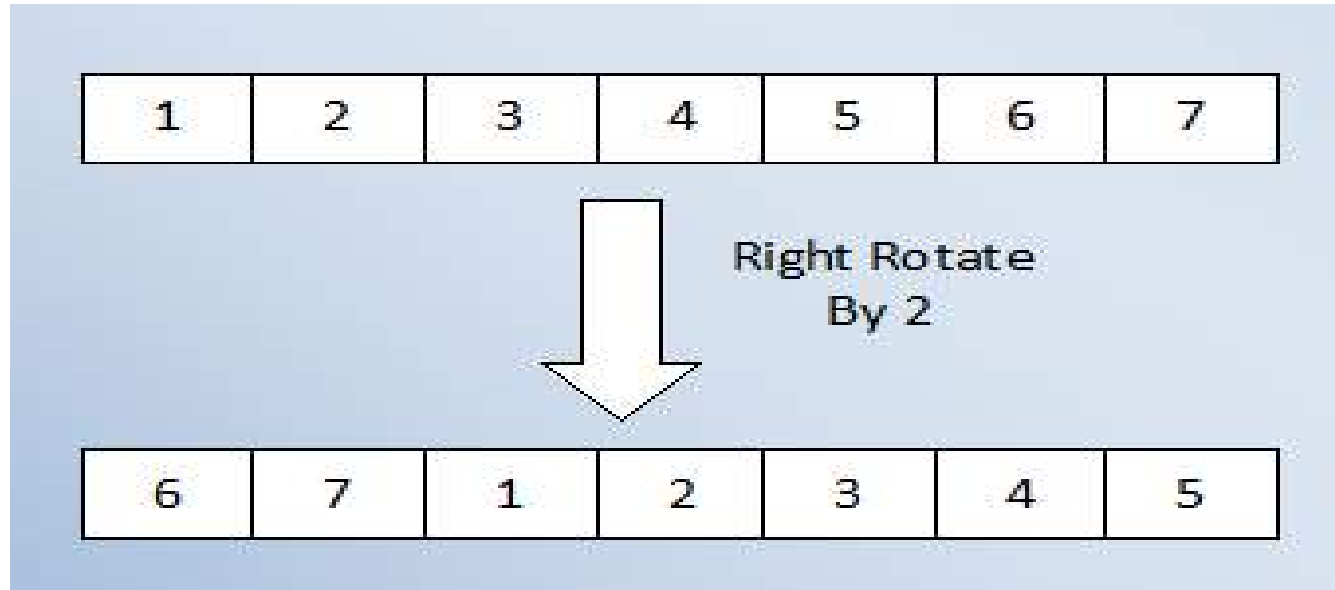
Method: 2 . Using Temporary Array [Assignment]

Input arr[] = [1, 2, 3, 4, 5, 6, 7], d = 2, n = 7

1. Store d elements in a temp array temp[] = [1, 2]
2. Shift rest of the arr[] arr[] = [3, 4, 5, 6, 7, 6, 7]
3. Store back the d elements arr[] = [3, 4, 5, 6, 7, 1, 2]

Rotate an array by d elements right:

Program: Write a C program to right rotate an array by d position. [Assignment]

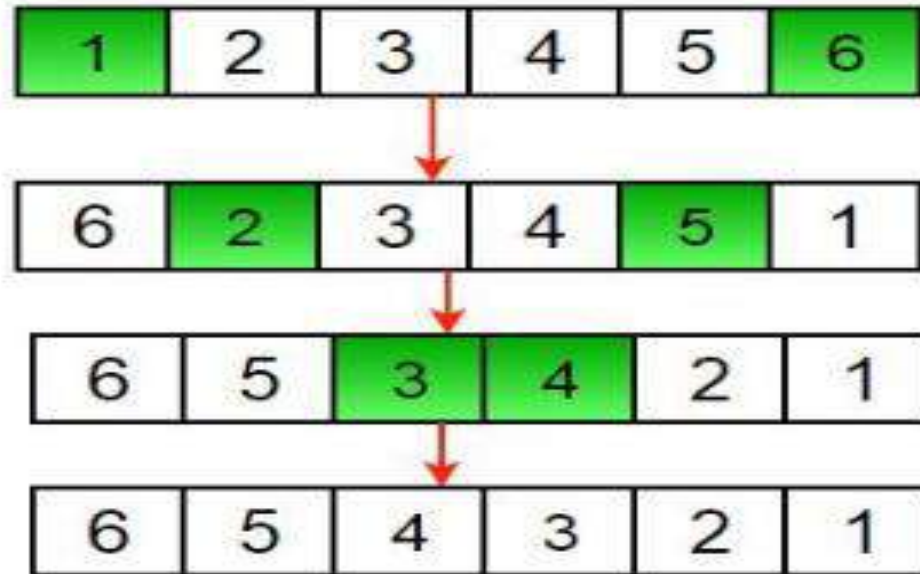


Reverse elements of an array:

Method

1. Let **A** be an array to be reversed with n number of elements.
2. Define two indexes i and j pointing first and last element of an array. [**i=0** , **j=n-1**]
3. Repeat the loop for **n/2** number of times.
 - a. In each iteration swap the array elements pointed by pointers i and j
 - b. After swapping modify indexes as **i=i+1** and **j=j-1**

Example:



Linked List

Linked List

- The term ***list*** refers to a linear collection of data items.
- One way to store such data is by means of ***Arrays and Linked List***.
- The ***Array*** implementation uses static structures where as ***Linked lists*** implementation uses dynamically allocated structures.

Array Implementation:

The array implementation of a list has certain drawbacks. These are:

1. Operations like insertion and deletion at a specified location in a List requires a lot of movement of data, therefore, leading to inefficient and time consuming algorithm.
2. Memory storage space is wasted; very often the List is much shorter than the array size declared.
3. List cannot grow in its size beyond the size of the declared array if required during program execution.

Linked List Implementation:

Following are the advantages of Linked Lists over Arrays:

1. Linked list is a dynamic structure where as array is a static structure.
2. Insertion and deletion of nodes requires no data movement.

Types of Linked List :

Following are the 4 types of linked lists:

1. Singly Linked List
2. Circular Linked List
3. Doubly Linked List
4. Circular Doubly Linked List

Singly Linked List:

- A **Singly Linked List** is a linear collection of data elements, called **nodes**.
- Here each node is divided into two fields. The first field contains the information of the element, and the second field contains the address of the next node in the list.
- The first field is called **Data field** and the second field is called **Next field**.
- Following figure shows the structure of a node in a linked list. The nodes in a linked list are called **self-referential** structures.

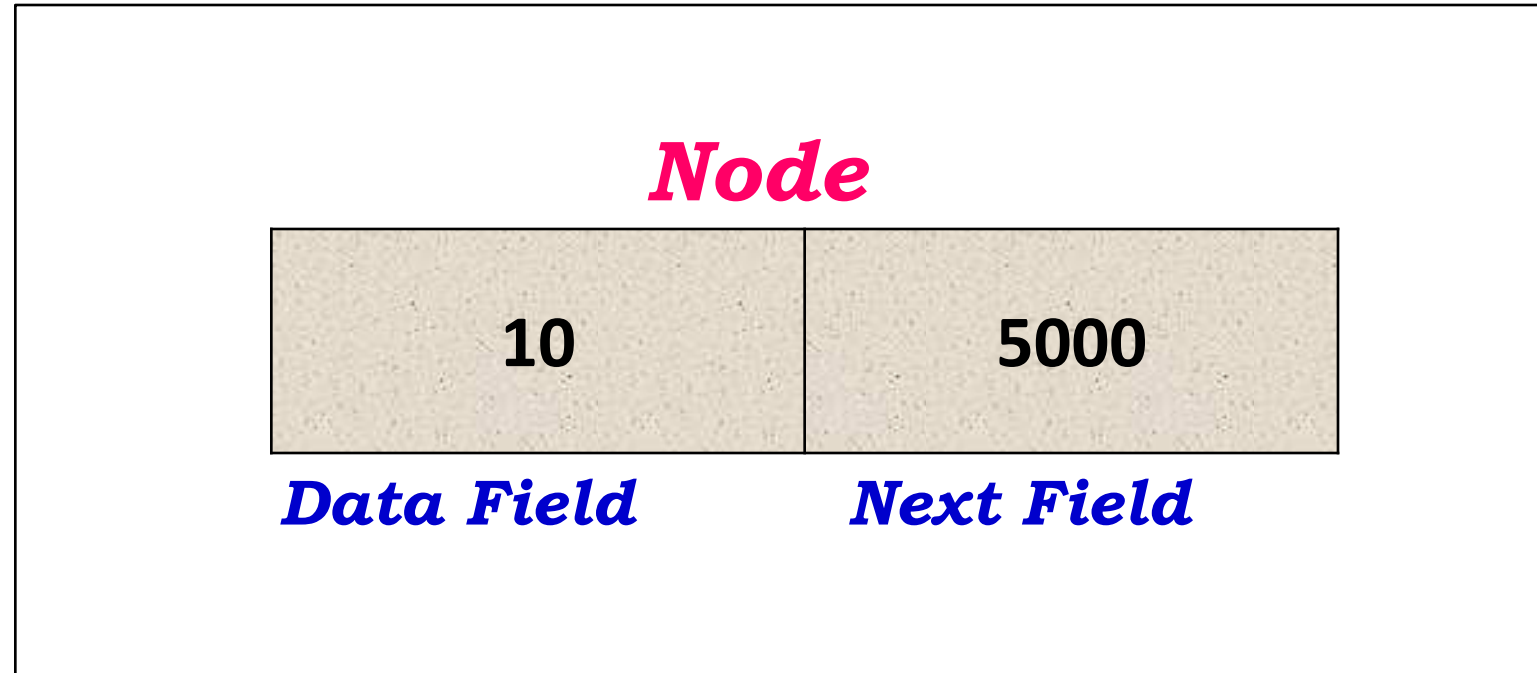
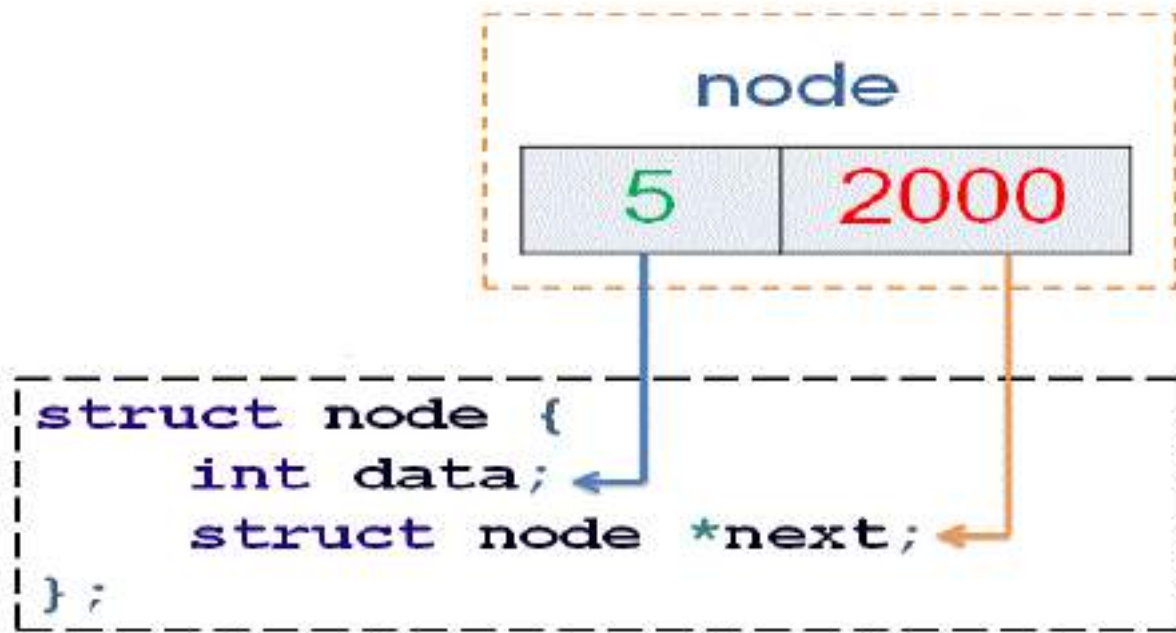


Figure: Structure of a node in a Linked List



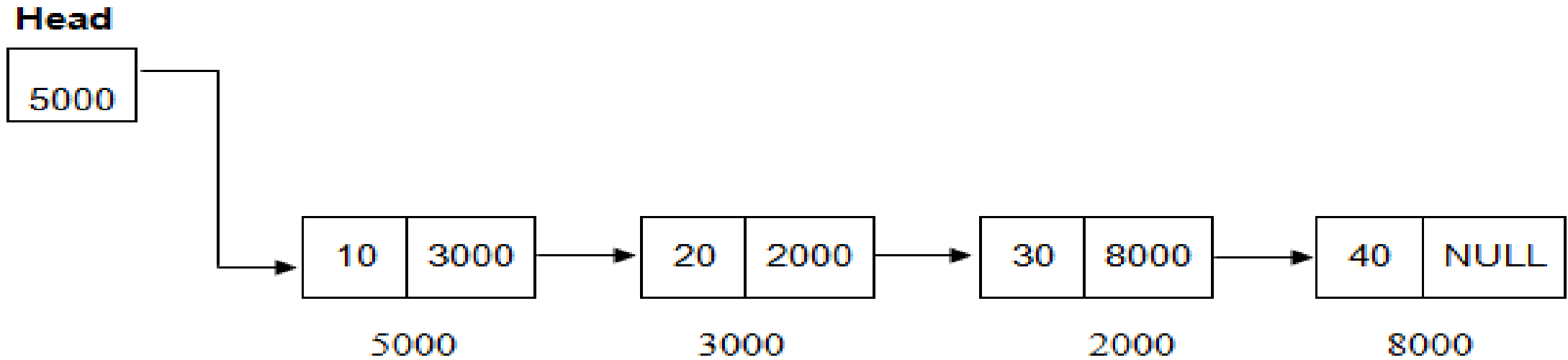
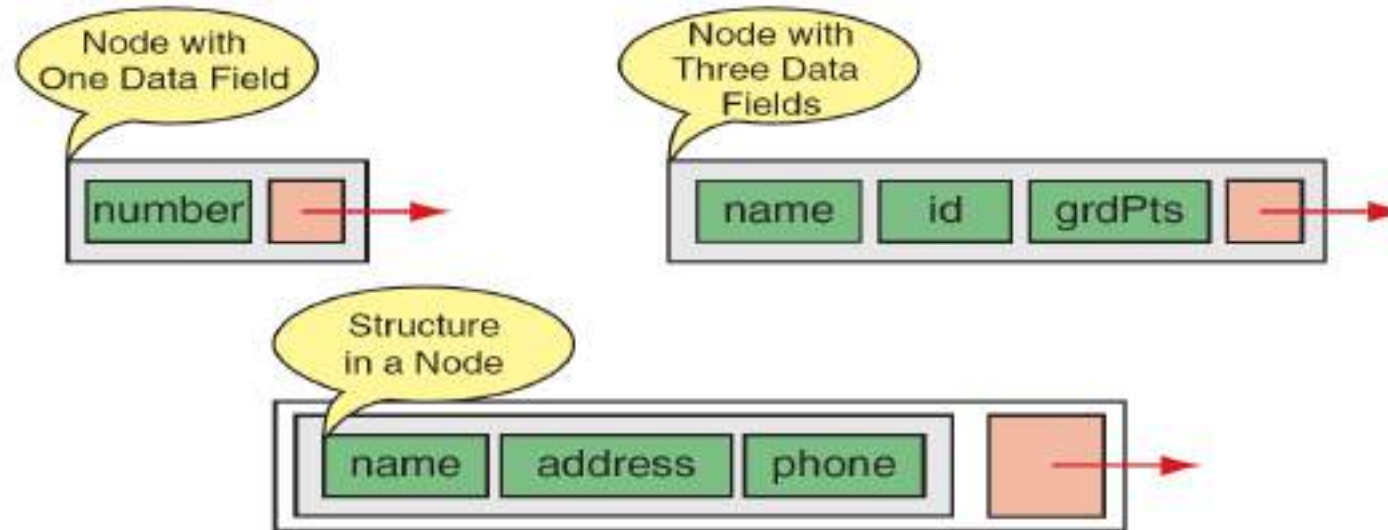


Figure: Singly Linked List with 4 Nodes

- There is an arrow from a node to the next node in the list.
- The pointer to the last node contains a special value called the ***null pointer***.
- Linked list also contains a pointer variable, called ***HEAD***, which points to the first node of the list.
- As nodes are added to a list, memory for a node is dynamically allocated. Thus the number of items that may be added to a list is limited only by the amount of memory available.
- If HEAD is NULL then list is said to be ***empty list*** or ***null list***.

- The data part in a node can be single field or multiple fields but the next part should be single field.



Operations on Linked List

- There are number of operations that can be performed on Linked list.
 - ☐ Traversal
 - ☐ Insertion
 - ☐ Deletion
 - ☐ Reversing list
 - ☐ Searching
 - ☐ Sorting

1. Insert a node:

- Insertion logic varies depending on where in the list the element is going to be inserted, first, middle or at the end. In all situations it is as simple as making the pointer point to different nodes and hence insertion at any place will be much faster.

a. Insert at Beginning

1. Create a new node
2. Make the **next** field of new node point to the node pointed by **head**
3. Make the **head** pointer point to new node

```
newnode -> next = head;  
head = newnode;
```


b. Insert in Middle

1. Create a new node.
2. We must point the new node to its successor by making the **next** field of new node point to the **next** of predecessor node.
3. Make the predecessor node point to new node.

***newnode -> next = temp -> next
temp -> next = newnode;***

c. Insert at End

1. Create a new node.
2. Make the ***next*** field of new node point to NULL.
3. Make the predecessor node point to new node.

```
temp -> next = newnode;  
newnode -> next = NULL;
```

2. Delete a Node:

- Again deletion logic varies depending on from where the node is getting deleted, beginning, middle or at the end.

a. Delete First Node

1. Store the node to be deleted in a temporary pointer
2. Make **head** point to the first nodes successor in the list
3. Delete the node pointed by temporary pointer.

```
temp = head;  
head = head -> next;  
free(temp);
```

b. Delete Middle Node

1. Store the node to be deleted in a temporary pointer.
2. Make the predecessor node point to the successor of the node being deleted.
3. Delete the node pointed by temporary pointer.

```
ptr -> next = temp -> next;  
free (temp);
```

c. Delete Last Node

1. Store the node to be deleted in a temporary pointer
2. Move the null pointer to the predecessor ***next*** field of last node.
3. Delete the node pointed by temporary pointer

```
temp = ptr -> next  
ptr -> next = NULL;  
free(temp);
```

SUMMARY

- Array is a fundamental data structure and many other data structure are implemented using this.

Implementing data structures such as stacks and queues

- Representing data in tables and matrices
- Creating dynamic data structures such as Hash Tables and Graph.
- When compared to other data structures, arrays have the advantages like random access (we can quickly access i-th item) and cache friendliness (all items are stored at contiguous location).

1. Identify the two types of efficiencies that are important for computer algorithms?

- a) Time efficiency and Higher power efficiency
- b) Higher power efficiency and Computational efficiency
- c) Computational efficiency and Space efficiency
- d) Space efficiency and Time Efficiency

2. Which one of the following asymptotic notation is used to provide lower bound constraint?

- a) X
- b) Ω
- c) Θ
- d) O

TERMINAL QUESTIONS

1. How is time complexity measured?
2. Which of the following best describes the useful criterion for comparing the efficiency of algorithms?
3. What does it mean to say that an algorithm X is asymptotically more efficient than Y?
4. Describe the various types of Asymptotic Notations with examples?

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

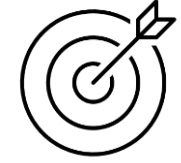
- 1 Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professional.

Sites and Web links:

1. <https://nptel.ac.in/courses/106102064>
2. <https://nptel.ac.in/courses/106101060/4>
3. <https://www.edx.org/course/algorithms-and-data-structures-1>
4. <https://in.udacity.com/course/intro-to-algorithms--cs215>
5. <https://www.coursera.org/learn/data-structures?action=enroll>
6. <https://www.geeksforgeeks.org/types-of-asymptotic-notations-in-complexity-analysis-of-algorithms/>

THANK YOU

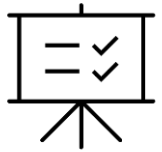
Advanced Algorithms & Data Structures



AIM OF THE SESSION

To familiarize students with the basic concept of Singly linked list

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate The Singly linked list
2. Describe the types of Singly linked list.
3. List out the operations possible on the Singly linked list.
4. Describe each operation
5. List out the advantages and applications of SLL.

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define Singly linked list
2. Describe two types of Singly linked list. And their operations
3. Summarize definition, types and operations of circular linked list and its applications

Linked List

- The term ***list*** refers to a linear collection of data items.
- One way to store such data is by means of ***Arrays and Linked List***.
- The ***Array*** implementation uses static structures where as ***Linked lists*** implementation uses dynamically allocated structures.

Array Implementation:

The array implementation of a list has certain drawbacks. These are:

1. Operations like insertion and deletion at a specified location in a List requires a lot of movement of data, therefore, leading to inefficient and time consuming algorithm.
2. Memory storage space is wasted; very often the List is much shorter than the array size declared.
3. List cannot grow in its size beyond the size of the declared array if required during program execution.

Linked List Implementation:

Following are the advantages of Linked Lists over Arrays:

1. Linked list is a dynamic structure where as array is a static structure.
2. Insertion and deletion of nodes requires no data movement.

Types of Linked List :

Following are the 4 types of linked lists:

1. Singly Linked List
2. Circular Linked List
3. Doubly Linked List
4. Circular Doubly Linked List

Singly Linked List:

- A **Singly Linked List** is a linear collection of data elements, called **nodes**.
- Here each node is divided into two fields. The first field contains the information of the element, and the second field contains the address of the next node in the list.
- The first field is called **Data field** and the second field is called **Next field**.
- Following figure shows the structure of a node in a linked list. The nodes in a linked list are called **self-referential** structures.

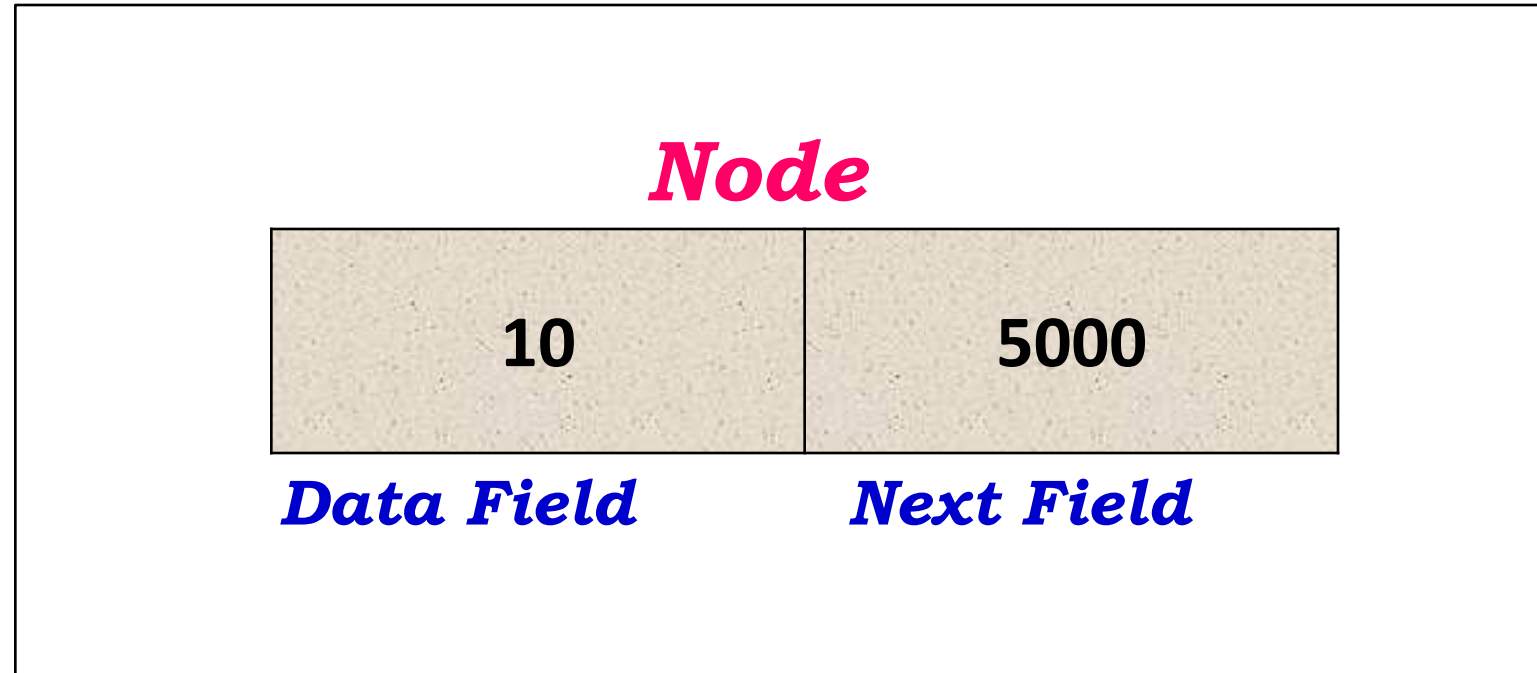
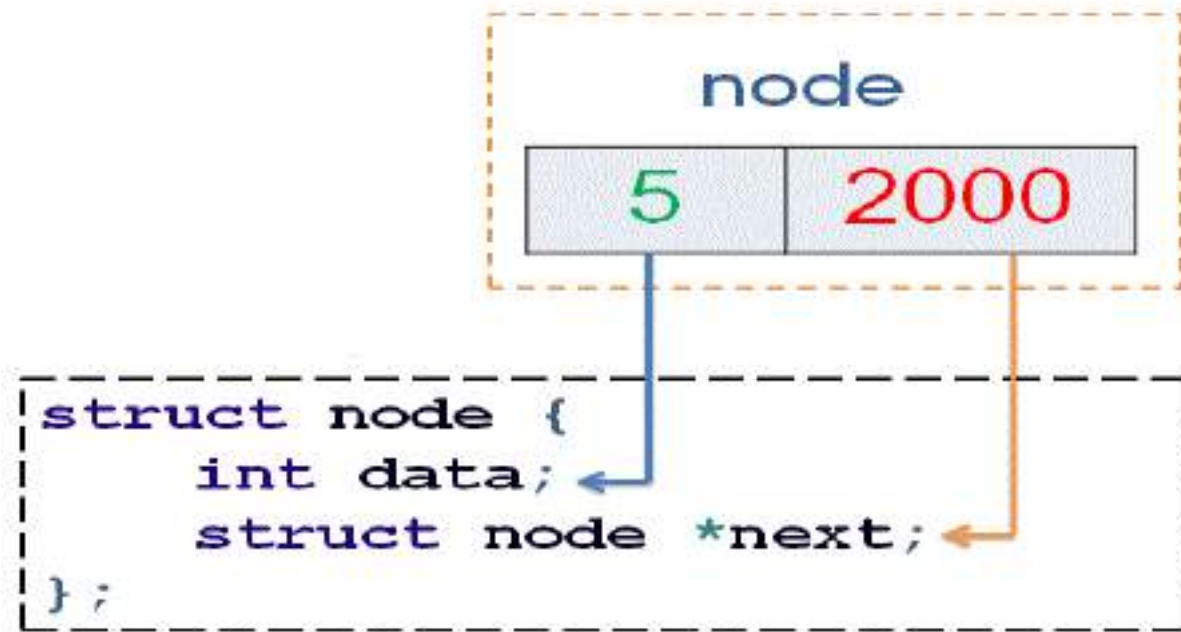


Figure: Structure of a node in a Linked List



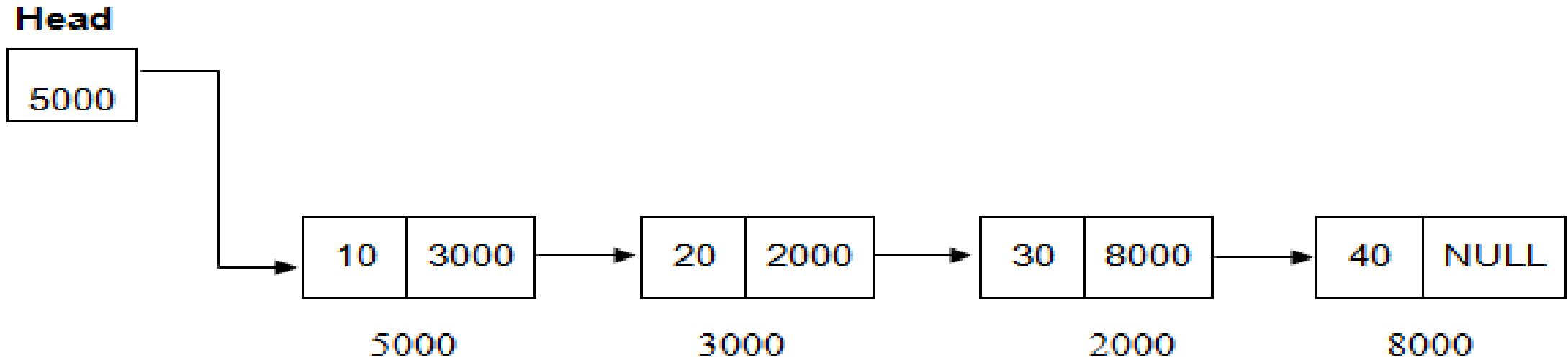
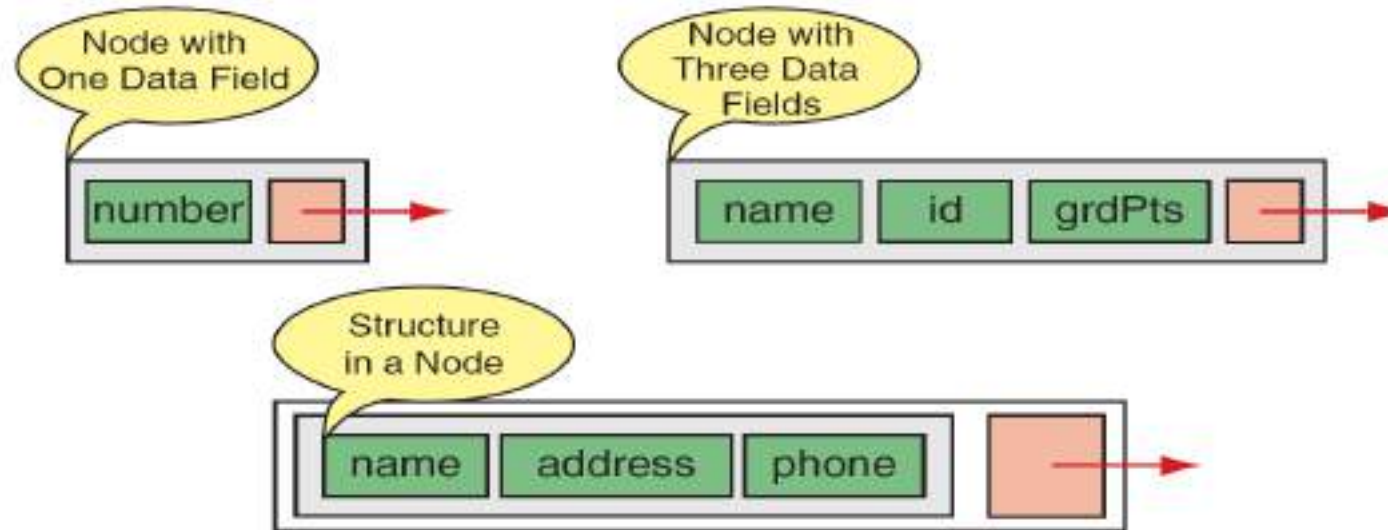


Figure: Singly Linked List with 4 Nodes

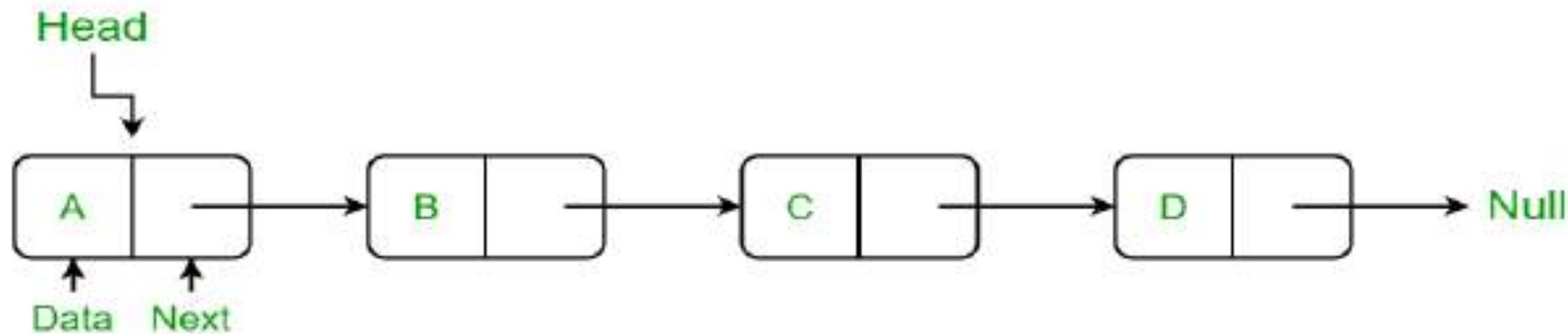
- There is an arrow from a node to the next node in the list.
- The pointer to the last node contains a special value called the ***null pointer***.
- Linked list also contains a pointer variable, called ***HEAD***, which points to the first node of the list.
- As nodes are added to a list, memory for a node is dynamically allocated. Thus the number of items that may be added to a list is limited only by the amount of memory available.
- If HEAD is NULL then list is said to be ***empty list*** or ***null list***.

- The data part in a node can be single field or multiple fields but the next part should be single field.



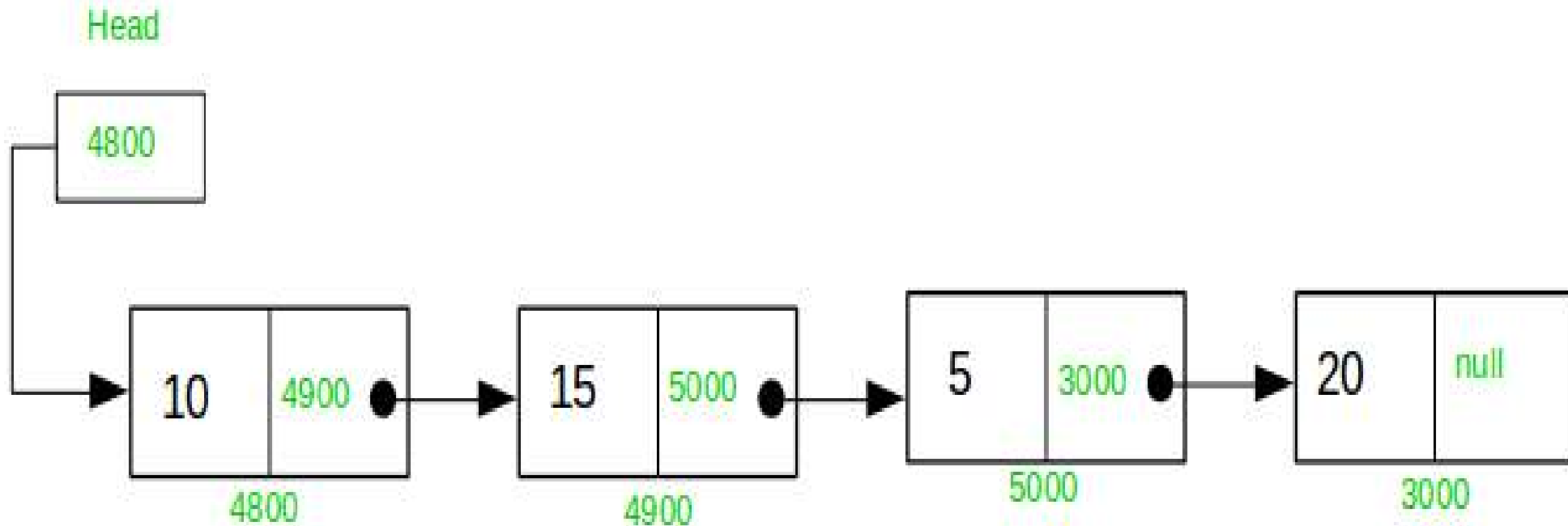
Singly Linked List INTRODUCTION

- A **singly linked list** is a linear data structure in which the elements are not stored in contiguous memory locations.
- Each element is connected only to its next element using a pointer.
- Every node points to its next node in the sequence and the last node points NULL.



Example of Singly Linked list with 4 nodes

Singly Linked list



OPERATIONS ON SINGLY LINKED LIST

In a singly linked list, we can perform the following operations...

1. Insertion
2. Deletion
3. Display

1. Insertion:

- i) Insertion At Beginning of the SLL
- ii) Insertion At End of the SLL
- iii) Insertion At Specific location in the SLL

2. Deletion:

- i) Deletion At Beginning of the SLL
- ii) Deletion At End of the SLL
- iii) Deletion At Specific location in the SLL

Setup the Singly Linked List

Before we implement those operations, first we need to **setup empty list**.

➤ **Perform the following steps to setup Empty SLL:.**

Step 1 - Define a **Node** structure with two members/fields **data** and **next**

Step 2 - Define a Node pointer '**head**' and set it to **NULL**.

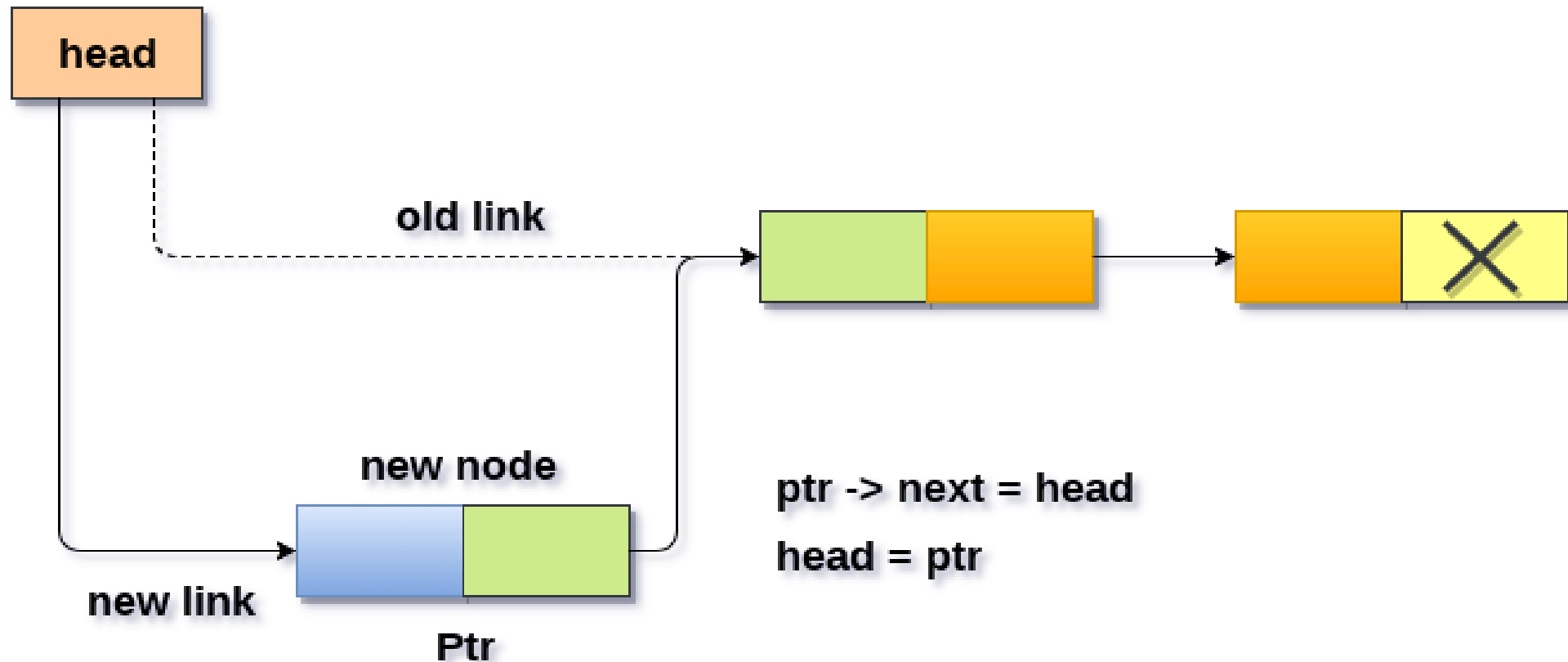
Step 3 – Then perform the Operations one by one.

The node is defined (represented) as:

```
struct Node
{
    int data;
    struct Node * next;
};
struct Node *head=NULL;
```

i) Insertion At Beginning of the SLL

It involves inserting any element at the front of the list. We just need to a few link adjustments to make the new node as the head of the list.



i) Insertion At Beginning of the SLL

We can insert a new node at beginning(before head node) of the Singly linked list.

Algorithm:

Step 1: IF PTR = NULL

Write OVERFLOW

Go to Step 7

[END OF IF]

Step 2: SET NEW_NODE = PTR

Step 3: SET PTR = PTR → NEXT

Step 4: SET NEW_NODE → DATA = VAL

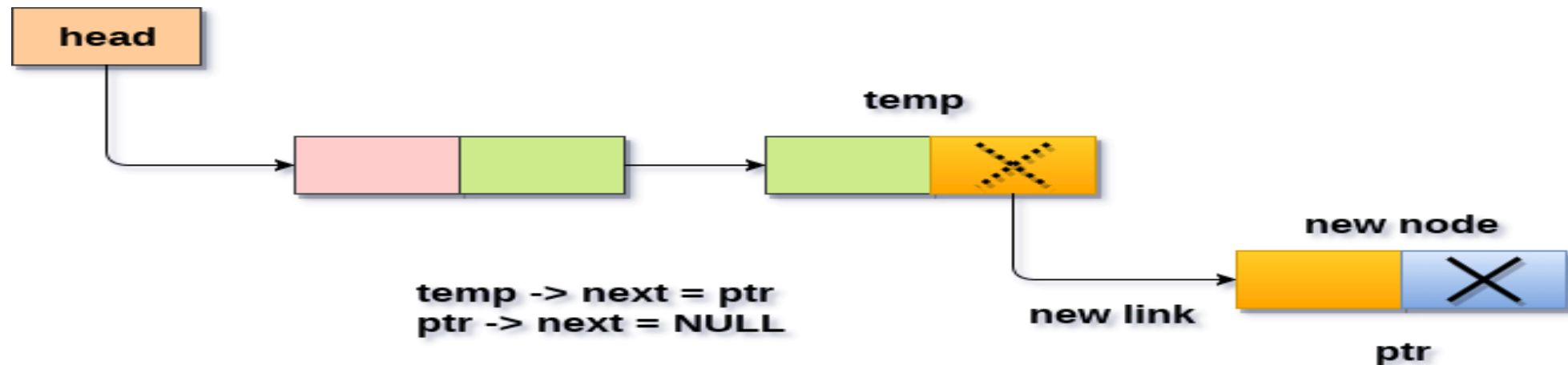
Step 5: SET NEW_NODE → NEXT = HEAD

Step 6: SET HEAD = NEW_NODE

Step 7: EXIT

ii) Insertion At End of the SLL

It involves insertion at the last of the linked list. The new node can be inserted as the only node in the list or it can be inserted as the last one. Different logics are implemented in each scenario.



Inserting node at the last into a non-empty list

ii) Insertion At End of the SLL

We can insert a new node at end(after tail/null node) of the singly linked list.

Algorithm:

Step 1: IF PTR = NULL Write OVERFLOW

Go to Step 1

[END OF IF]

Step 2: SET NEW_NODE = PTR

Step 3: SET PTR = PTR - > NEXT

Step 4: SET NEW_NODE - > DATA = VAL

Step 5: SET NEW_NODE - > NEXT = NULL

Step 6: SET PTR = HEAD

Step 7: Repeat Step 8 while PTR - > NEXT != NULL

Step 8: SET PTR = PTR - > NEXT

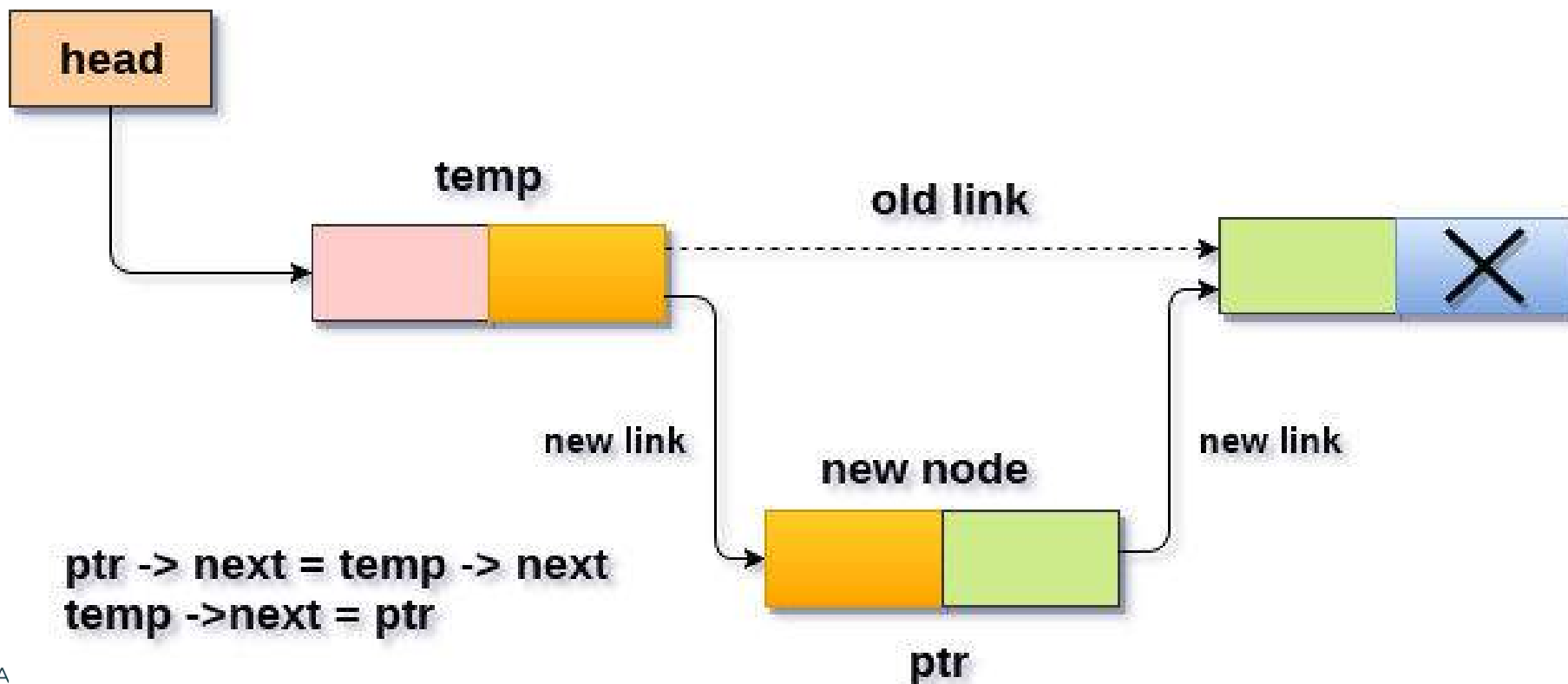
[END OF LOOP]

Step 9: SET PTR - > NEXT = NEW_NODE

Step 10: EXIT

iii) Insertion At Specific location in the SLL

It involves insertion after the specified node of the linked list. We need to skip the desired number of nodes in order to reach the node after which the new node will be inserted.



iii) Insertion At Specific location of the SLL

Algorithm:

STEP 1: IF PTR = NULL

WRITE OVERFLOW

GOTO STEP 12

END OF IF

STEP 2: SET NEW_NODE = PTR

STEP 3: NEW_NODE → DATA = VAL

STEP 4: SET TEMP = HEAD

STEP 5: SET I = 0

STEP 6: REPEAT STEP 5 AND 6 UNTIL I

STEP 7: TEMP = TEMP → NEXT

STEP 8: IF TEMP = NULL

WRITE "DESIRED NODE NOT PRESENT"

GOTO STEP 12

END OF IF

END OF LOOP

STEP 9: PTR → NEXT = TEMP → NEXT

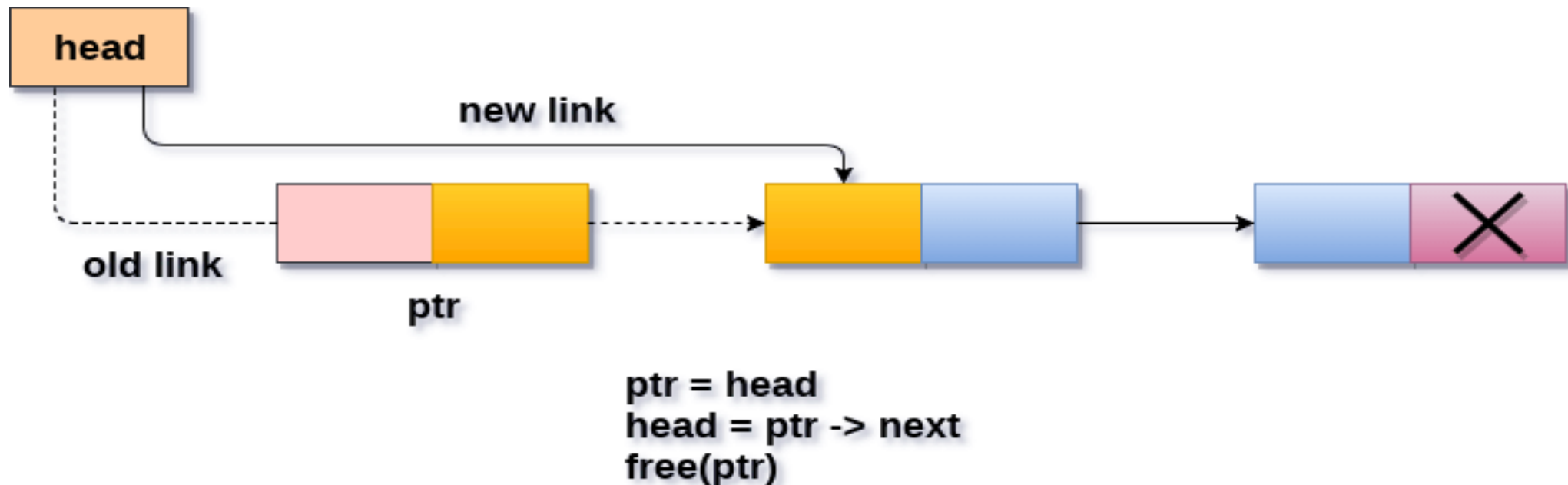
STEP 10: TEMP → NEXT = PTR

STEP 11: SET PTR = NEW_NODE

STEP 12: EXIT

i) Deletion At Beginning of the SLL

It involves deletion of a node from the beginning of the list. This is the simplest operation among all. It just need a few adjustments in the node pointers.



Deleting a node from the beginning

i) Deletion At Beginning of the SLL

We can delete a node from the beginning of the list.

Algorithm:

Step 1: IF HEAD = NULL

Write UNDERFLOW

Go to Step 5

[END OF IF]

Step 2: SET PTR = HEAD

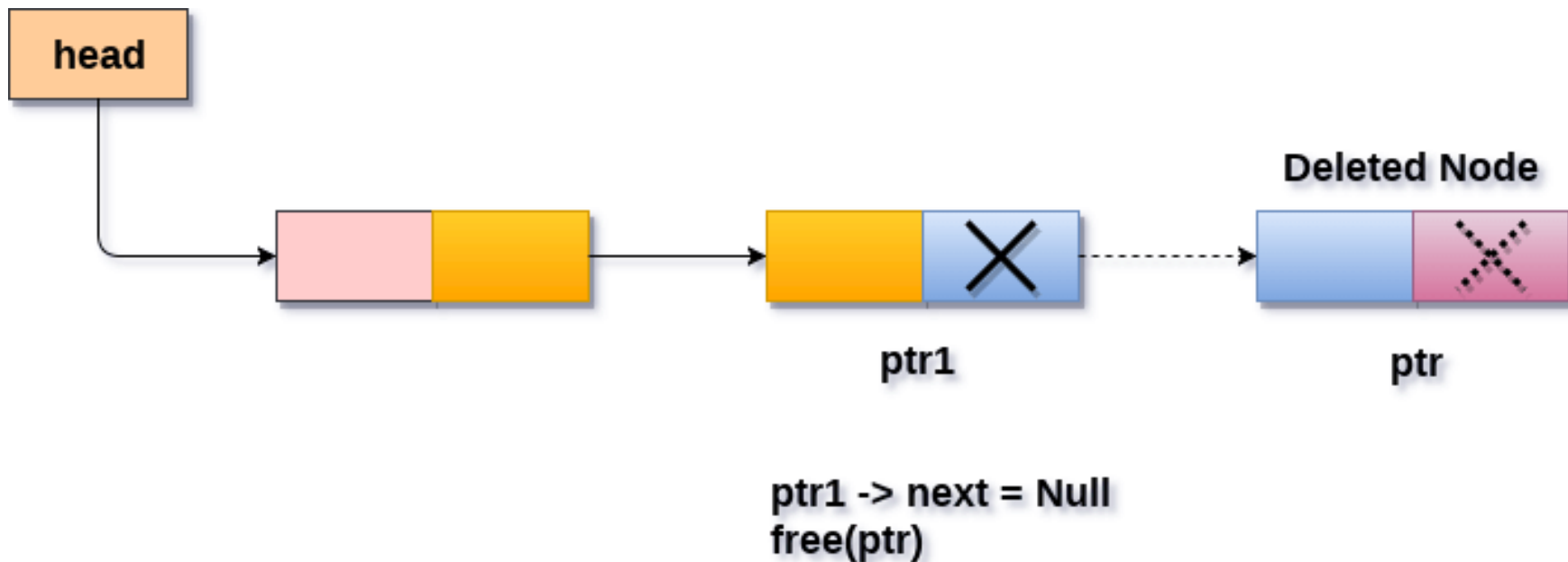
Step 3: SET HEAD = HEAD -> NEXT

Step 4: FREE PTR

Step 5: EXIT

ii) Deletion At End of the SLL

It involves deleting the last node of the list. The list can either be empty or full. Different logic is implemented for the different scenarios.



Deleting a node from the last

ii) Deletion At End of the SLL

We can delete a node at end of the singly linked list.

Algorithm:

Step 1: IF HEAD = NULL

Write UNDERFLOW

Go to Step 8

[END OF IF]

Step 2: SET PTR = HEAD

Step 3: Repeat Steps 4 and 5 while PTR -> NEXT != NULL

Step 4: SET PREPTR = PTR

Step 5: SET PTR = PTR -> NEXT

[END OF LOOP]

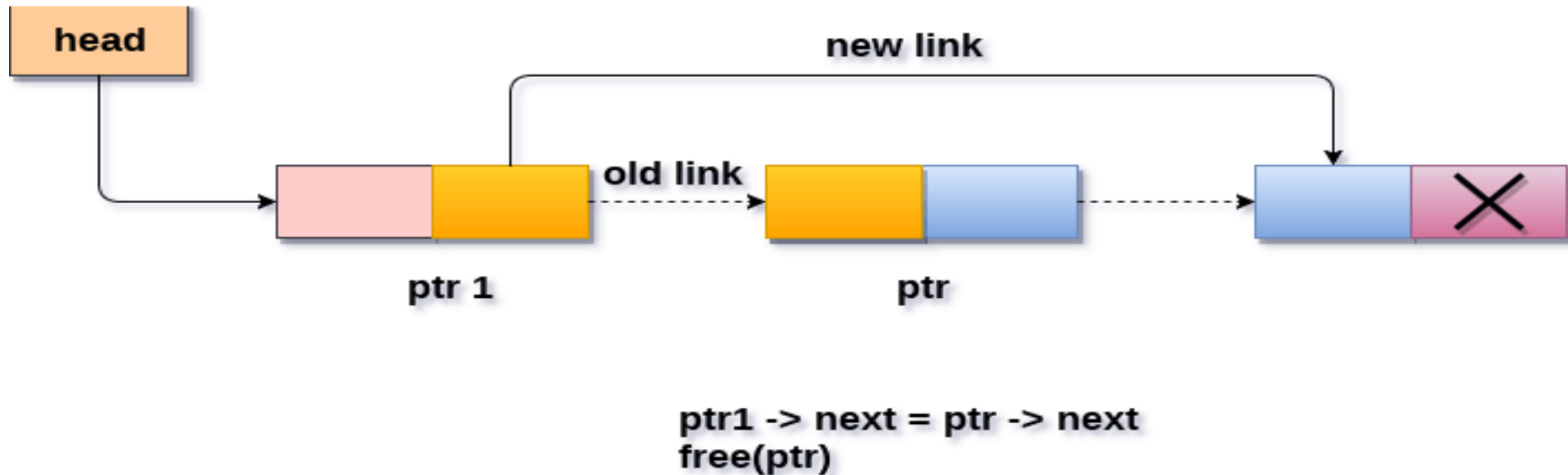
Step 6: SET PREPTR -> NEXT = NULL

Step 7: FREE PTR

Step 8: EXIT

iii) Deletion At Specific location in the SLL

It involves deleting the node after the specified node in the list. we need to skip the desired number of nodes to reach the node after which the node will be deleted. This requires traversing through the list.



Deletion a node from specified position

iii) Deletion At Specific location of the SLL

Algorithm:

STEP 1: IF HEAD = NULL

WRITE UNDERFLOW

GOTO STEP 10

END OF IF

STEP 2: SET TEMP = HEAD

STEP 3: SET I = 0

STEP 4: REPEAT STEP 5 TO 8 UNTIL I

STEP 5: TEMP1 = TEMP

STEP 6: TEMP = TEMP → NEXT

STEP 7: IF TEMP = NULL

WRITE "DESIRED NODE NOT PRESENT"

GOTO STEP 12

END OF IF

STEP 8: I = I+1

END OF LOOP

STEP 9: TEMP1 → NEXT = TEMP → NEXT

STEP 10: FREE TEMP

STEP 11: EXIT

Display Operation in Singly Linked List

We can use the following steps to display the elements of a single linked list

Algorithm:

Step 1 - Check whether list is Empty ($\text{head} == \text{NULL}$)

Step 2 - If it is Empty then, display 'List is Empty!!!' and terminate the function.

Step 3 - If it is Not Empty then, define a Node pointer 'temp' and initialize with head.

Step 4 - Keep displaying temp $\rightarrow$ data with an arrow ($\text{---}\rightarrow$) until temp reaches to the last node

Step 5 - Finally display temp $\rightarrow$ data with arrow pointing to NULL (temp $\rightarrow$ data $\text{---}\rightarrow$ NULL).

SUMMARY

Singly linked list is a basic linked list type. Singly linked list is a collection of nodes linked together in a sequential way where each node of singly linked list contains a data field and an address field which contains the reference of the next node. Singly linked list can contain multiple data fields but should contain at least single address field pointing to its connected next node

SELF-ASSESSMENT QUESTIONS

A linear collection of data elements where the linear node is given by means of pointer is called?

- a) **Linked list**
- b) Node list
- c) Primitive list
- d) Unordered list

In linked list each node contains a minimum of two fields. One field is data field to store the data second field is??

- a) Pointer to character
- b) Pointer to integer
- c) **Pointer to node**
- d) Node

TERMINAL QUESTIONS

- 1.How will you represent a linked list in a graphical view?
- 2.Explain Singly Linked List in short.
- 3.How many pointers are necessary to implement a simple Linked List?
- 4.Which type of memory allocation is referred for Linked List
- 5.What are the main differences between the Linked List and Linear Array?

Reference Books:

1. Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2010 , Second Edition, Pearson Education.
2. 2. Ellis Horowitz, Fundamentals of Data Structures in C: Second Edition, 2015
3. A.V.Aho, J. E. Hopcroft, and J. D. Ullman, “Data Structures And Algorithms”, Pearson Education, First Edition
Reprint 2003.

Sites and Web links:

1. <https://nptel.ac.in/courses/106102064>
2. <https://in.udacity.com/course/intro-to-algorithms--cs215>
3. <https://www.coursera.org/learn/data-structures?action=enroll>

THANK YOU

Advanced Algorithms & Data Structures

AIM OF THE SESSION

To familiarize students with the basic concept of stack operations using Linked Lists

INSTRUCTIONAL OBJECTIVES

This Session is designed to:

1. Demonstrate how to implement a stack using a linked list.
2. Discuss the advantages of using a linked list for a stack, such as dynamic memory allocation.
3. Describe the fundamental operations of a stack (push, pop, and peek).
4. Analyze the time complexity of stack operations in a linked list implementation..

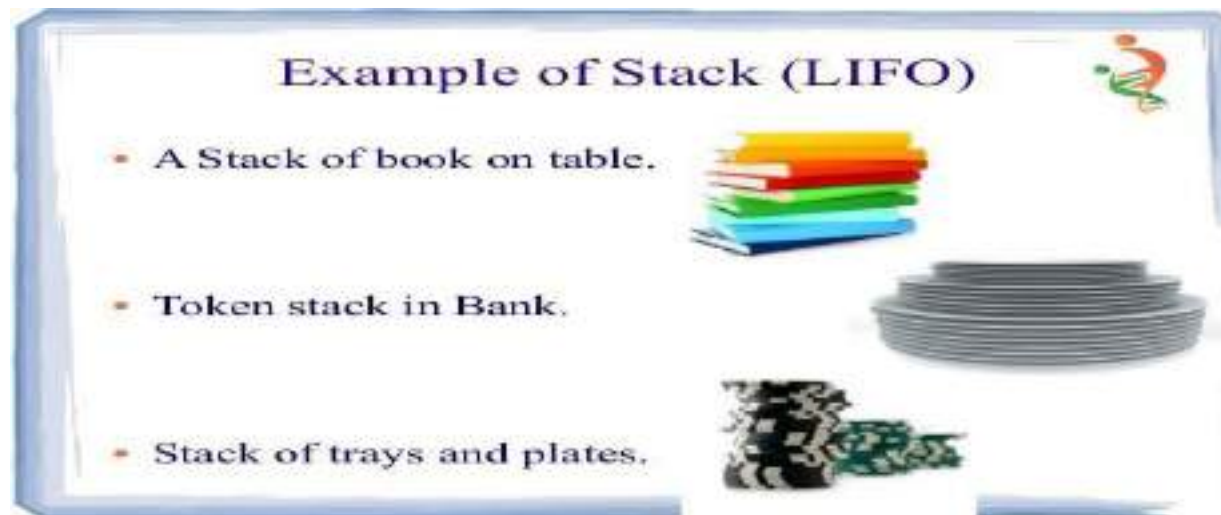
LEARNING OUTCOMES

At the end of this session, you should be able to:

- Evaluate the time complexities of the different Algorithmic solutions for various types of Problems.
- Write the notations for calculating the running time complexity of different algorithms

Session -Introduction

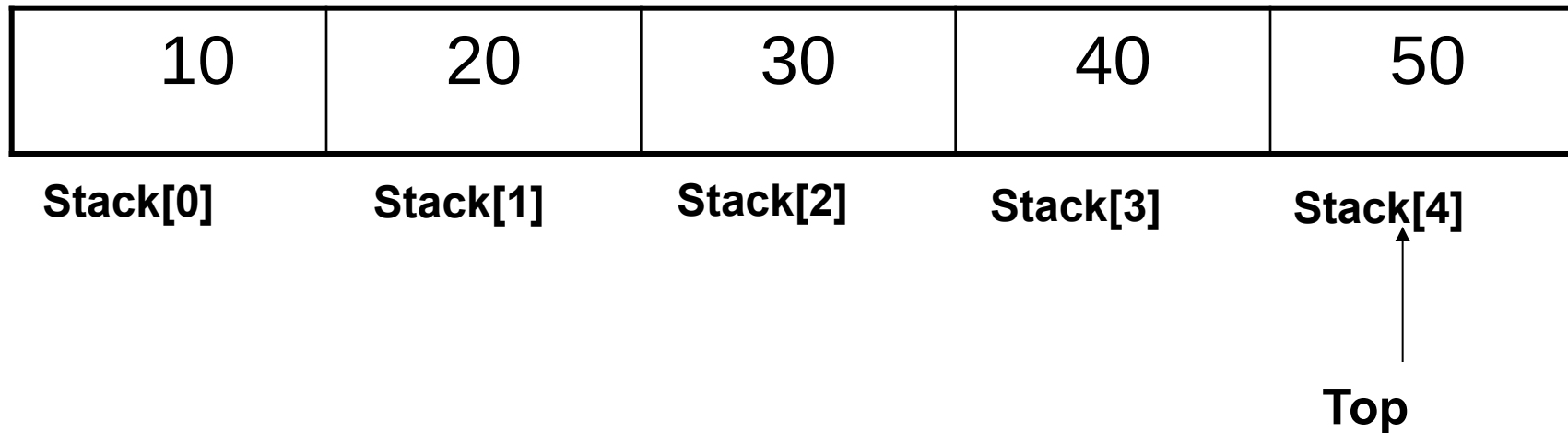
➤ Stack is a linear data structure which implements LIFO (Last In First Out) pattern where insertions and deletions are done at only one side i.e., top of the stack.

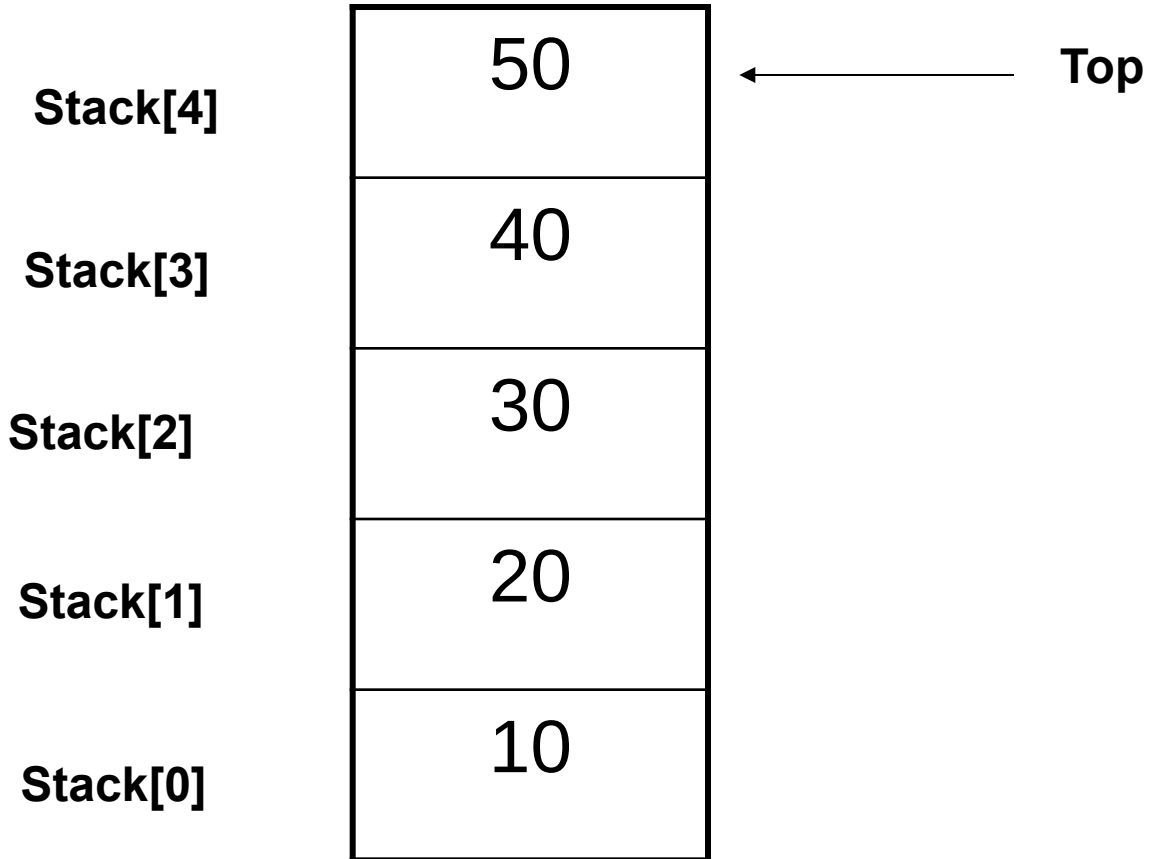


stack

- A ***stack*** is a linear list in which items may be added or removed only at one end, called the top of the stack.
- The last item to be added to a stack is the first item to be removed. Accordingly, stacks are also called ***Last-In-First-Out (LIFO)*** lists or ***First-In-Last-Out (FILO)*** lists.

- A stack may be pictorially represented as follows:





- The basic operations associated with stacks are:
 - **“Push”** is the term used to insert an element into a stack.
 - **“Pop”** is the term used to delete an element from a stack.
 - **“Peek”** is an operation that returns the value of the topmost element of the stack.

Examples:

1. A stack of dishes
2. A stack of coins
3. A stack of folded towels
4. A stack of bills
5. A railway system for shunting cars

- The stack data structure can be implemented in two different following ways:
 - Stack Implementation using Arrays [Static Implementation]
 - Stack Implementation using Linked List [Dynamic Implementation]

Array representation of a stack:

- Stack may be represented in a computer in various ways, usually by means of a *one-dimensional array*.
- **Top** is a variable, which points top most element of the stack.
- Initially when the stack is empty, **Top** has a value of “**-1**” and when the stack contains a single element, **Top** has a value of “**0**” and when the stack contains two elements, **Top** has a value of “**1**” and so on.

- Each time a new element is inserted in the stack, the top is incremented by “*one*” before the element is placed on the stack.

[Top=Top+1;]

- The **Top** is decremented by “*one*” each time a deletion is made from the stack.

[Top = Top – 1;]

- A variable **MaxSize**, which gives the maximum number of elements that can be held by the stack.

- Following is the condition is used to check whether the Stack is Full or not:

$$(\mathbf{Top == MaxSize-1})$$

- Following is the condition is used to check whether the Stack is Empty or not:

$$(\mathbf{Top == -1})$$

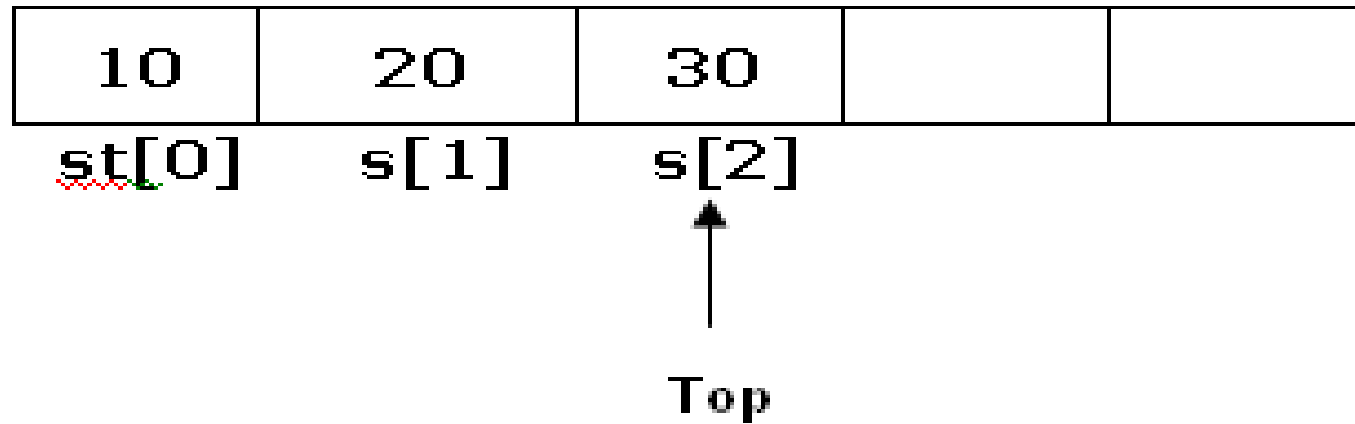


Fig: Array representation of stack.

- The right most occupied element of the stack represents its top element. Since **Top=2**, the stack has three elements 10, 20, 30 and there is room for two elements in the stack.

Algorithm for push:

Step.1: [Check whether the stack is full]

If $\text{Top} \geq \text{Max}-1$ then

Print 'Stack Overflow'

Return

Step.2: [Increment pointer Top]

$\text{Top} \leftarrow \text{Top} + 1.$

Step.3: [Insert ITEM at top of the stack]

$\text{Stack}[\text{top}] \leftarrow \text{ITEM}$

Step.4: Return.

Algorithm for pop:

Step.1: [Checking whether the stack is empty]

If Top = -1 **then**

 Print 'Stack Underflow'

 Return

Step.2: [retrieving the topmost element from stack into a temporary variable K]

$k \leftarrow \text{Stack}[\text{Top}]$

Step.3: [Decrement the pointer Top]

$\text{Top} \leftarrow \text{Top} - 1$

Step.4: return(k).

Implementation of *push* operations:

```
Void push()
{
    int ele;
    if(Top == (MaxSize-1))
    {
        printf("Stack Is Overflow \n");
    }
    else
    {
        printf("Enter the Element to be Insert : ");
        scanf("%d",&ele);
        Top=Top+1;
        Stack[Top] = ele;
    }
}
```


Implementation of *pop* operations:

```
void Deletion()  
{  
    if(Top== -1)  
    {  
        printf("Stack Is Underflow \n");  
    }  
    else  
    {  
        printf("The Deleted Element is : %d \n", Stack[Top]);  
        Top = Top - 1;  
    }  
}
```

Stack Applications:

1. A stack is useful to convert infix expression into postfix expression.
2. A stack is useful to evaluate the value of postfix expression.
3. It can be used in function calls.
4. Compiler application uses the stack in parenthesis matching.
5. A stack can also be used in string processing to evaluate reverse of a string.
6. A stack is useful in writing recursive calls
7. Redo-undo features at many places like editors, photoshop.
8. Used in many algorithms like Towers of Hanoi, Tree Traversals, Histogram problem.
9. *Backtracking (game playing, finding paths, exhaustive searching).*

Stack using an array - drawback

- If we implement the stack using an array, we need to specify the array size at the beginning(at compile time).
- We can't change the size of an array at runtime. So, it will only work for a fixed number of elements.

Solution:

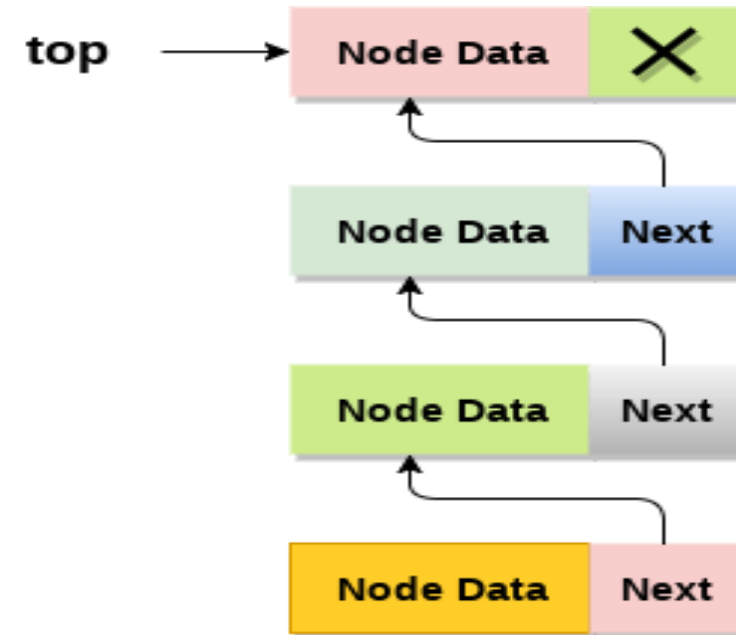
- We can implement the stack using the linked list.
- In the linked list, we can change its size at runtime.

Operations on Stack

- **push():** It inserts an element to the top of the stack. It takes $O(1)$ time, as each node is inserted at the head/top of the linked list.
- **pop():** It removes an element from the top of the stack. It takes $O(1)$ time, as the top always points to the newly inserted node.
- **Display():** It displays the elements of the stack.

Node structure

```
struct node
{
    int data;
    struct node * next;
};
node *top=NULL;
```



Stack

PUSH OPERATION ON STACK

ALGORITHM:

Step 1 - Create a **newNode** with given value.

Step 2 - Check whether stack is **Empty** (**top == NULL**)

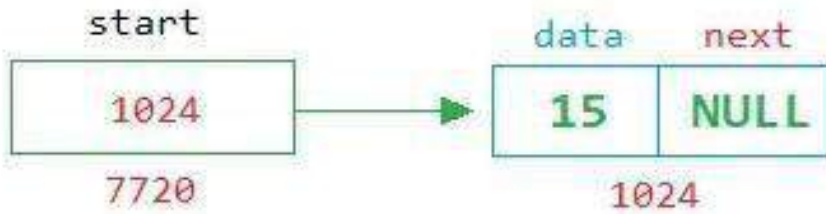
Step 3 - If it is **Empty**, then set **newNode** $\rightarrow$ **next = NULL**.

Step 4 - If it is **Not Empty**, then set **newNode** $\rightarrow$ **next = top**.

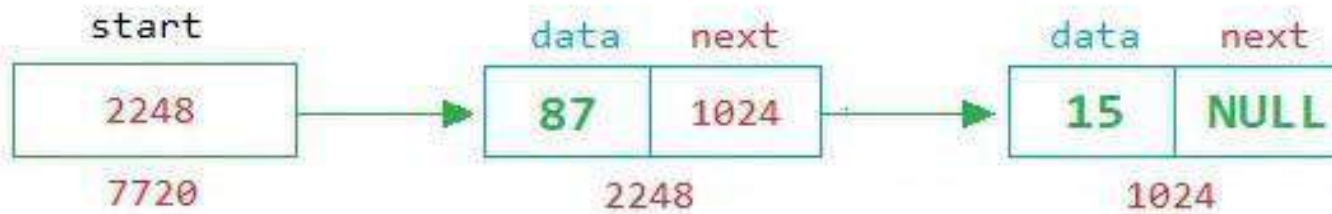
Step 5 - Finally, set **top = newNode**.



Push 15



Push 87



POP OPERATION ON STACK

ALGORITHM:

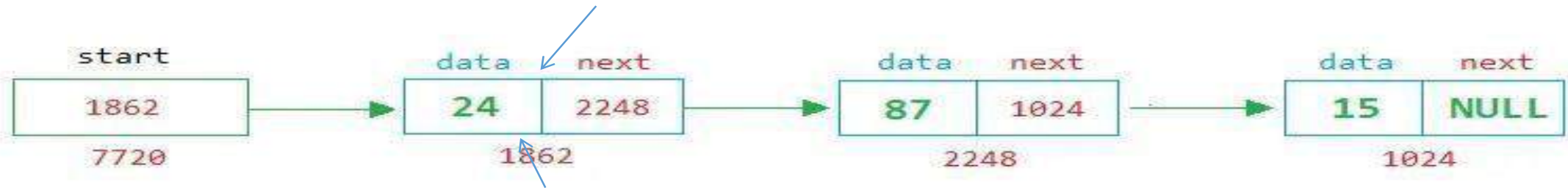
Step 1 - Check whether **stack** is **Empty** (**top == NULL**).

Step 2 - If it is **Empty**, then display "**Stack is Empty!!! Deletion is not possible!!!**" and terminate the function

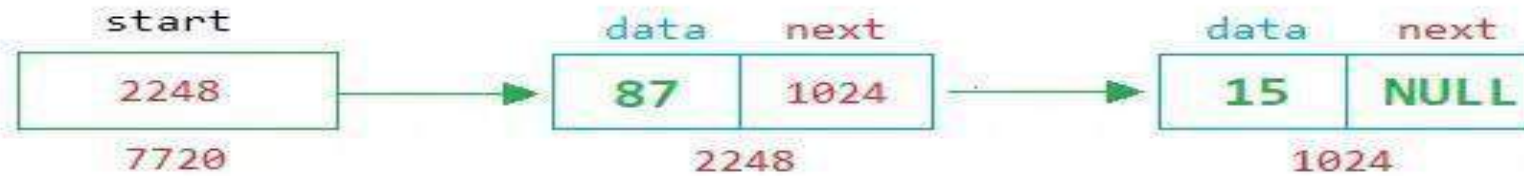
Step 3 - If it is **Not Empty**, then define a **Node** pointer '**temp**' and set it to '**top**'.

Step 4 - Then set '**top = top → next**'.

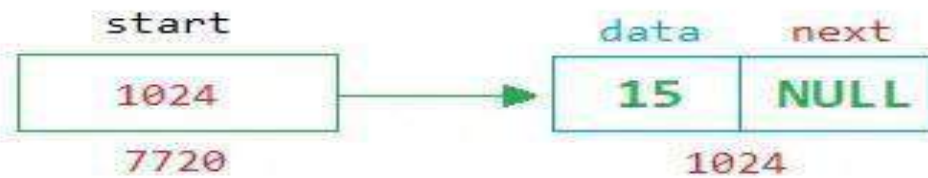
Step 5 - Finally, delete '**temp**'. (**free(temp)**).



Pop (Remove 24)



Pop (Remove 87)



Display

Algorithm:

Step 1 - Check whether stack is **Empty** (**top == NULL**).

Step 2 - If it is **Empty**, then display '**Stack is Empty!!!**' and terminate the function.

Step 3 - If it is **Not Empty**, then define a Node pointer '**temp**' and initialize with **top**.

Step 4 - Display '**temp → data --->**' and move it to the next node. Repeat the same until **temp** reaches to the first node in the stack. (**temp → next != NULL**).

Step 5 - Finally! Display '**temp → data ---> NULL**'.

SUMMARY

stack Implementation, a stack contains a top pointer. which is the “head” of the stack where pushing and popping items happens at the head of the list. The first node has a null in the link field and second node-link has the first node address in the link field and so on and the last node address is in the “top” pointer.

The main advantage of using a linked list over arrays is that it is possible to implement a stack that can shrink or grow as much as needed. Using an array will put a restriction on the maximum capacity of the array which can lead to stack overflow. Here each new node will be dynamically allocated. so overflow is not possible

SELF-ASSESSMENT QUESTIONS

1. What is the best case time complexity of deleting a node in a Singly Linked list?

- a) $O(n)$
- b) $O(n^2)$
- c) $O(n \log n)$
- d) $O(1)$**

2 What does 'stack overflow' refer to?

- a) accessing item from an undefined stack
- b) adding items to a full stack**
- c) removing items from an empty stack
- d) index out of bounds exception

TERMINAL QUESTIONS

- 1. Compare Array vs Linked list stack implementations.**
- 2. Implement push operation on stack using linked list.**
- 3. Implement pop operation on stack using linked list.**
- 4. Analyze the time complexity of push and pop operation on stack using linked list.**

Reference Books:

1. Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2010 , Second Edition, Pearson Education.
2. Ellis Horowitz, Fundamentals of Data Structures in C: Second Edition, 2015
3. A.V.Aho, J. E. Hopcroft, and J. D. Ullman, “Data Structures And Algorithms”, Pearson Education, First Edition Reprint 2003.

Sites and Web links:

1. <https://nptel.ac.in/courses/106102064>
2. <https://in.udacity.com/course/intro-to-algorithms--cs215>
3. <https://www.coursera.org/learn/data-structures?action=enroll>

THANK YOU

Advanced Algorithms & Data Structures

AIM OF THE SESSION

To familiarize students with the basic concept of Queue Using Singly Linked List

INSTRUCTIONAL OBJECTIVES

This Session is designed to:

1. Demonstrate the Queue Using Singly Linked List
2. List out the operations possible on the Queue Using Singly Linked List.
3. Describe the each operation
4. List out the advantages and applications of Queue Using Singly Linked List.

LEARNING OUTCOMES

At the end of this session, you should be able to:

1. Define Queue Using Singly Linked List
2. Describe the insertion and deletion operations of Queue Using Singly Linked List
3. Summarize definition and operations of Queue Using Singly Linked List

Queue INTRODUCTION

- A queue is an ordered list in which all insertions take place at one end, the rear, while all deletions take place at the other end, the front.
- The most common occurrence of a queue in computer applications is for the scheduling of jobs.
- In batch processing the jobs are "queued-up" as they are read-in and executed, one after another in the order they were received.

Queue INTRODUCTION Contd....

- When we talk of queues we talk about two distinct ends: the front and the rear.
- Additions to the queue take place at the rear. Deletions are made from the front.
- So, if a job is submitted for execution, it joins at the rear of the job queue.
- The job at the front of the queue is the next one to be executed.

Types of Queue

- Simple Queue or Linear Queue
- Circular Queue
- Priority Queue
- Double Ended Queue (or Deque)

Operations performed on queue

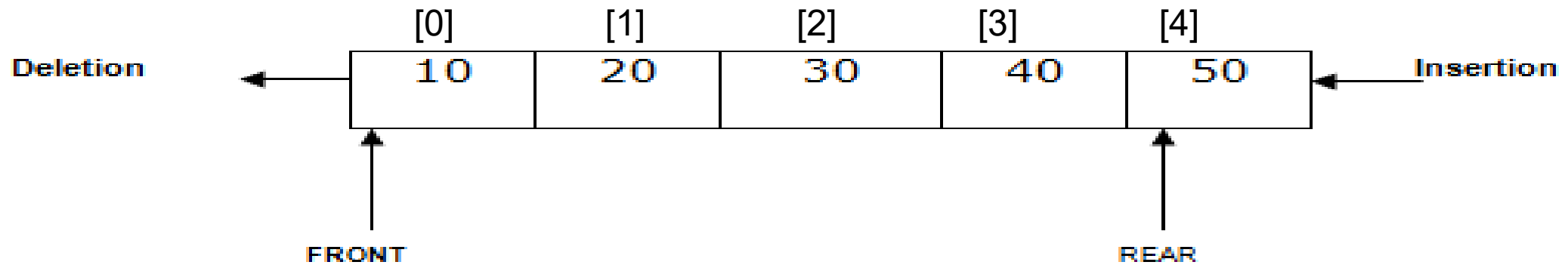
The fundamental operations that can be performed on queue are listed as follows -

- **Enqueue:** The Enqueue operation is used to insert the element at the rear end of the queue. It returns void.
- **Dequeue:** It performs the deletion from the front-end of the queue. It also returns the element which has been removed from the front-end. It returns an integer value.
- **Peek:** This is the third operation that returns the element, which is pointed by the front pointer in the queue but does not delete it.
- **Queue overflow (isfull):** It shows the overflow condition when the queue is completely full.
- **Queue underflow (isempty):** It shows the underflow condition when the Queue is empty, i.e., no elements are in the Queue.

Linear Queue:

- A **queue** is an ordered list in which items may be added only at one end called the **rear** and items may be removed only at the other end called **front**.
- **Examples:**
 1. A line of passengers waiting to buy tickets in a reservation counter. Each new passenger gets in line at the rear; the passenger at the front of the line is served.
 2. A queue can be found in a time-sharing computer system where many users share the system simultaneously.
 3. A line of cars waiting to proceed in some fixed direction at an intersection of streets.

- The first element, which is added into the queue, will be the first one to be removed. Thus queues are also called ***First-In-Fist-Out lists (FIFO)*** or ***Last-In-Last-Out lists (LILO)***.



Array Representation of a Queue:

- Queues may be represented in the computer memory by means of linear arrays. There are two pointer variables namely FRONT and REAR, FRONT denotes the location of the first element of the queue. The REAR describes the location of the rear element of the queue.
- **FRONT**==**-1** will indicate that the queue is empty. Whenever an item is added to the queue, the value of **REAR** is increased by **1**, that is **REAR=REAR+1**, whenever an 'item' is deleted
- from the queue, the value of **FRONT** is increment by **1**. That is **FRONT=FRONT+1**. **FRONT==REAR** if there are no elements in the queue. Note that the **FRONT** of the queue will be the first location of the array.

Algorithm: Insertion into a Queue

Step.1:[Check whether Queue is full]

If $\text{Rear} \geq \text{Max}-1$ then

Print 'Queue Overflow'

Return

Step.2:[Increment the Rear index]

$\text{Rear} \leftarrow \text{Rear} + 1$

Step.3:[Insert the new ITEM at rear end of queue */

$Q[\text{Rear}] \leftarrow \text{ITEM}$

Step.4:[if initially the queue is empty, then adjust Front index]

If $\text{Front} = -1$ then

$\text{Front} = 0$

Step.5: return.

Algorithm: Deletion from a Queue

Step.1: [Check whether the queue is empty]

 If Front=-1 then

 Print 'Queue Underflow'

 Return

Step.2: [Delete the queue element at front end and store into a temporary variable]

$K \leftarrow Q[\text{Front}]$

Step.3: [if queue is empty after deletion, then set front and rear indexes to -1, else adjust the Front index]

 If Front = Rear then

 Begin

 Front $\leftarrow$ -1

 Rear $\leftarrow$ -1

 End

 Else

 Front $\leftarrow$ Front + 1

Step.4: return(k).

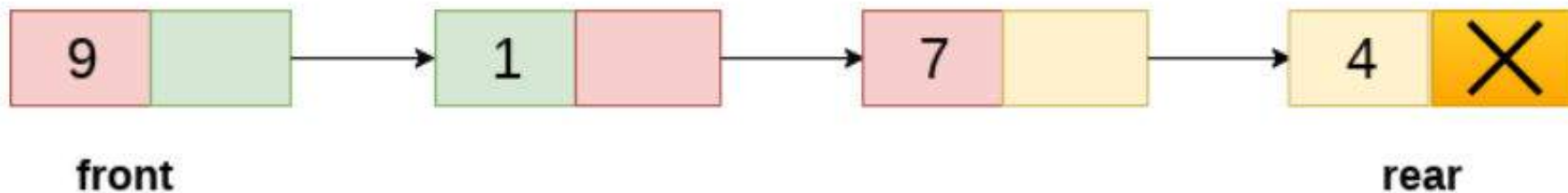
Applications of Queue data structure:

1. Queue is useful in CPU scheduling, Disk Scheduling. When multiple processes require CPU at the same time, various CPU scheduling algorithms are used which are implemented using Queue data structure.
2. When data is transferred asynchronously between two processes. Queue is used for synchronization. Examples: IO Buffers, pipes, file IO, etc.
3. In print spooling, documents are loaded into a buffer and then the printer pulls them off the buffer at its own rate. Spooling also lets you place a number of print jobs on a queue instead of waiting for each one to finish before specifying the next one.

4. Breadth First search in a Graph .It is an algorithm for traversing or searching graph data structures. It starts at some arbitrary node of a graph and explores the neighbour nodes first, before moving to the next level neighbours.
5. Handling of interrupts in real-time systems. The interrupts are handled in the same order as they arrive, First come first served.
6. In real life, Call Center phone systems will use Queues, to hold people calling them in an order, until a service representative is free.

Linked Queue INTRODUCTION Contd....

➤ The linked representation of queue is shown in the following figure.



Linked Queue

Operations on Linked Queue

➤ There are two basic operations which can be implemented on the linked queues.

1. Insertion Operation and
2. Deletion Operation.

➤ The insert operation append the queue by adding an element to the end of the queue.

➤ Deletion operation removes the element that is first inserted among all the queue elements.

SUMMARY

- A queue is an ordered list in which all insertions take place at one end, the rear, while all deletions take place at the other end, the front.
- The array implementation can not be used for the large scale applications where the queues are implemented.
- One of the alternative of array implementation is linked list implementation of queue.
- In a queue using singly linked list, we perform the following operations...
 - i) Insertion ii) Deletion iii) Display iv) Search
- The insert operation append the queue by adding an element to the end of the queue.
- Deletion operation removes the element that is first inserted among all the queue elements.

SELF-ASSESSMENT QUESTIONS

1. In linked list implementation of a queue, where does a new element be inserted?

- a) At the head of link list
- b) At the centre position in the link list
- c) **At the tail of the link list**
- d) At any position in the linked list

2. In linked list implementation of a queue, front and rear pointers are tracked. Which of these pointers will change during an insertion into EMPTY queue?

- a) Only front pointer
- b) Only rear pointer
- c) **Both front and rear pointer**
- d) No pointer will be changed

- 1. Describe About Queue using Singly Linked List?**
- 2. List out different operations can perform on Queue using Singly Linked Lists?**
- 3. Analyze the time complexity of the Queue using Singly Linked List for the operations such as insertion, deletion, and search.**
- 4. Write a C language program that implement the Queue using Singly Linked List.**
- 5. Summarize What is Queue using Singly Linked List and operations possible on it?**

Reference Books:

1. Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2010 , Second Edition, Pearson Education.
2. Ellis Horowitz, Fundamentals of Data Structures in C: Second Edition, 2015
3. A.V.Aho, J. E. Hopcroft, and J. D. Ullman, “Data Structures And Algorithms”, Pearson Education, First Edition Reprint 2003.

Sites and Web links:

1. <https://nptel.ac.in/courses/106102064>
2. <https://in.udacity.com/course/intro-to-algorithms--cs215>
3. <https://www.coursera.org/learn/data-structures?action=enroll>

THANK YOU

Advanced Algorithms & Data Structures

AIM OF THE SESSION

To familiarize students with the basic concept of AVL Tree

INSTRUCTIONAL OBJECTIVES

This Session is designed to:

1. Demonstrate the AVL Trees
2. List out the operations possible on the AVL Trees.
3. Describe the each operation
4. Describe the rotations of AVL Trees.

LEARNING OUTCOMES

At the end of this session, you should be able to:

1. Define AVL Trees
2. Describe four rotations of AVL Trees and their operations
3. Summarize definition, types and operations of AVL Trees and its applications

AVL Tree Introduction

AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962. The tree is named AVL in honour of its inventors.

AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.

Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

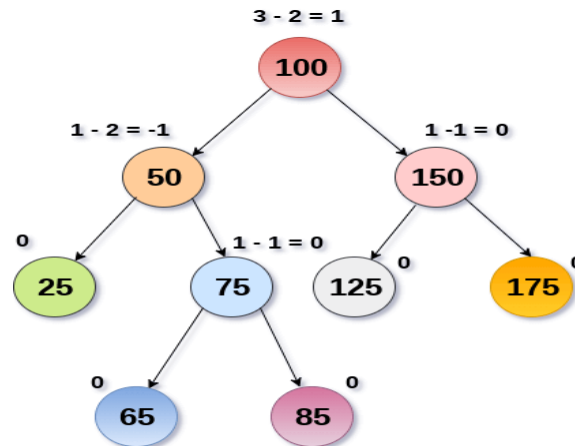
Balance Factor (k) = height (left(k)) - height (right(k))

If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.

If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.

If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

An AVL tree is given in the following figure. We can see that, balance factor associated with each node is in between -1 and +1. therefore, it is an example of AVL tree.



AVL Tree

Complexity

Algorithm	Average case	Worst case
Space	$O(n)$	$O(n)$
Search	$O(\log n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$

Operations on AVL tree

Due to the fact that, AVL tree is also a binary search tree therefore, all the operations are performed in the same way as they are performed in a binary search tree. Searching and traversing do not lead to the violation in property of AVL tree.

SN	Operation	Description
1	Insertion	Insertion in AVL tree is performed in the same way as it is performed in a binary search tree. However, it may lead to violation in the AVL tree property and therefore the tree may need balancing. The tree can be balanced by applying rotations.
2	Deletion	Deletion can also be performed in the same way as it is performed in a binary search tree. Deletion may also disturb the balance of the tree therefore, various types of rotations are used to rebalance the tree.

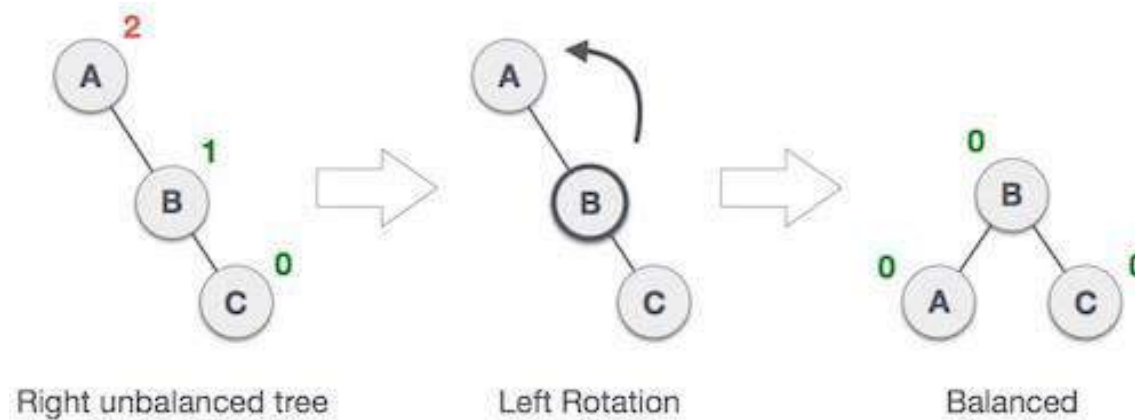
AVL Rotations

We perform rotation in AVL tree only in case if Balance Factor is other than **-1, 0, and 1**. There are basically four types of rotations which are as follows:

1. L L rotation: Inserted node is in the left subtree of left subtree of A
2. R R rotation : Inserted node is in the right subtree of right subtree of A
3. L R rotation : Inserted node is in the right subtree of left subtree of A
4. R L rotation : Inserted node is in the left subtree of right subtree of A

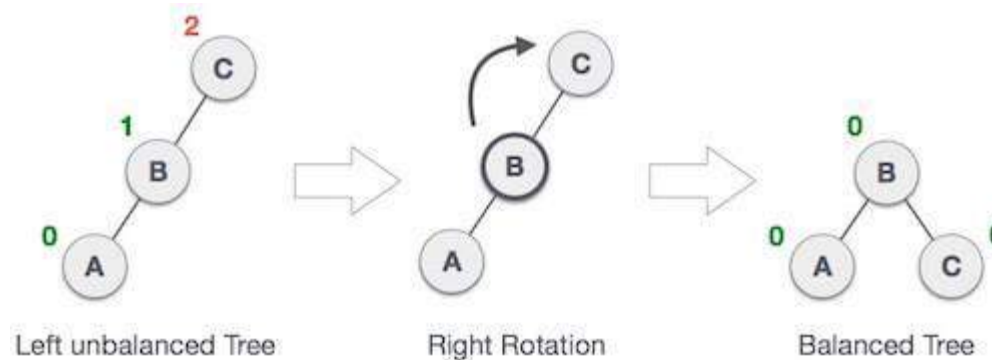
1. RR Rotation

When BST becomes unbalanced, due to a node is inserted into the right subtree of the right subtree of A, then we perform RR rotation, [RR rotation](#) is an anticlockwise rotation, which is applied on the edge below a node having balance factor -2



2. LL Rotation

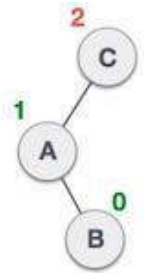
When BST becomes unbalanced, due to a node is inserted into the left subtree of the left subtree of C, then we perform LL rotation, [LL rotation](#) is clockwise rotation, which is applied on the edge below a node having balance factor 2.



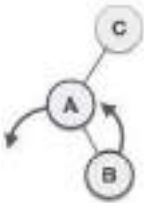
Above example, node C has balance factor 2 because a node A is inserted in the left subtree of C left subtree. We perform the LL rotation on the edge below A.

3. LR Rotation

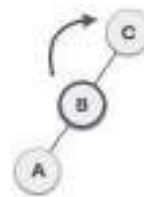
Double rotations are bit tougher than single rotation which has already explained above. LR rotation = RR rotation + LL rotation, i.e., first RR rotation is performed on subtree and then LL rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.



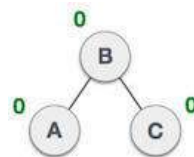
A node B has been inserted into the right subtree of A the left subtree of C, because of which C has become an unbalanced node having balance factor 2. This case is L R rotation where: Inserted node is in the right subtree of left subtree of C



After performing RR rotation, node C is still unbalanced, i.e., having balance factor 2, as inserted node A is in the left of left of C



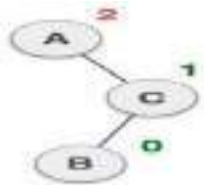
Now we perform LL clockwise rotation on full tree, i.e. on node C. node C has now become the right subtree of node B, A is left subtree of B



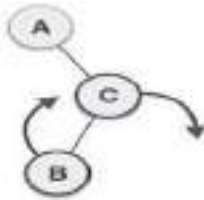
Balance factor of each node is now either -1, 0, or 1, i.e. BST is balanced now.

4. RL Rotation

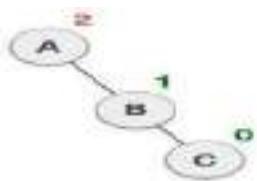
As already discussed, that double rotations are bit tougher than single rotation which has already explained above. RL rotation = LL rotation + RR rotation, i.e., first LL rotation is performed on subtree and then RR rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.



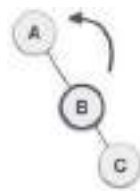
A node **B** has been inserted into the left subtree of **C** the right subtree of **A**, because of which **A** has become an unbalanced node having balance factor -2. This case is RL rotation where: Inserted node is in the left subtree of right subtree of **A**



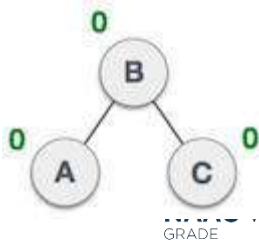
As RL rotation = LL rotation + RR rotation, hence, LL (clockwise) on subtree rooted at **C** is performed first. By doing RR rotation, node **C** has become the right subtree of **B**.



After performing LL rotation, node **A** is still unbalanced, i.e. having balance factor -2, which is because of the right-subtree of the right-subtree node **A**.



Now we perform RR rotation (anticlockwise rotation) on full tree, i.e. on node **A**. node **C** has now become the right subtree of node **B**, and node **A** has become the left subtree of **B**.



Balance factor of each node is now either -1, 0, or 1, i.e., BST is balanced now.

AVL TREES

AVL Tree:

- One of the more popular balanced trees, known as an AVL tree, was introduced in 1962 by Adelson-Velski and Landis.

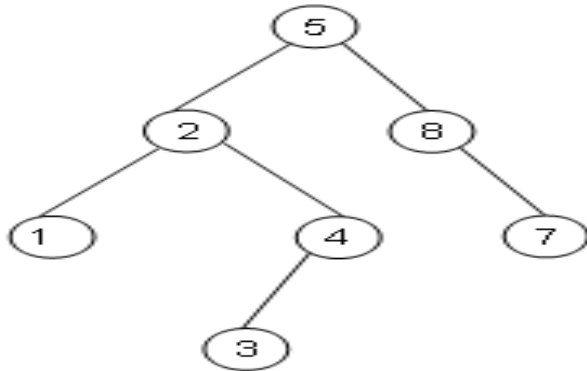
Definition: 1

- An empty binary tree is an AVL tree. If T is a nonempty binary tree with T_L and T_R as its left and right sub trees, the T is an AVL tree iff
 - 1. T_L and T_R are AVL trees and
 - 2. $|h_L - h_R| \leq 1$ where h_L and h_R are the heights of T_L and T_R , respectively.
- An AVL search tree is a binary search tree that is also an AVL tree.

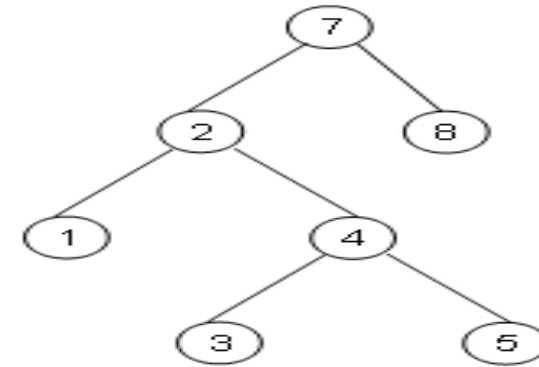
Definition: 2

- An AVL tree is a binary search tree in which for *every* node in the tree, the height of the left and right sub trees differ by at most 1.

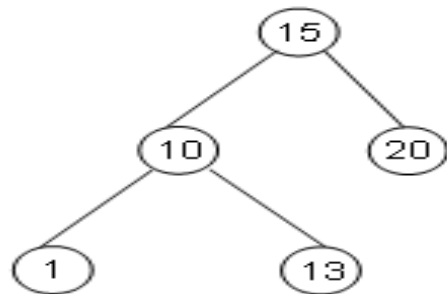
Examples:



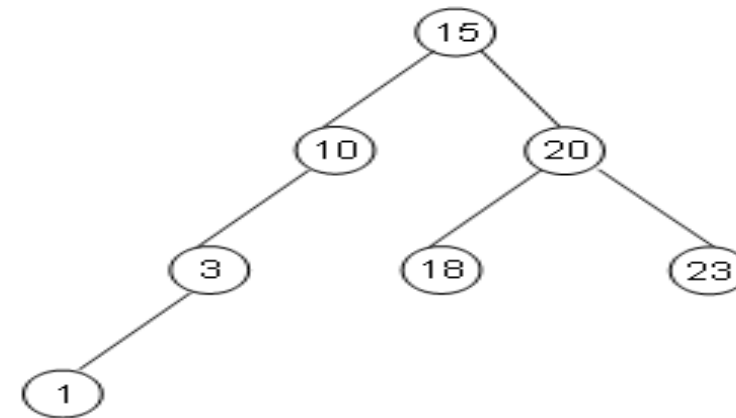
Fig(a) An AVL Tree



Fig(b) Not An Tree

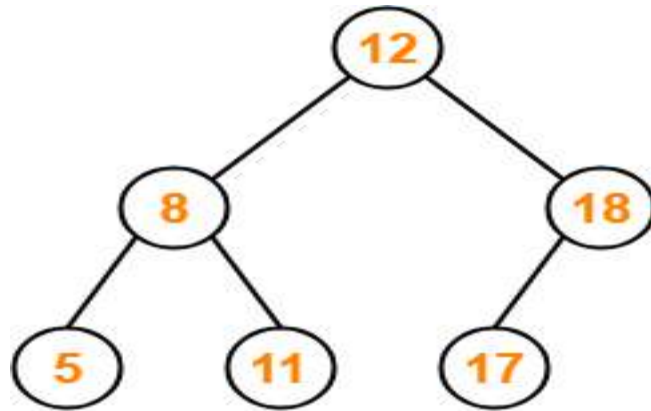


Fig(c) An AVL Tree

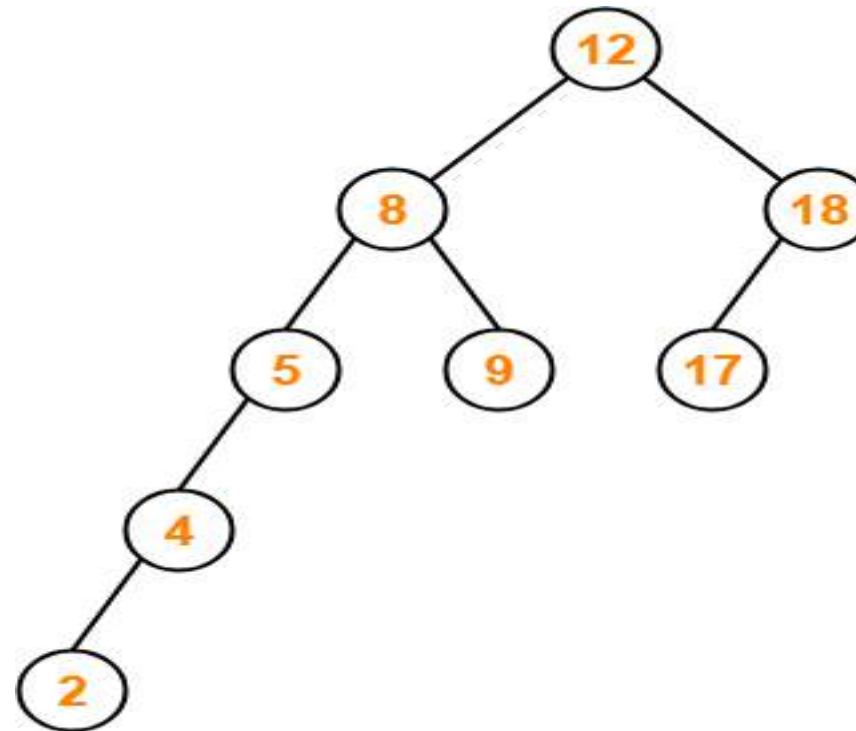


Fig(d) Not AVL Tree

Examples:



AVL Tree Example



Not an AVL Tree

Balance Factor:

- In AVL tree, Balance factor is defined for every node.
- Balance factor of a node = Height of its left subtree – Height of its right subtree
- In AVL tree, Balance factor of every node is either 0 or 1 or -1.

AVL Tree Operations:

- Like **BST Operations**, commonly performed operations on AVL tree are:
 - a. Search Operation
 - b. Insertion Operation
 - c. Deletion Operation

- After performing any operation on AVL tree, the balance factor of each node is checked.
- There are following two cases possible:

Case-01:

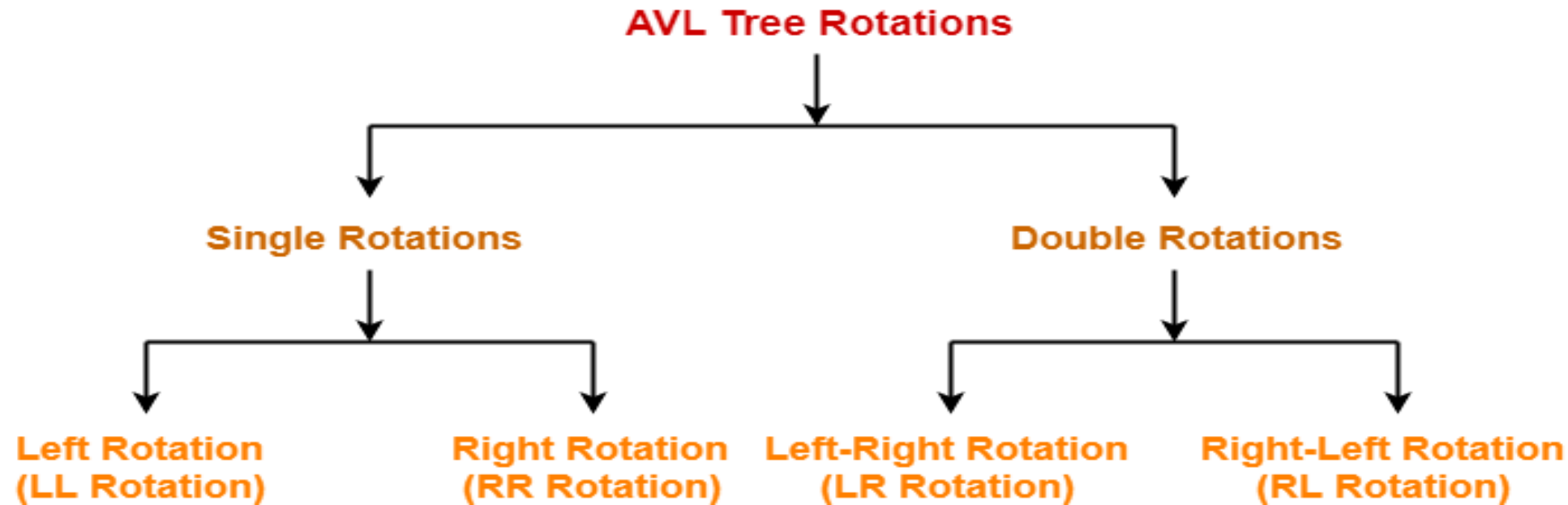
- After the operation, the balance factor of each node is either 0 or 1 or -1.
- In this case, the AVL tree is considered to be balanced.
- The operation is concluded.

Case-02:

- After the operation, the balance factor of at least one node is not 0 or 1 or -1.
- In this case, the AVL tree is considered to be imbalanced.
- Rotations are then performed to balance the tree.

AVL Tree Rotations:

- Rotation is the process of moving the nodes to make tree balanced.

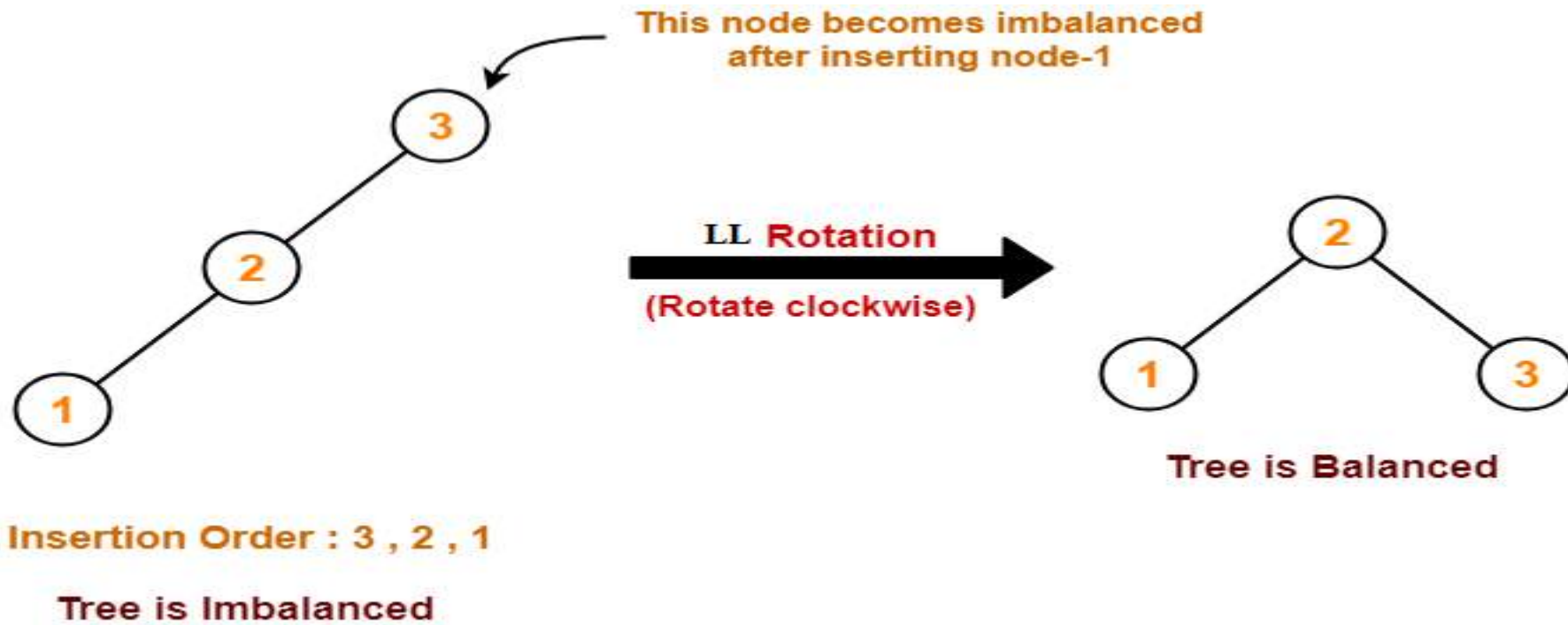


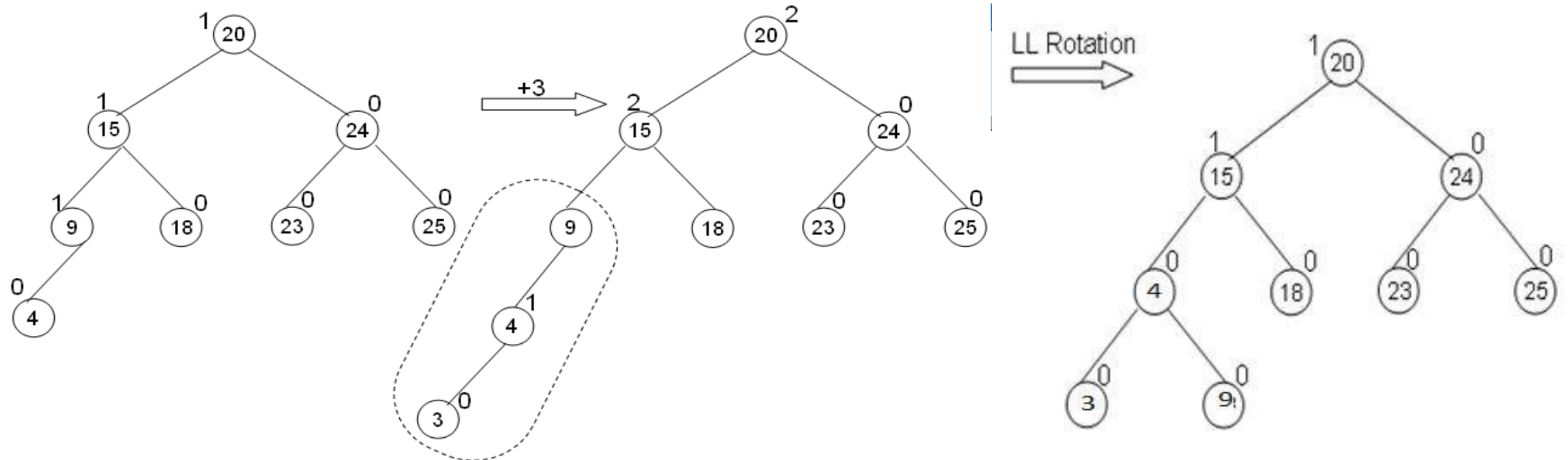
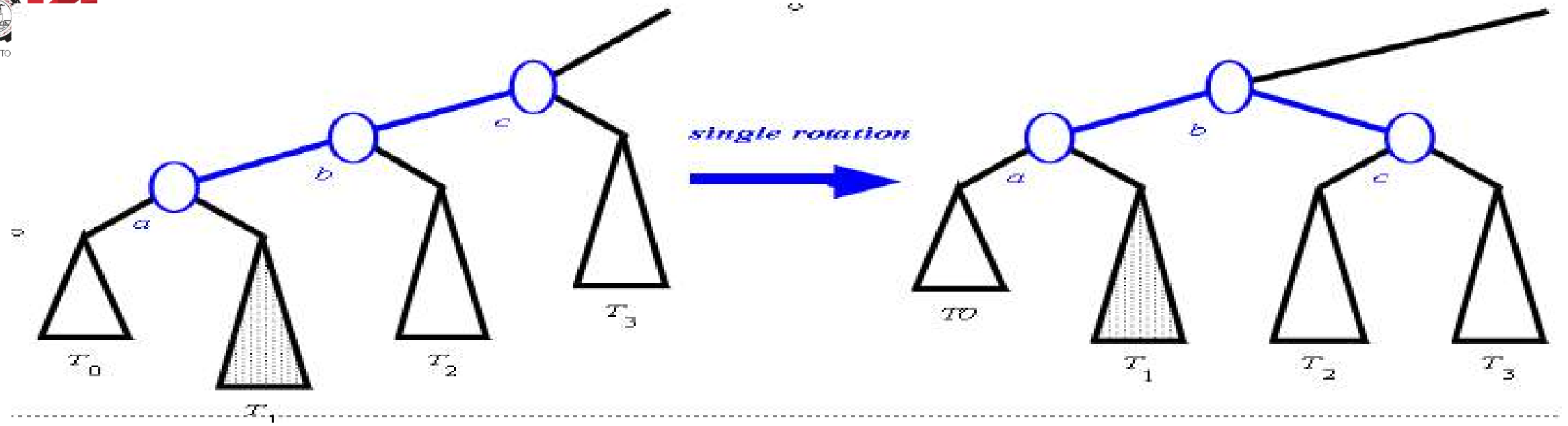
1. Left Rotation (LL Rotation)
2. Right Rotation (RR Rotation)
3. Left-Right Rotation (LR Rotation)
4. Right-Left Rotation (RL Rotation)

Rotations:

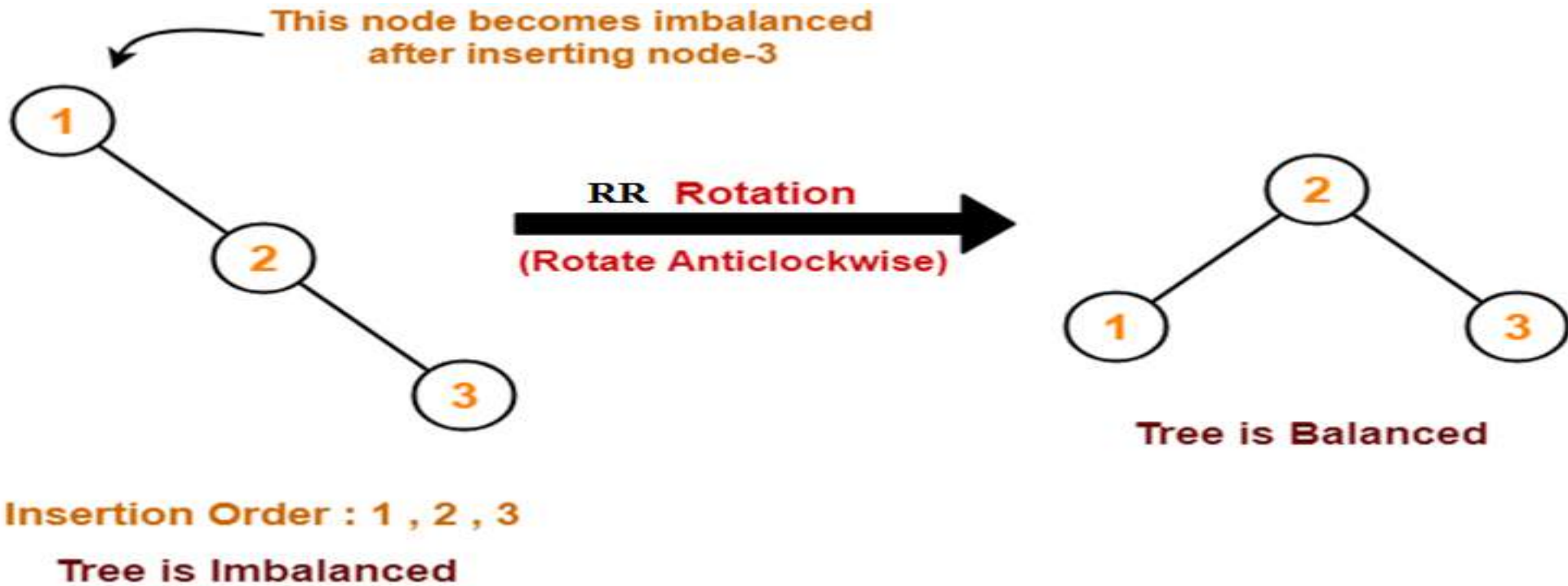
- The insert and delete operations of AVL tree are the same as binary search tree (BST).
- Since an insertion (deletion) involves adding (deleting) a tree node, this can only increase (decrease) the heights of some subtree(s) by 1.
- Thus, the AVL tree property may be violated.
- If the AVL tree property is violated at a node x , it means that the heights of $\text{left}(x)$ and $\text{right}(x)$ differ by exactly 2.
- After the insertion or deletion operations, we need to examine the tree and see if any node violates the AVL tree property.
- If the AVL tree property is violated at node x , single or double rotation will be applied to x to restore the AVL tree property.
- Rotation will be applied in a bottom-up manner starting at the place of insertion (deletion).
- Thus, when we perform a rotation at x , the AVL tree property is restored at all proper descendants of x . This fact is important.

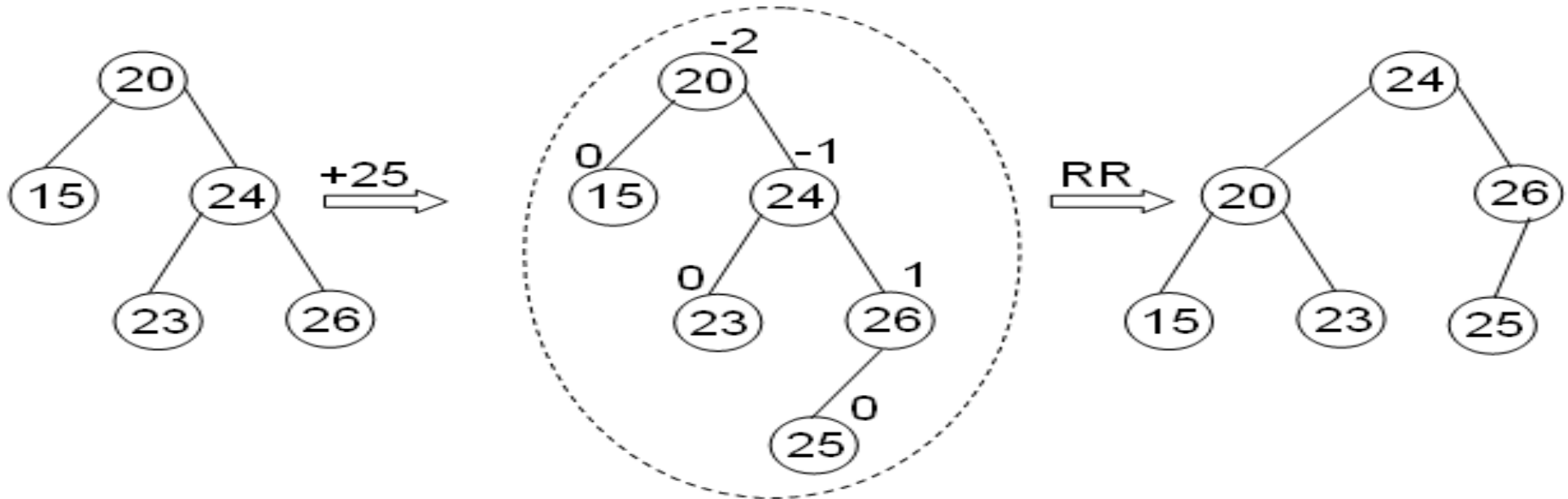
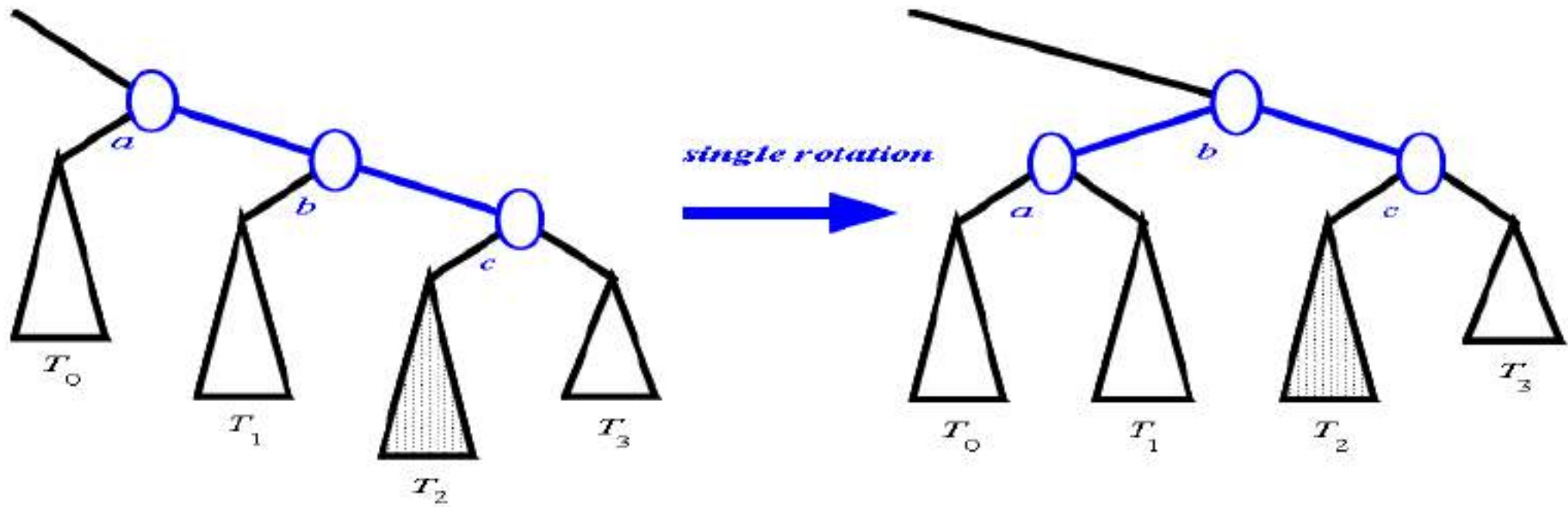
LL Rotation:



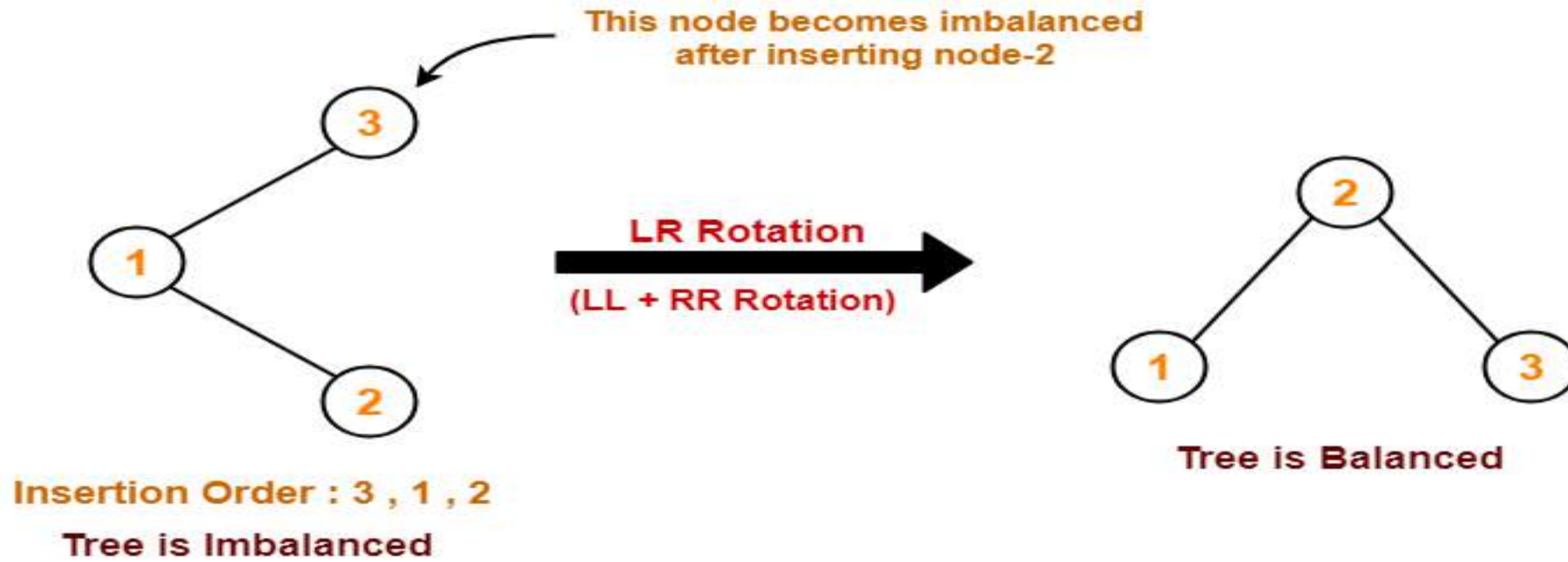


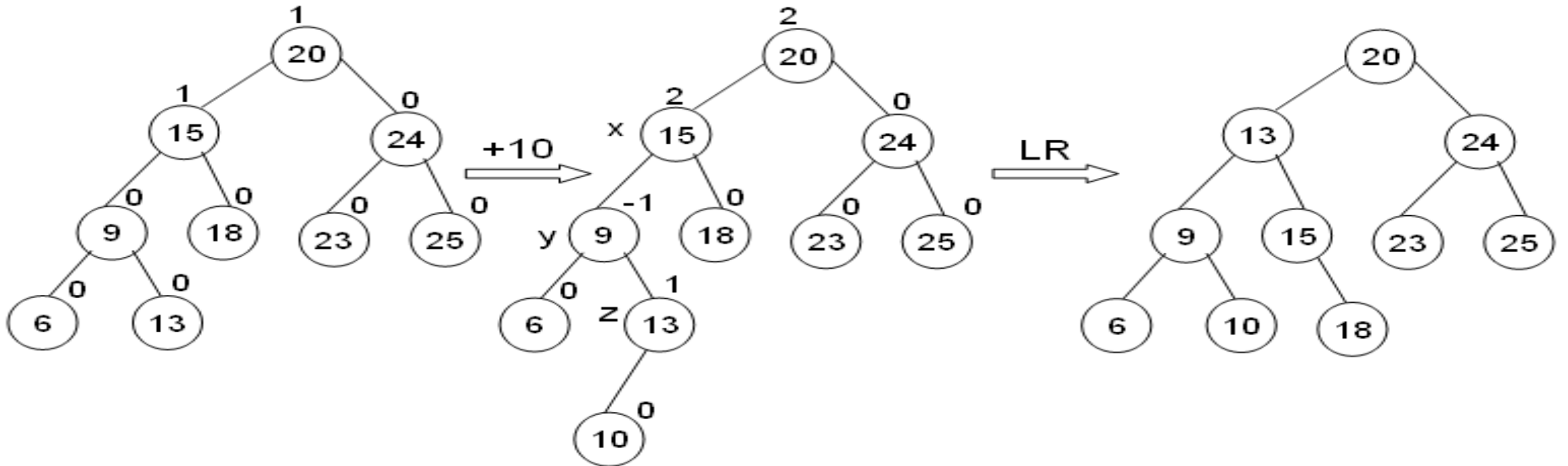
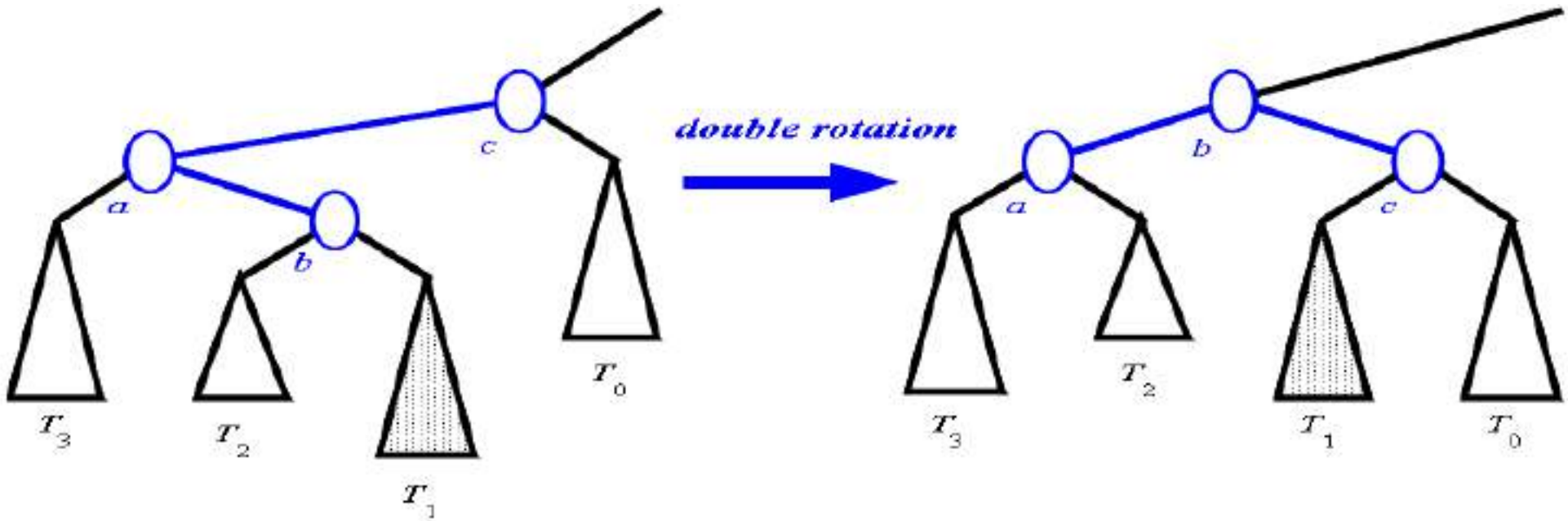
RR Rotation:



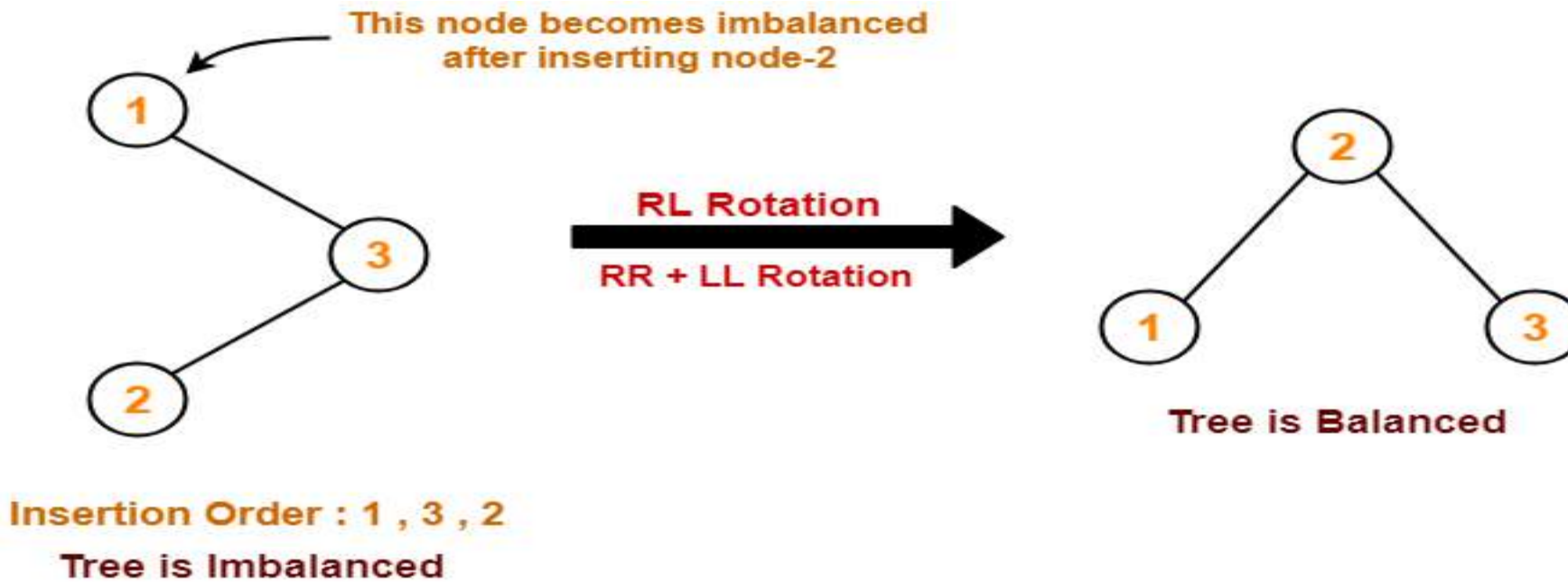


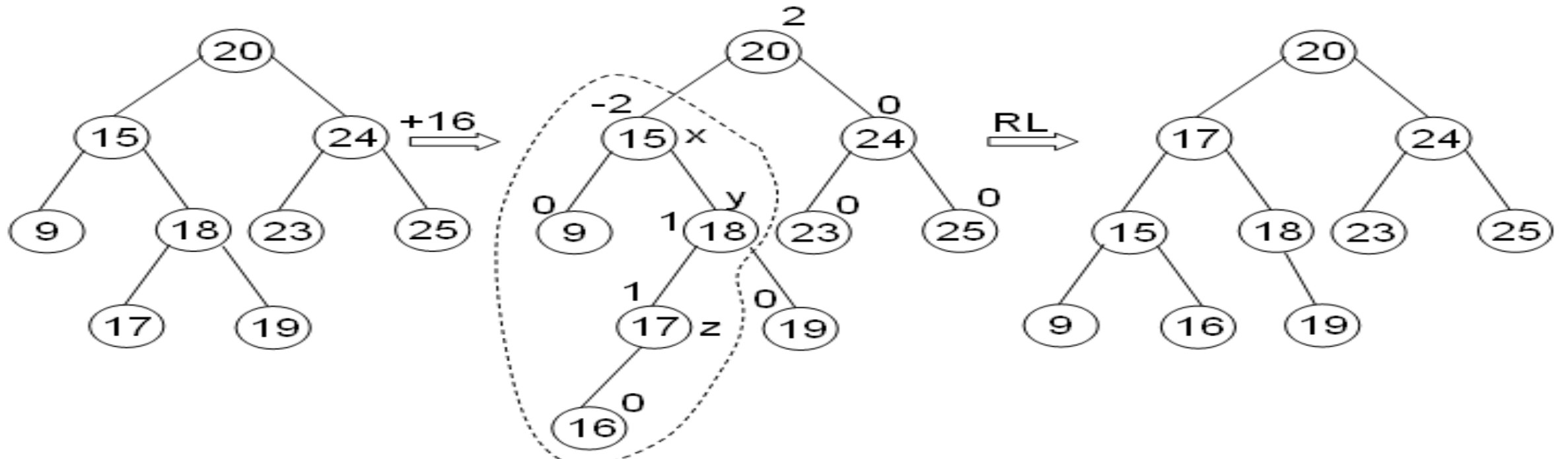
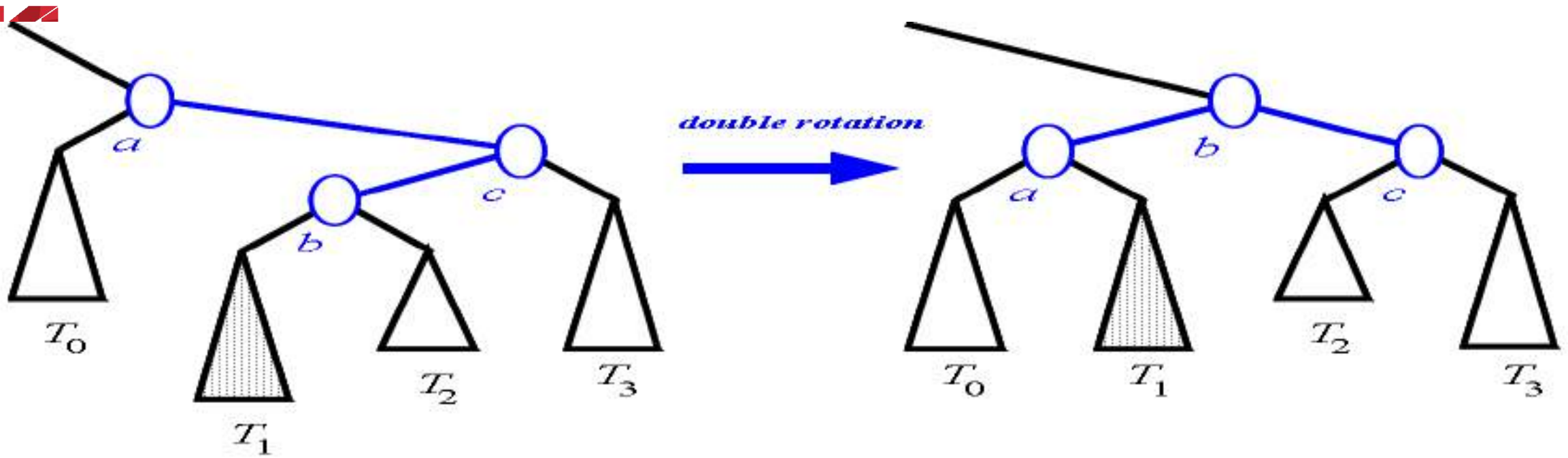
LR Rotation:





RL Rotation:





Insertion:

- Perform normal BST insertion.
- Check and restore AVL tree property.
 - Trace from path of inserted leaf towards the root, and check if the AVL tree property is violated.
 - Check to see if heights of $\text{left}(x)$ and $\text{right}(x)$ height differ by at most 1.
 - If the AVL tree property is violated, there are 4 rotation cases to restore the AVL tree property.
- Trace from path of inserted leaf towards the root, and check if the AVL tree property is violated. Perform rotation if necessary.

- For insertion, once we perform (single or double) rotation at a node x , the AVL tree property is already restored. We need not to perform any rotation at any ancestor of x .
- Thus one rotation is enough to restore the AVL tree property.
- There are 4 different cases (actually 2), so don't mix up them!
- The time complexity to perform a rotation is $O(1)$.
- The time complexity to insert, and find a node that violates the AVL property is dependent on the height of the tree, which is $O(\log(n))$.
- So insertion takes $O(\log(n))$.

Deletion:

- Delete a node x as in ordinary BST (Note that x is either a leaf or x has exactly one child.).
- Check and restore the AVL tree property.
- Trace from path of deleted node towards the root, and check if the AVL tree property is violated.
- Similar to an insertion operation, there are four cases to restore the AVL tree property.
- The only difference from insertion is that after we perform a rotation at x , we may have to perform a rotation at some ancestors of x . $\diamond$ It may involve several rotations.

- Therefore, we must continue to trace the path until we reach the root.
- The time complexity to delete a node is dependent on the height of the tree, which is also $O(\log(n))$.

An Extended Example

Insert 3,2,1,4,5,6,7, 16,15,14

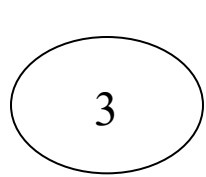


Fig 1

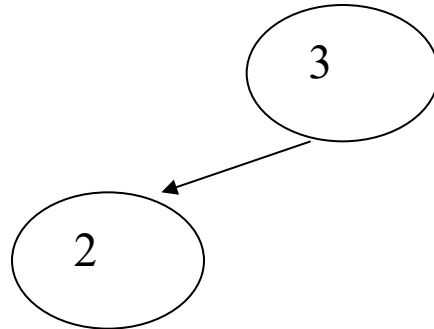


Fig 2

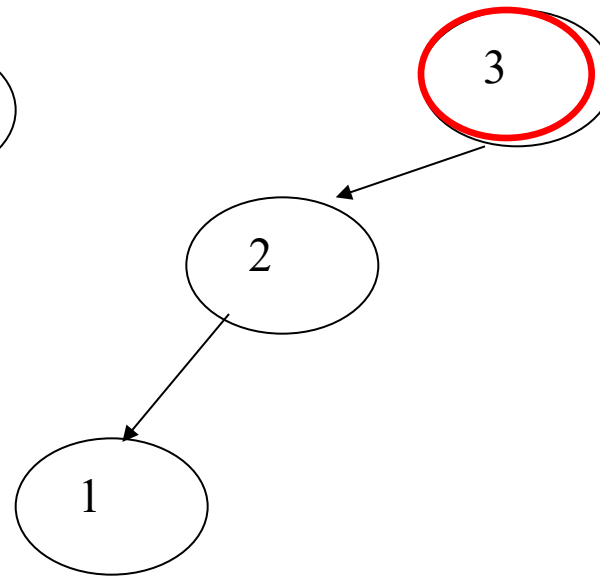


Fig 3

Single rotation

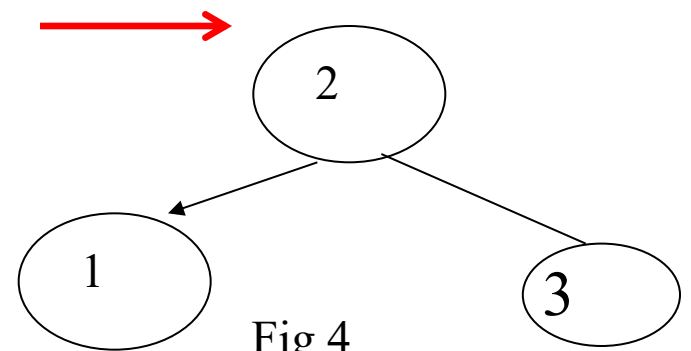


Fig 4

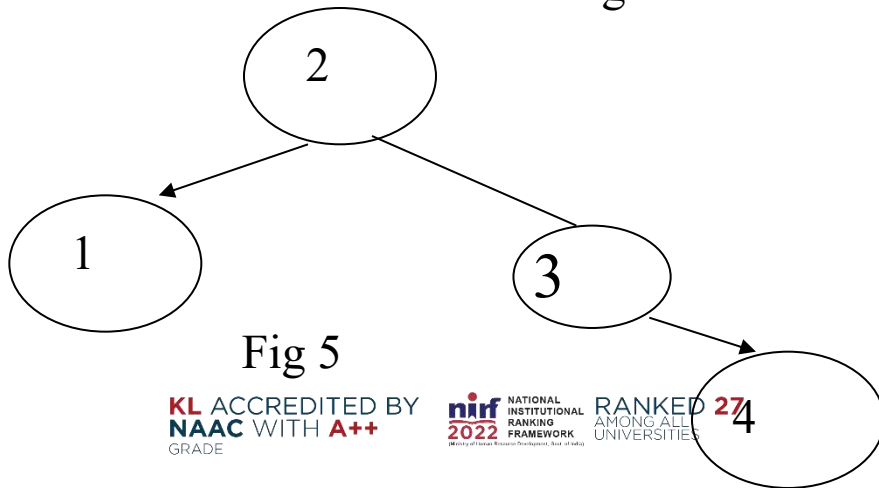


Fig 5

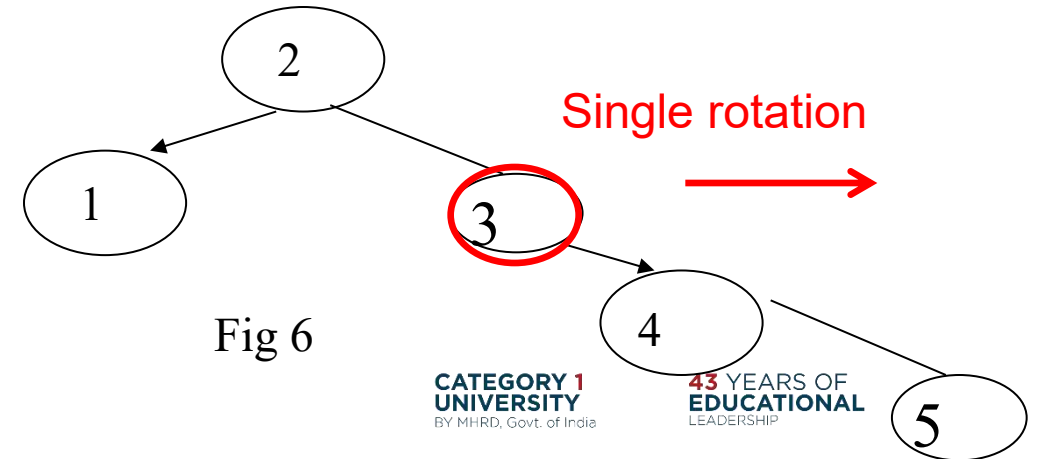


Fig 6

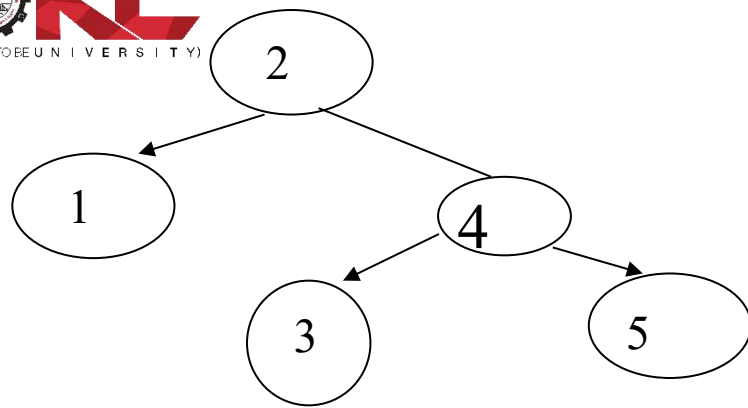


Fig 7

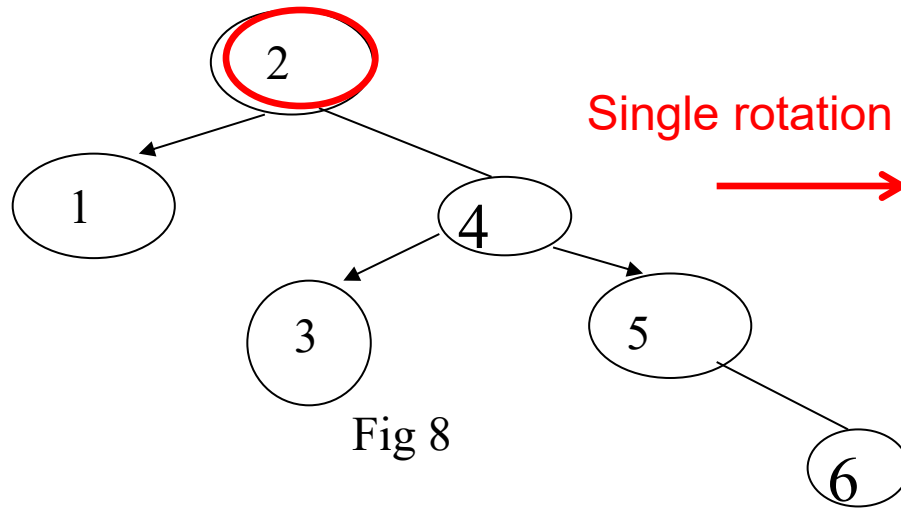


Fig 8

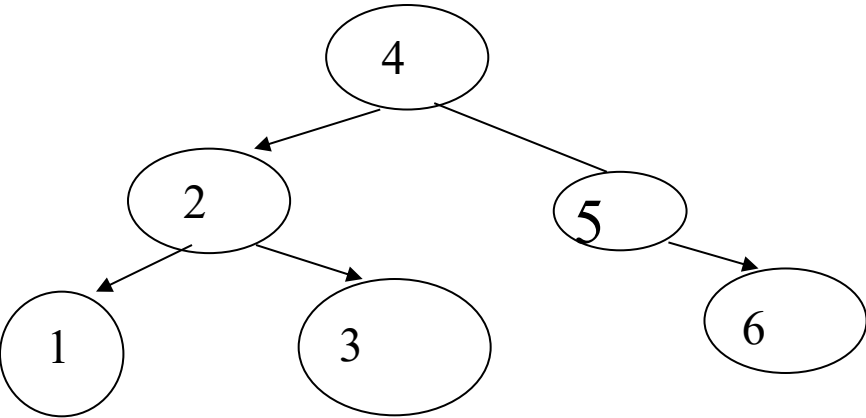


Fig 9

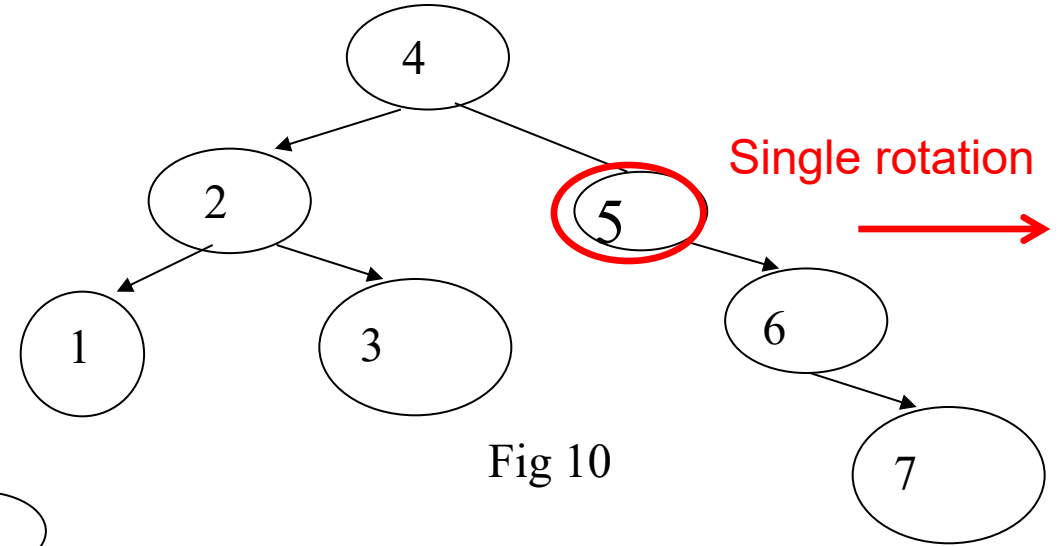


Fig 10

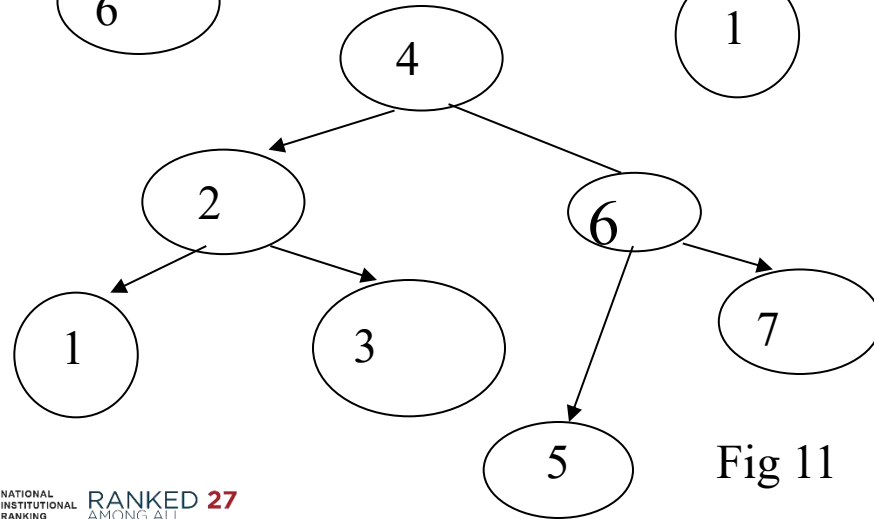


Fig 11

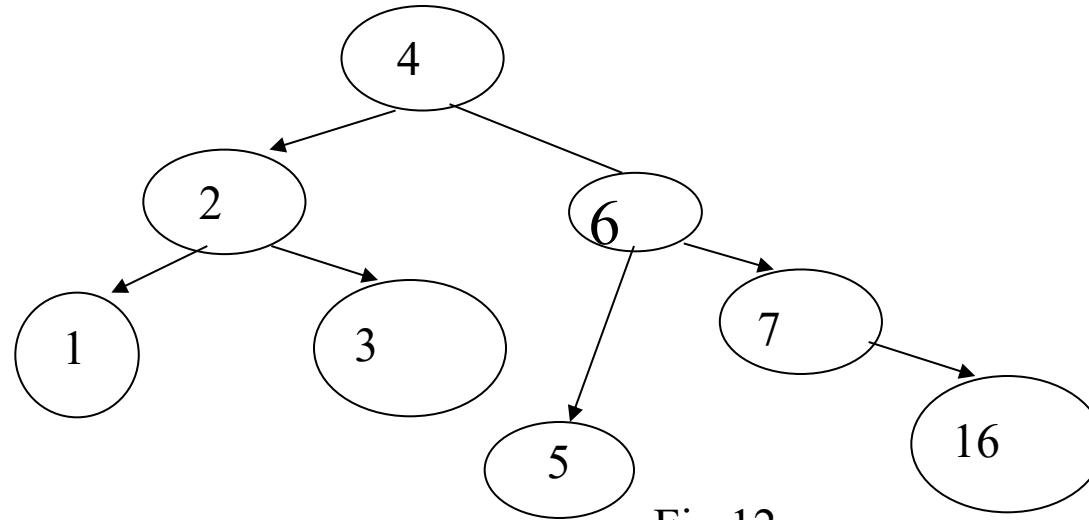


Fig 12

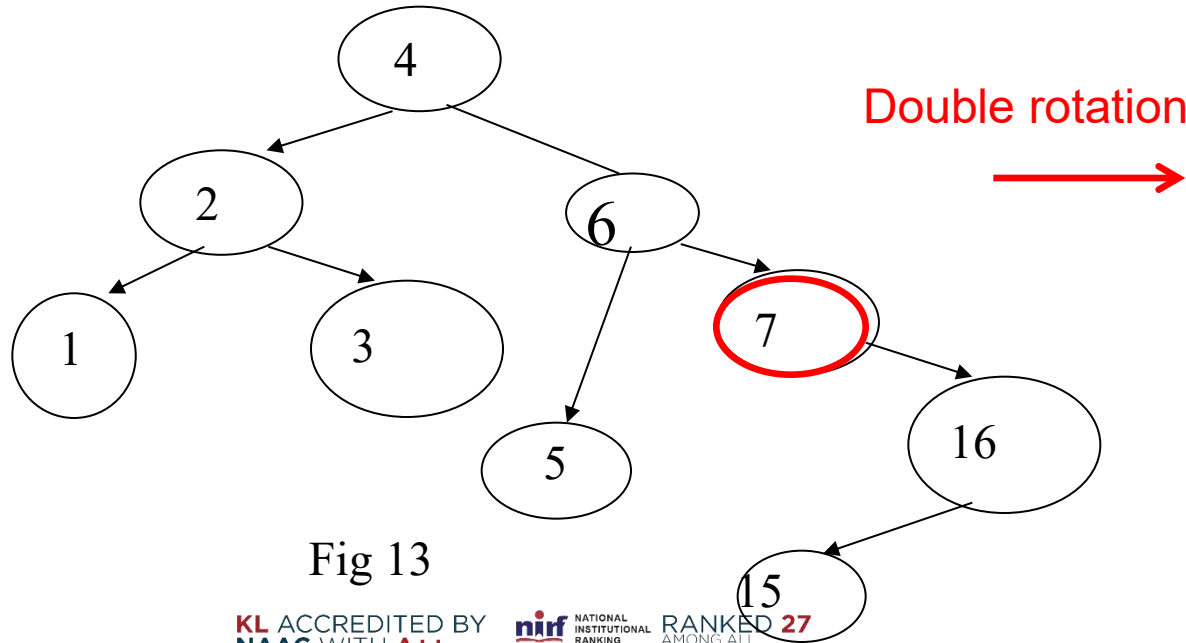


Fig 13

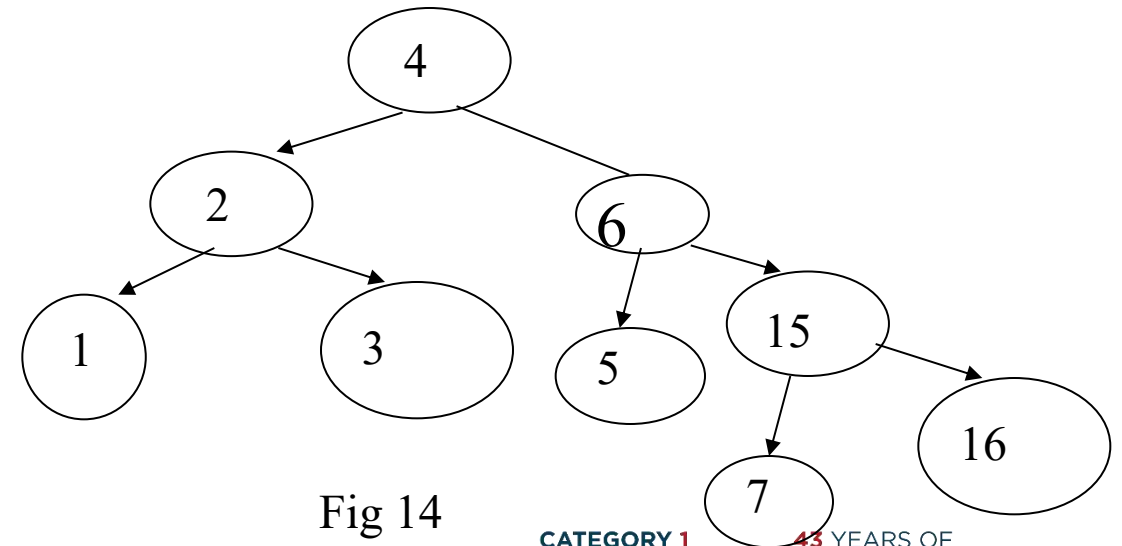
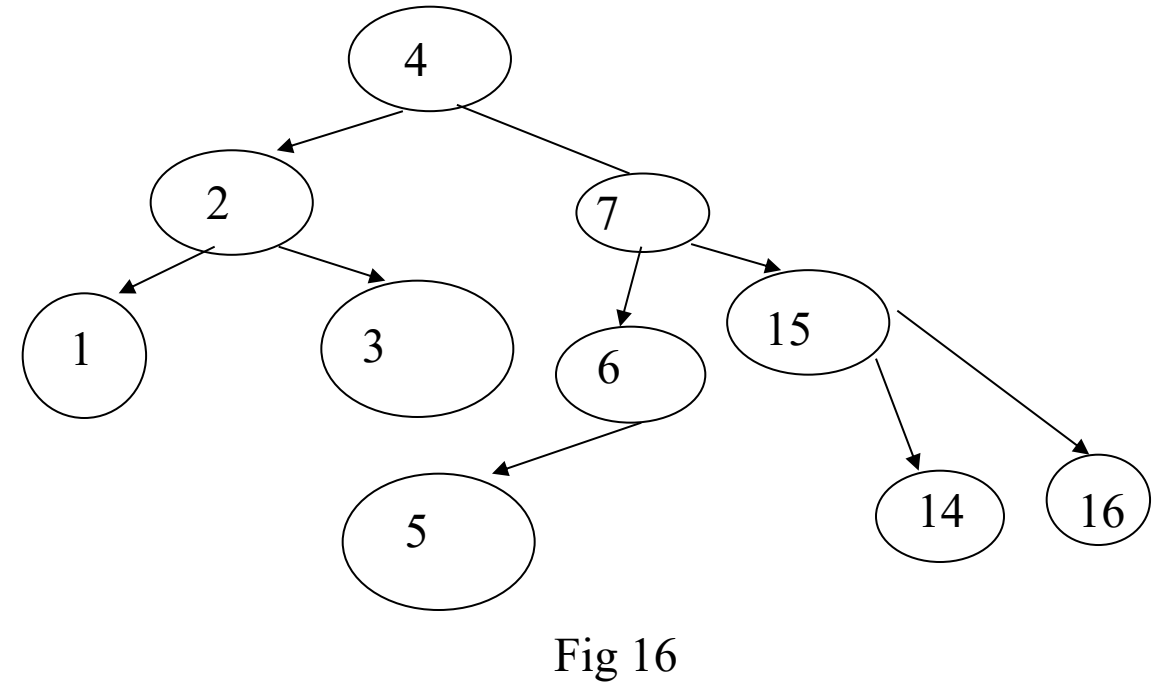
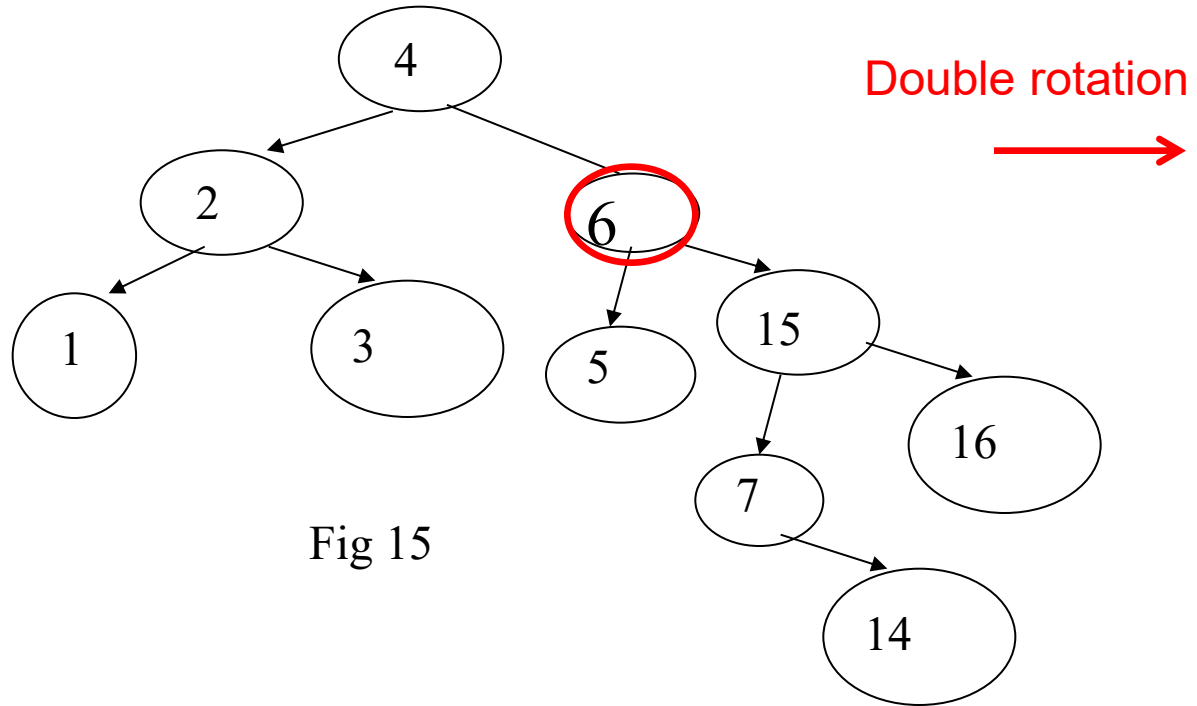


Fig 14



SUMMARY

- AVL Tree is a height-balanced BST that maintains its height by performing tree rotations according to a Balancing Criteria.
- Balancing Criteria: For all nodes, the sub tree height difference (balance factor) should be at most 1
- The valid values of the balance factor are -1, 0, and +1.
- The insert and delete operation require rotations to be performed after violating the balance factor.
- The balance factor of a node is defined as the difference between the height of the left and right sub tree of that node.
- Tree Rotations are changes in the structure of the tree, done only on 3 nodes without affecting the elements' order.

SELF-ASSESSMENT QUESTIONS

1. How to calculate the balance factor of a node in an AVL Tree?

- a) height of the left sub tree - height of the right sub tree
- b) height of the right sub tree - height of the left sub tree
- c) absolute difference between the heights of the left and right sub trees.
- d) sum of the heights of the left and right sub trees.

2. What is the primary purpose of rotations in AVL Trees?

- a) To increase the overall height of the tree.
- b) To maintain a balanced structure and ensure efficient operations.
- c) To create a perfectly symmetrical tree shape.
- d) To sort the nodes in ascending or descending order.

TERMINAL QUESTIONS

1. Describe About AVL Trees?
2. Explain the concept of balance factor in an AVL tree. What are the acceptable values for a node's balance factor?
3. Describe the four different types of rotations in an AVL tree (left, right, double left, double right). When is each type of rotation used?
4. Given a sequence of elements, construct an AVL tree by inserting them one by one. Explain your choices for insertion order and justify any performed rotations.

Reference Books:

1. Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2010 , Second Edition, Pearson Education.
2. Ellis Horowitz, Fundamentals of Data Structures in C: Second Edition, 2015
3. A.V.Aho, J. E. Hopcroft, and J. D. Ullman, “Data Structures And Algorithms”, Pearson Education, First Edition Reprint 2003.

Sites and Web links:

1. <https://nptel.ac.in/courses/106102064>
2. <https://in.udacity.com/course/intro-to-algorithms--cs215>
3. <https://www.coursera.org/learn/data-structures?action=enroll>

THANK YOU

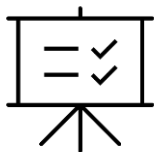
Advanced Algorithms & Data Structures

AIM OF THE SESSION



1. The students familiar with the basic concepts of root nodes, parents and Childrens nodes with real time examples and programming syntax.
2. These knowledge of basic concepts they can construction and demonstrate the **RED-BLACK** tree.

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate **RED-BLACK** Tree Constructions with best, worst and average cases of time and space complexity
2. Describe **RED-BLACK** Tree basic terminology and why it is needed
3. List out the general ideas about **RED-BLACK** construction and why it is needed in data structure
4. Describe the structure of **RED-BLACK** tree constructions and its significance usages in computer science

LEARNING OUTCOMES



At the end of this session, you should be able to:

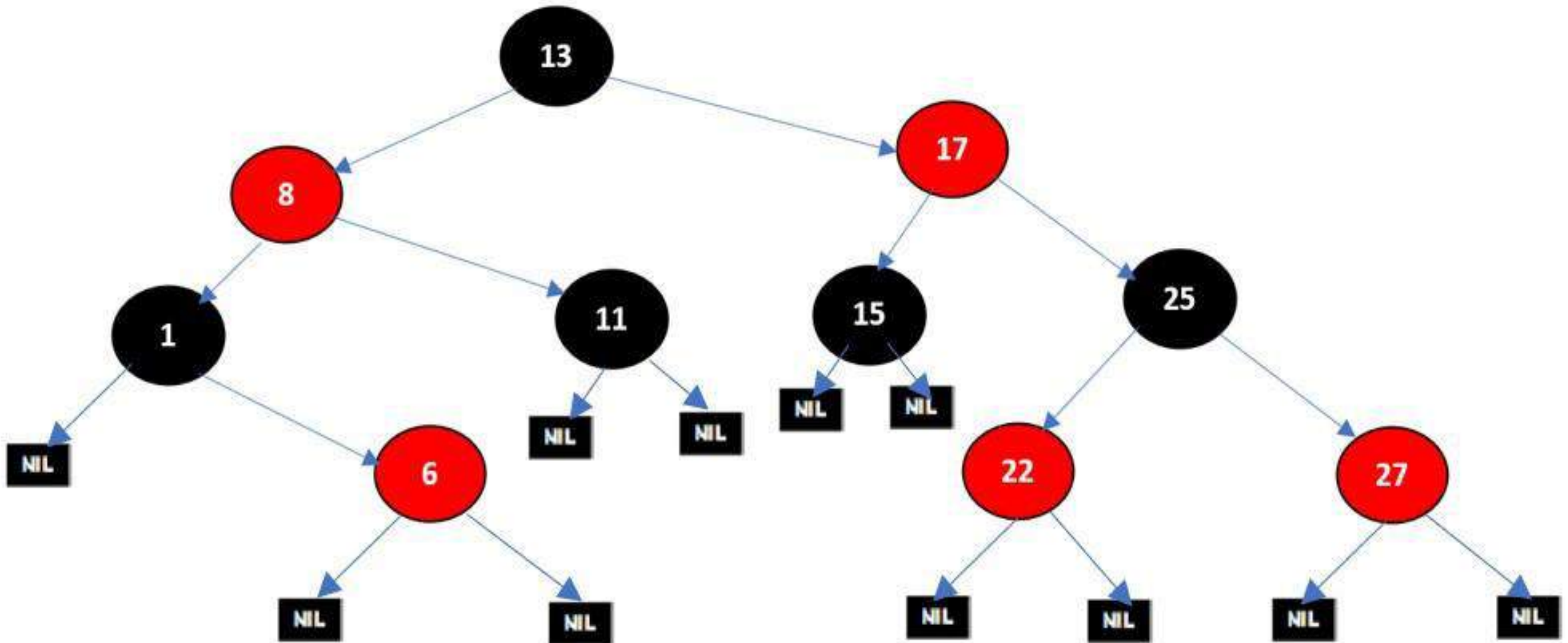
1. Define the concept of **RED-BLACK** Tree constructions phases, time and space complexity
2. Describe: **RED-BLACK** Tree constructions phases with all possible conditions
3. Summarize the **RED-BLACK** Tree importance in computer science in cases of storing and managing data

INTRODUCTION

- A **Red-Black** Trees are a type of self-balancing binary search tree.
- Why they are named as “**Red-Black**” because each node in the tree is marked with one of two colors, either **red** or **black**.
- **Red-Black** Trees were introduced by Rudolf Bayer and Volker Strassen in 1972.
- These colours are used to ensure that the tree remains balanced during insertions and deletions.

EXAMPLE OF RED-BLACK TREE

Constructing **Red-Black** tree with 10 nodes



REQUIREMENTS

- A **red-black** tree is a Binary tree where a particular node has colour as an extra attribute, either red or black.
- **Red-black** trees maintain a slightly looser height invariant than AVL trees.
- In AVL trees, balancing restricts the difference in heights to at most one.
- For **red-black** trees, we have a different set of rules related to the colors of the nodes

SYNTAX OF RED-BLACK TREE

To create a node in **Red-Black** Trees

Syntax:

```
struct Node  
{  
    int key;  
    enum Color color;  
    struct Node *left, *right, *parent;
```

PROPERTIES OF RED-BLACK TREE

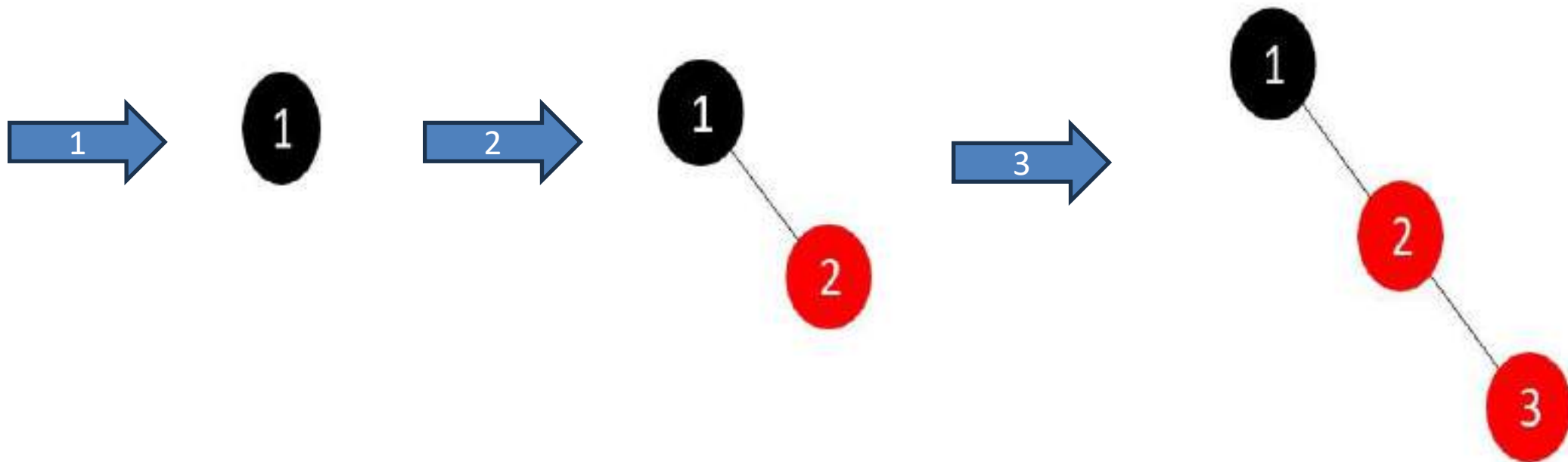
1. **Root property:** The root is **black**.
2. **External property:** Every leaf (Leaf is a NULL child of a node) is **black** in **Red-Black** tree.
3. **Internal property:** The children of a **red** node are **black**. Hence possible parent of **red** node is a **black** node.
4. **Depth property:** All the leaves have the same **black** depth.
5. **Path property:** Every simple path from root to descendant leaf node contains same number of **black** nodes.

RULES FOR INSERTION OF NODE

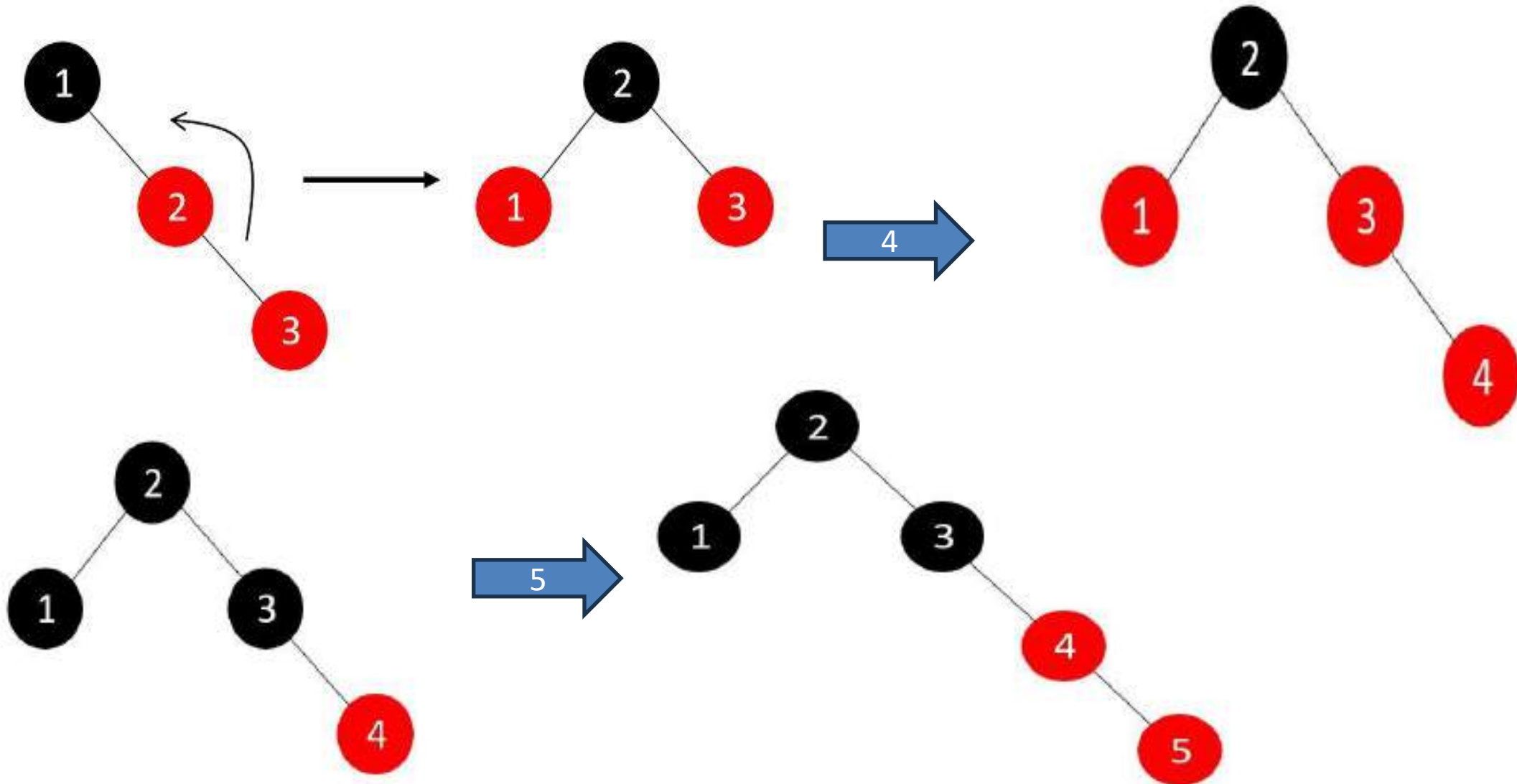
- If the tree is empty, make newnode as root and color it with **black**.
- If tree is not empty, create newnode as leaf node and color it with **red**.
- If parent of newnode is **black** then exit.
- If parent of newnode is **red** then check the color of parent sibling
 - a) If parent sibling color is **black** or null rotate and recolor.
 - b) If parent sibling color is **red**, then recolor parent and parent sibling
 - c) And check parent's parent is not root node. Then recolor and recheck.

CONSTRUCTION OF RB TREES

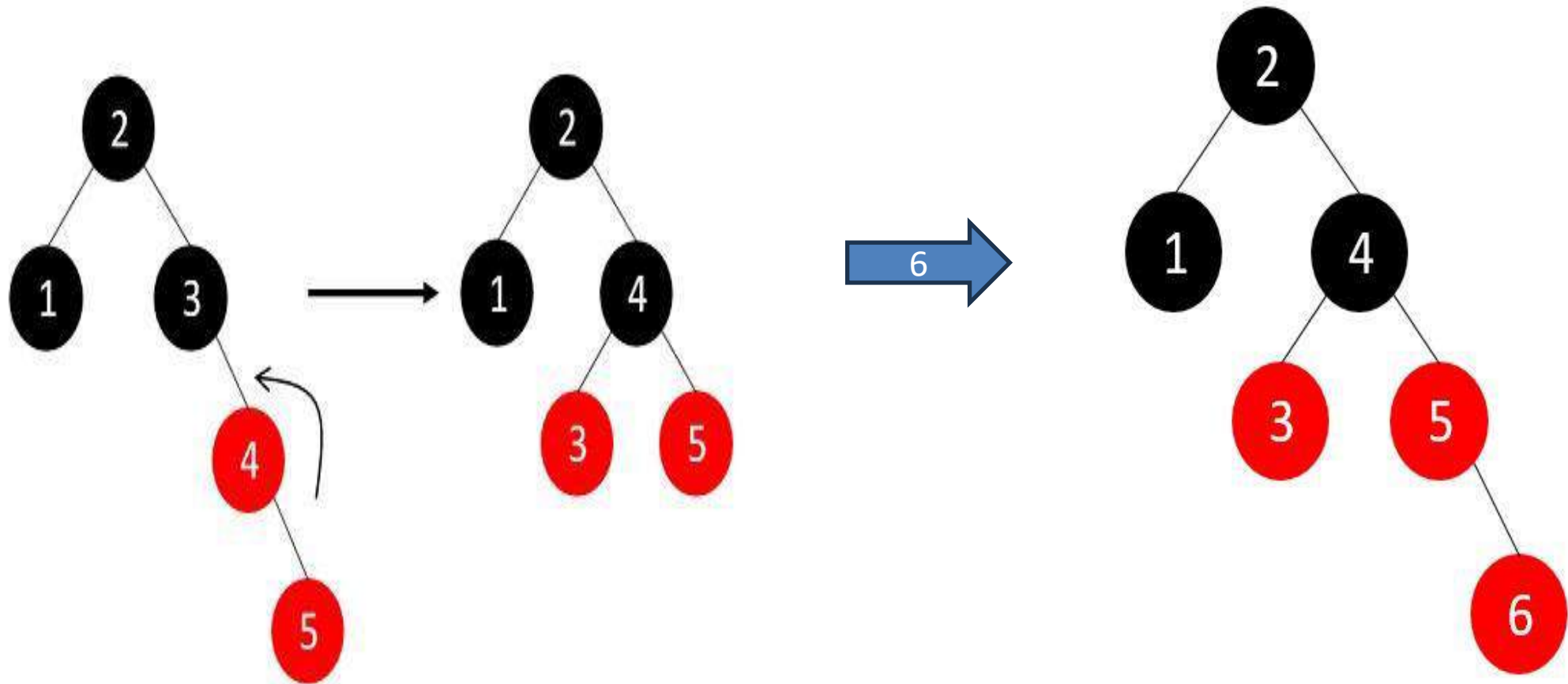
- Let's understand the inserting elements with following the **red-black** tree properties and rules
- Construct a red black tree with 1 to 7 numbers.



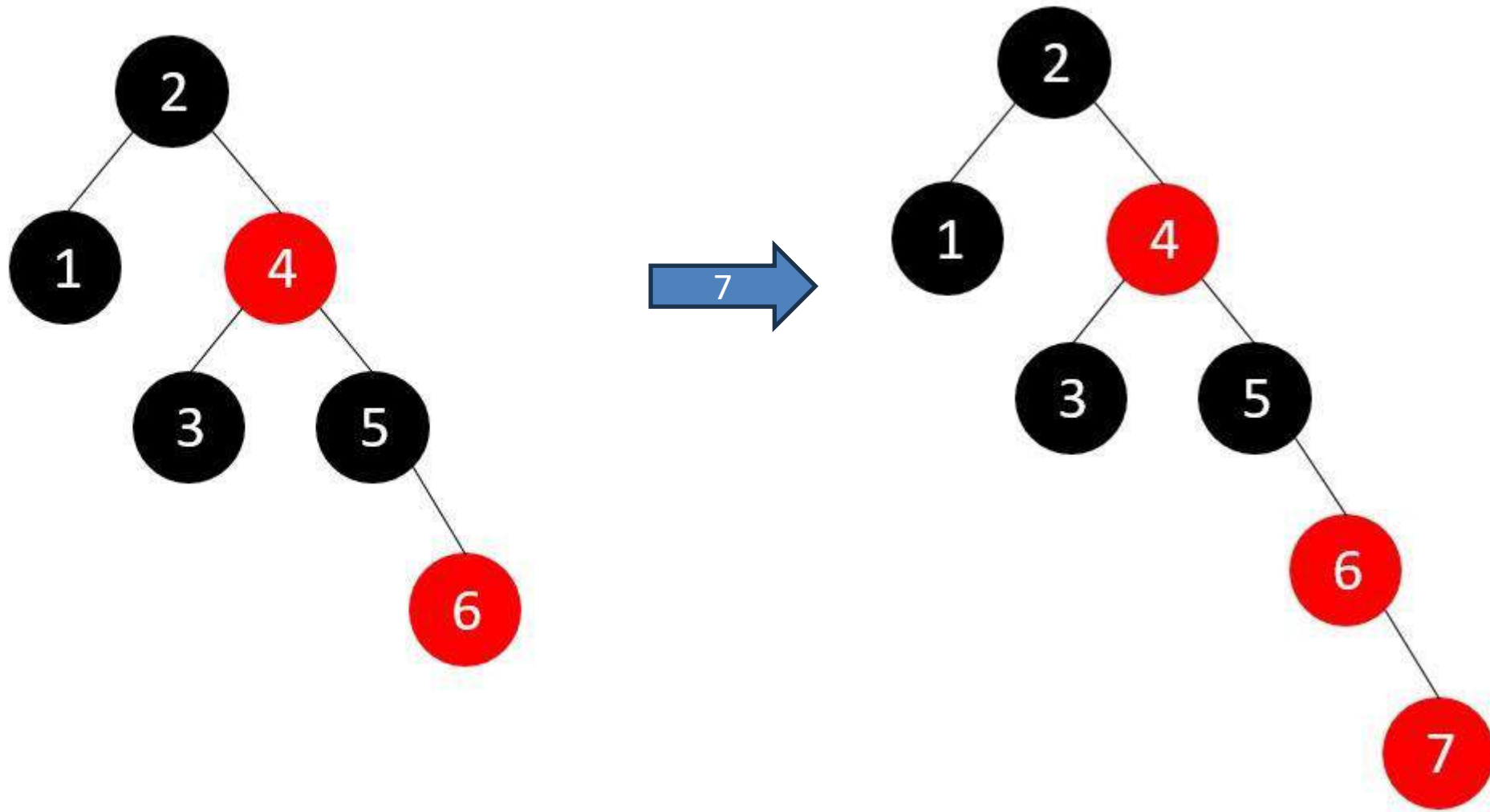
Example of RED-BLACK Tree



Example of RED-BLACK Tree

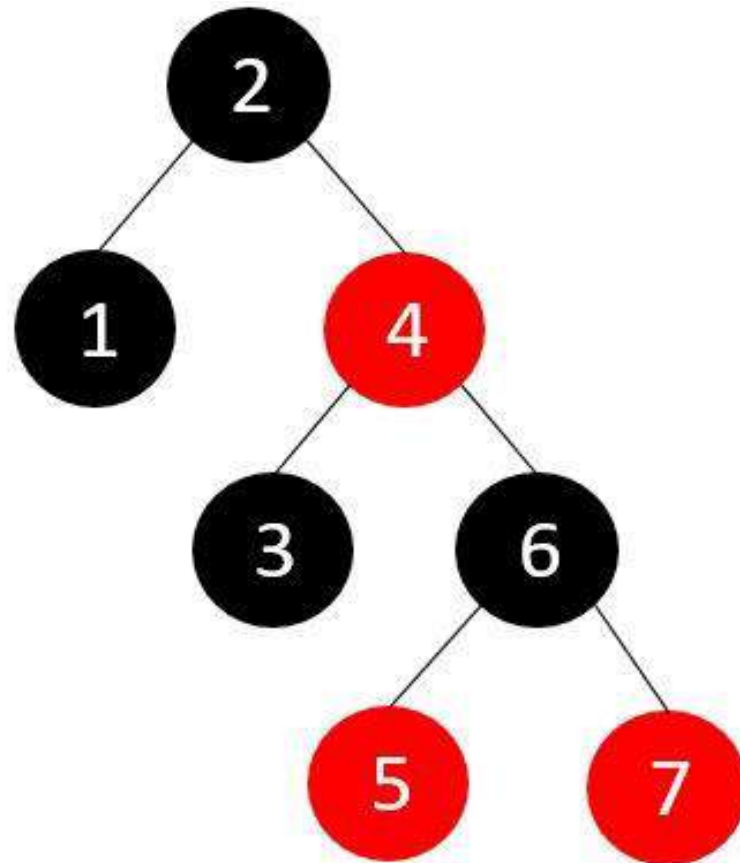


Example of RED-BLACK Tree



Example of RED-BLACK Tree

The final **Red** **Black** Tree with 1 to 7 Numbers



ADVANTAGES

- **Red Black** Trees have a guaranteed time complexity of $O(\log n)$ for basic operations like insertion, deletion, and searching.
- **Red Black** Trees are self-balancing.
- **Red Black** Trees can be used in a wide range of applications due to their efficient performance and versatility.
- The process which is used to maintain balance in **Red Black** Trees is relatively simple and easy to understand.

DISADVANTAGES

- A **red black** trees in a lot of cases is that it is a binary tree and thus lookups are $O(\log(n))$ where as hash tables have a lookup of $O(1)$.
- **Red Black** Trees require one extra bit of storage for each node to store the color of the node (red or black).
- Complexity of Implementation.
- Although **Red Black** Trees provide efficient performance for basic operations, they may not be the best choice for certain types of data or specific use cases.

EXAMPLE

1. Describe **RED** –**BLACK** tree with proper syntax and example
2. Construct the **RED-BLACK** tree with these elements
 - a) 10,85,15,70,20,60,30,50,65,80,90,40,5,55.
 - b) 10,18,7,15,16,30,25,40,60,2,1,70.
3. List out the time complexities of **RED** –**BLACK** tree based on different cases
4. Analyze the different cases of **RED-BLACK** tree operations and example

- The **Red-Black** tree properties ensure a balanced structure, limiting the longest path from the root to any leaf to at most twice the length of the shortest path.
- This balance guarantees efficient average-case performance for search, insertion, and deletion operations, with a worst-case time complexity of $O(\log n)$.

SELF-ASSESSMENT QUESTIONS

1. What is the color of the root node in a Red-Black Tree?

- A. Red
- B. Black
- C. Alternates between red and black
- D. It has no color

2. What is the worst-case time complexity for searching a key in a Red-Black Tree?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n \log n)$

3. Which self-balancing tree structure is a common alternative to Red-Black Trees?

- A. AVL Trees
- B. B-Trees
- C. Quad Trees
- D. Binary Heaps

TERMINAL QUESTIONS

1. What is the color of root in **red black** tree?
2. What is the color property of **red black** tree?
3. What is the color of null node?

TEXTBOOKS:

- 1) Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2010 , Second Edition, Pearson Education.
- 2) Ellis Horowitz, Fundamentals of Data Structures in C: Second Edition, 2015

REFERENCE BOOKS:

- A.V.Aho, J. E. Hopcroft, and J. D. Ullman, “Data Structures And Algorithms”, Pearson Education, First Edition Reprint 2003.
- Horowitz, Sahni, Anderson Freed, “Fundamentals of data structures in C” , Second Edition-2007.
- R. F. Gilberg, B. A. Forouzan, “Data Structures”, Second Edition, Thomson India Edition, 2005
- Robert Kruse, C.L. Tondo, Bruce Leung, Shashi Mogalla, “Data Structures & Program Design in C”, Fourth Edition-2007.

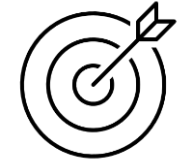
WEB REFERENCES/MOOCs:

- <https://nptel.ac.in/courses/106102064>
- <https://nptel.ac.in/courses/106101060/4>
- <https://www.edx.org/course/algorithms-and-data-structures-1>
- <https://in.udacity.com/course/intro-to-algorithms--cs215>
- <https://www.coursera.org/learn/data-structures?action=enroll>

THANK YOU

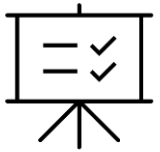
Advanced Algorithms & Data Structures

AIM OF THE SESSION



To familiarize students with the basic concept of Binary tree

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate The Binary tree.
2. Describe the types of Binary tree traversals.
3. Constructing Binary tree and traversals.
4. Constructing an expression tree.
5. List out the advantages and applications of Binary tree.

LEARNING OUTCOMES



At the end of this session, you should be able to:

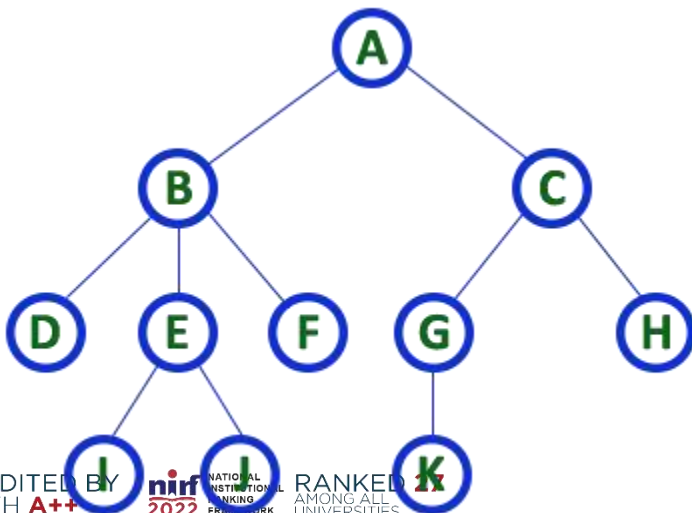
1. Define Binary tree
2. Describe the various traversals in Binary tree Expression tree construction
3. Summarize definition, types and operations of Binary tree and its applications

Binary tree

INTRODUCTION

➤ A binary tree is a special type of tree data structure in which every node can have a **maximum of 2 children**. One is known as a left child and the other is known as right child.

Example



TREE with 11 nodes and 10 edges

- In any tree with 'N' nodes there will be maximum of 'N-1' edges
- In a tree every individual element is called as 'NODE'

Tree - Terminology

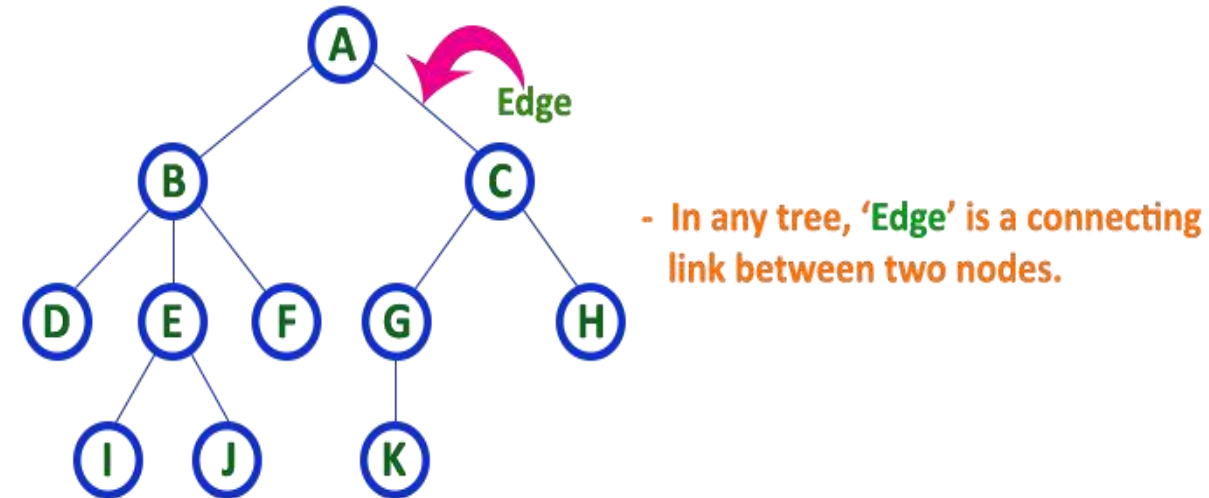
1. Root

In a tree data structure, the first node is called as **Root Node**.



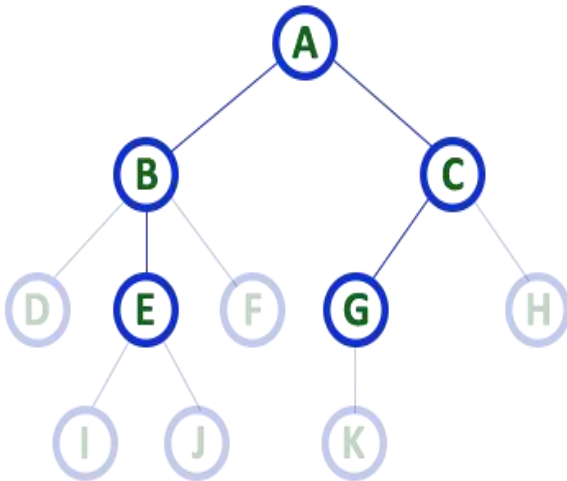
2. Edge

In a tree data structure, the connecting link between any two nodes is called as EDGE. In a tree with 'N' number of nodes there will be a maximum of 'N-1' number of edges.



Tree - Terminology

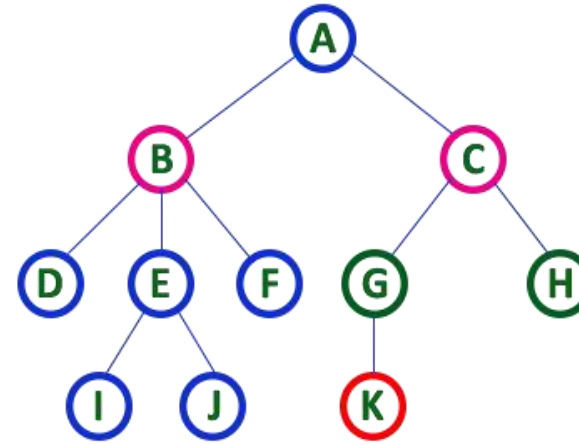
3. Parent



Here A, B, C, E & G are **Parent** nodes

- In any tree the node which has child / children is called '**Parent**'
- A node which is predecessor of any other node is called '**Parent**'

4. Child



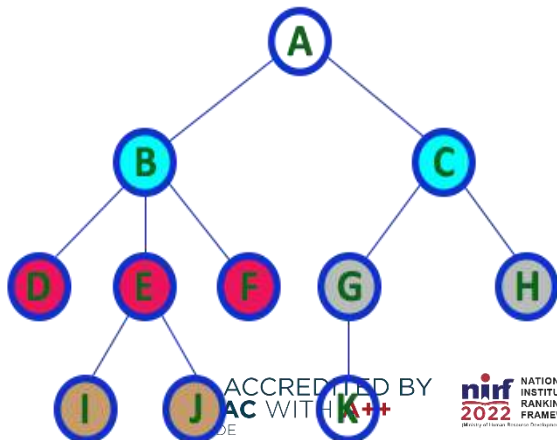
Here B & C are **Children** of A

Here G & H are **Children** of C

Here K is **Child** of G

- descendant of any node is called as **CHILD** Node

5. Siblings



Here B & C are **Siblings**

Here D, E & F are **Siblings**

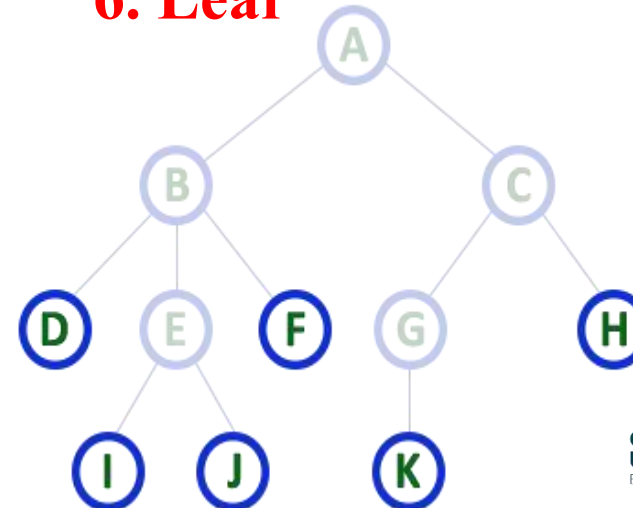
Here G & H are **Siblings**

Here I & J are **Siblings**

- In any tree the nodes which has same Parent are called '**Siblings**'

- The children of a Parent are called '**Siblings**'

6. Leaf

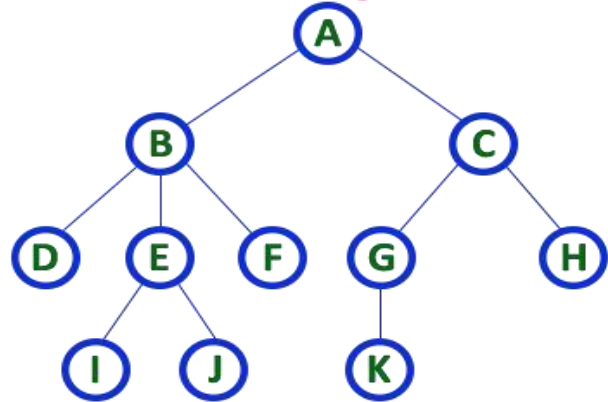


Here D, I, J, F, K & H are **Leaf** nodes

- In any tree the node which does not have children is called '**Leaf**'
- A node without successors is called a '**leaf**' node

Tree - Terminology

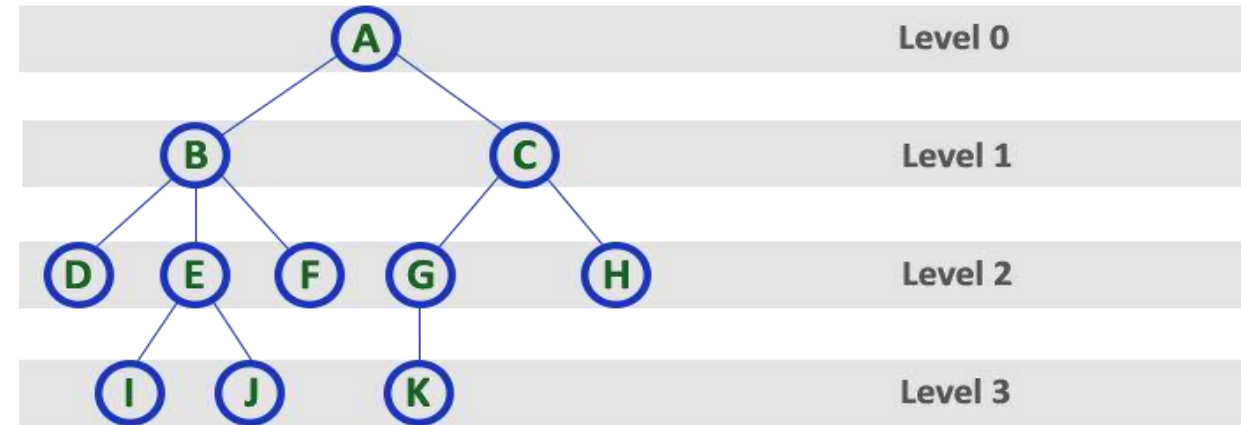
7. Degree



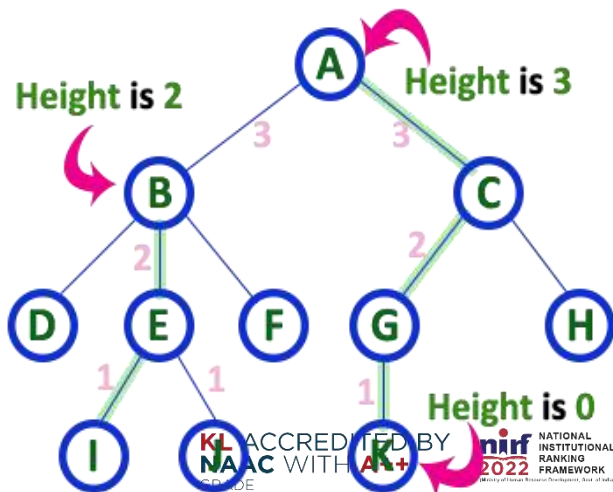
Here **Degree** of B is 3
Here **Degree** of A is 2
Here **Degree** of F is 0

- In any tree, '**Degree**' of a node is total number of children it has.

8. Level



9. Height

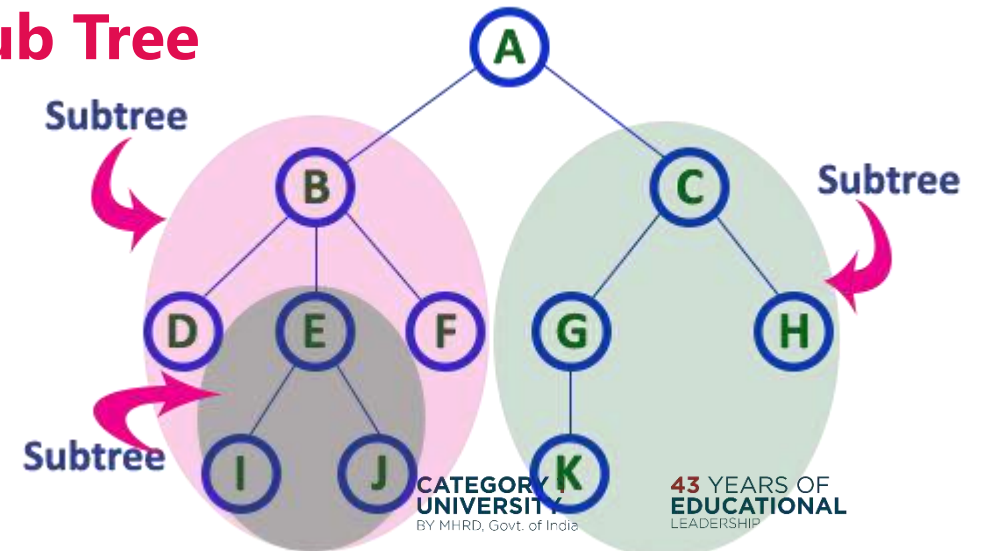


Here **Height** of tree is 3

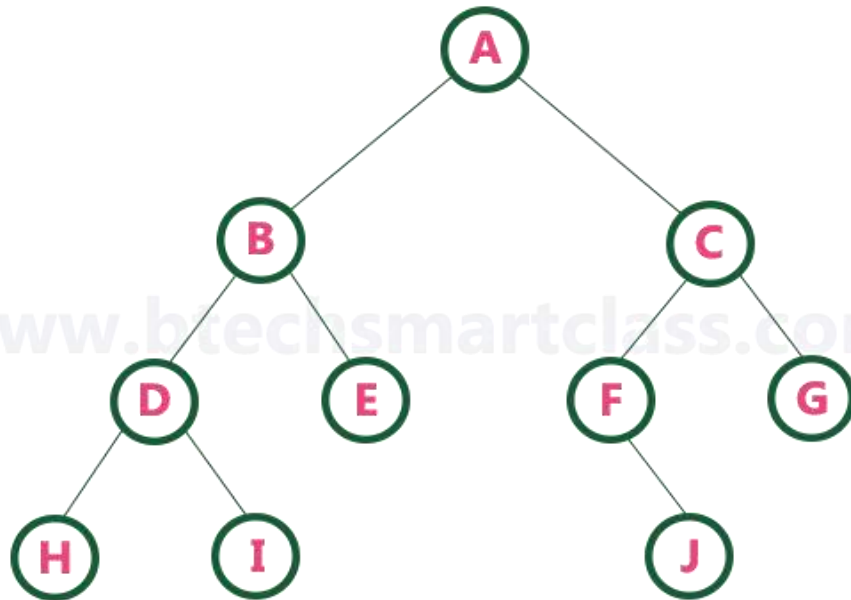
- In any tree, '**Height of Node**' is total number of Edges from leaf to that node in longest path.

- In any tree, '**Height of Tree**' is the height of the root node.

10. Sub Tree



Example of Binary Tree with nodes



Types of Binary Tree Representations

1. Array Representation of Binary Tree
2. Linked List Representation of Binary Tree

1. Array Representation of Binary Tree

In array representation of a binary tree, we use one-dimensional array (1-D Array) to represent a binary tree. Consider the above example of a binary tree and it is represented as follows...



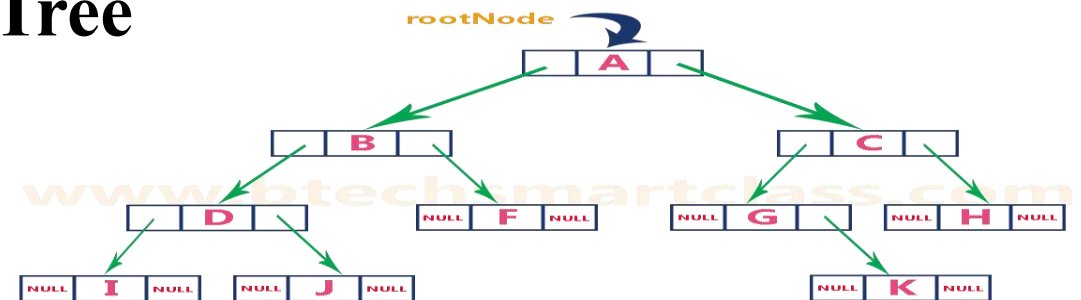
Node occupies $a[i]$

Left child $\rightarrow [2*i+1]$

Right child $\rightarrow [2*i+2]$

Parent child $\rightarrow [(i-1)/2]$ ex: 55, 44, 66, 33, 10, 20

2. Linked List Representation of Binary Tree



Types of binary tree traversals.

There are three types of binary tree traversals.

1.In - Order Traversal

2.Pre - Order Traversal

3.Post - Order Traversal

1.In - Order Traversal (leftChild - root - rightChild)

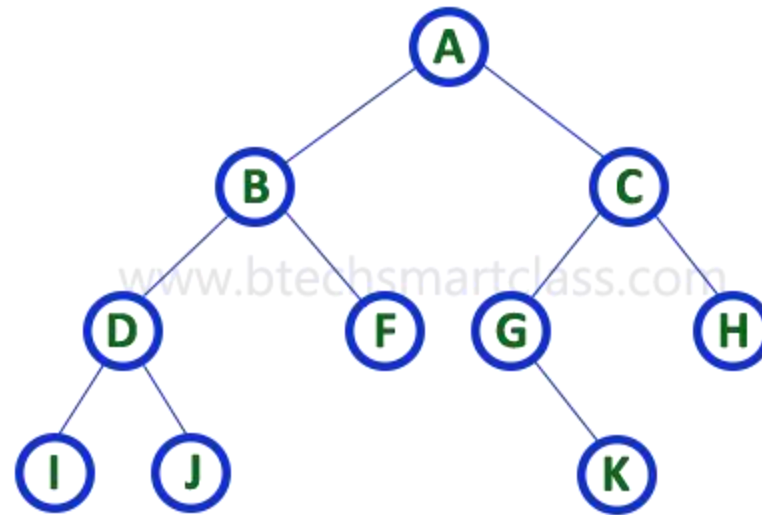
In In-Order traversal, the root node is visited between the left child and right child. In this traversal, the left child node is visited first, then the root node is visited and later we go for visiting the right child node. This in-order traversal is applicable for every root node of all subtrees in the tree. This is performed recursively for all nodes in the tree.

Example of In-Order Traversal

(Cont..)

In-Order Traversal for below example of binary tree is

I - D - J - B - F - A - G - K - C - H



---2nd Visit---

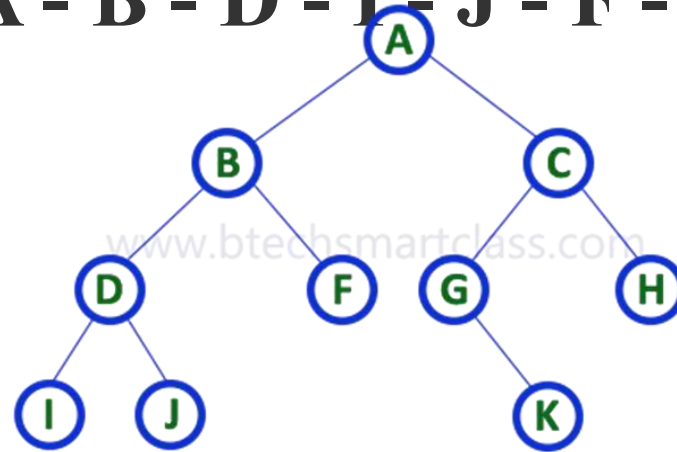
2. Pre - Order Traversal (root - leftChild - rightChild)

In Pre-Order traversal, the root node is visited before the left child and right child nodes. In this traversal, the root node is visited first, then its left child and later its right child. This pre-order traversal is applicable for every root node of all subtrees in the tree.

Example of Pre -Order Traversal

Pre-Order Traversal for below example binary tree is

A - B - D - I - J - F - C - G - K - H



---1st Visit---

- **3. Post - Order Traversal (leftChild - rightChild - root)** ----last visit----

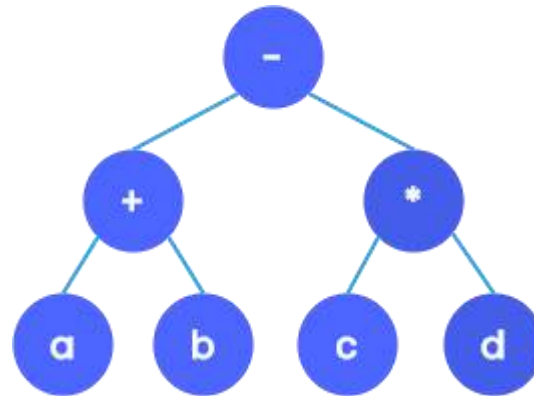
In Post-Order traversal, the root node is visited after left child and right child. In this traversal, left child node is visited first, then its right child and then its root node.

This is recursively performed until the right most node is visited.

Expression Tree

An expression tree is a special type of **binary tree** that is used to store algebraic expressions.

- Consider the algebraic expression given as: $X = (a + b) - (c * d)$. This can be represented using a binary tree as follows:

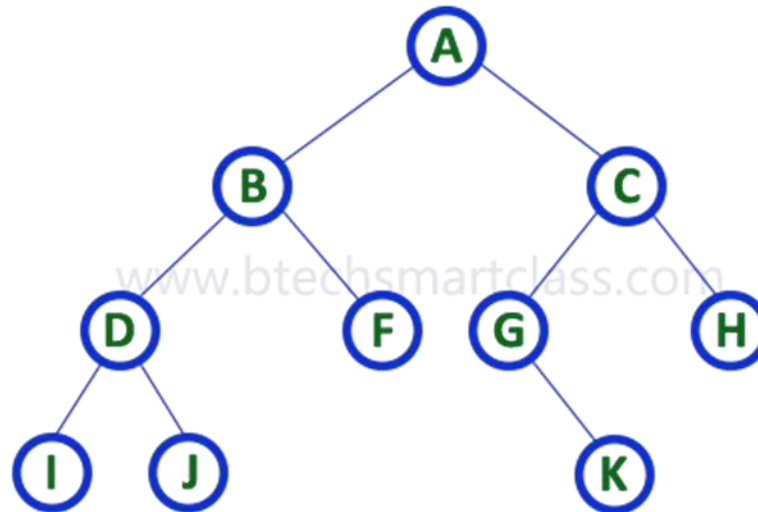


- The **in-order traversal** of the tree returns the **infix expression**.
- Similarly, the **pre-order** and **post-order** traversal of the expression tree will return **prefix** and **postfix** expression respectively.

Example of Pre -Order Traversal

Post-Order Traversal for above example binary tree is

I - J - D - F - B - K - G - H - C - A



Constructing an expression tree

- An expression tree can be constructed with the help of a **stack**.
- Consider the postfix expression $a\ b\ -\ c\ *$

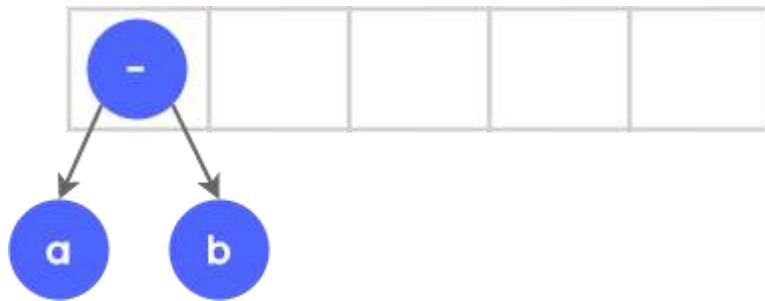
The following steps are done to create an expression tree.

- If the character is an operand, PUSH it on to the stack.
- If the character is an operator, POP two values from the stack and insert them as the child of operator. A pointer to the new root is pushed on to the stack.

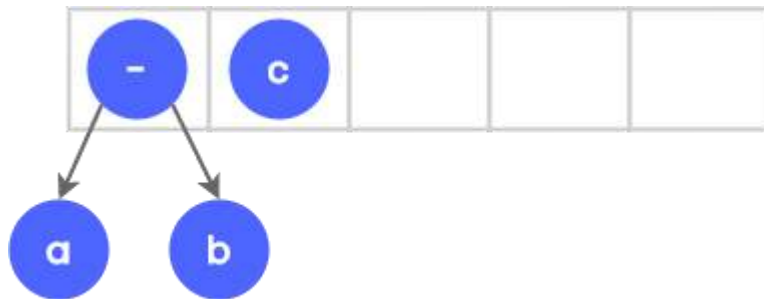
Since the first two characters are operands, PUSH it onto the stack.



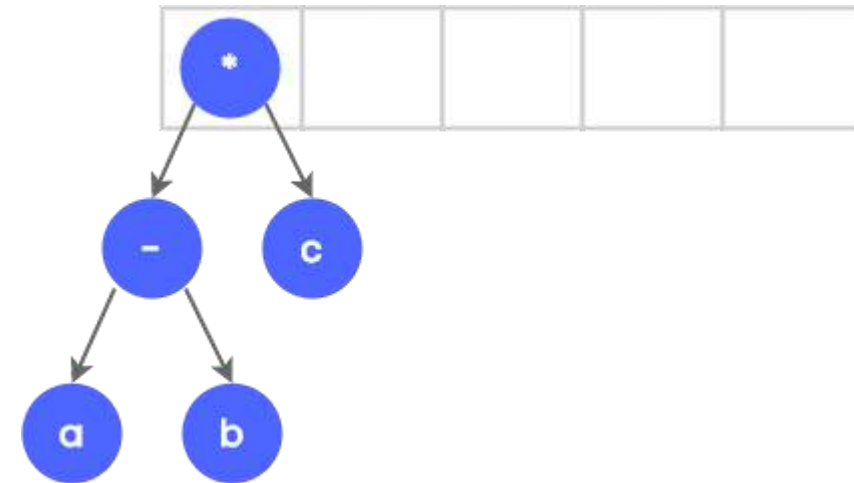
When the (-) symbol is read, a and b are popped from the stack and are added as the child of node -. A pointer to the new node is now pushed onto the stack.



- Now c is read and is pushed onto the stack.



Finally, the operator (*) is read, so we pop c and (-) from the stack and add it as the child of node *.



IMPORTANT FACTS RELATED TO THE SESSION

Advantages of a Binary Trees

- Binary trees are conceptually simple and easy to implement.
- Binary trees provide efficient search and retrieval operations.
- Traversing a binary tree is a well-defined process, and various algorithms (in-order, pre-order, post-order) allow for easy navigation and processing of all elements in the tree.

Disadvantages of Binary Trees

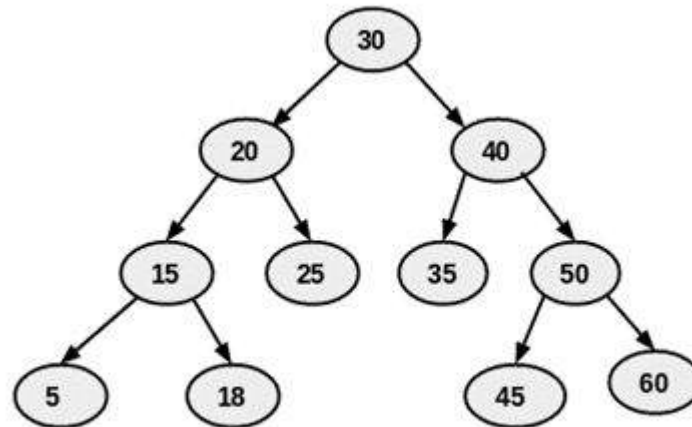
- Binary trees can have higher memory overhead compared to other data structures.
- Although binary trees are conceptually simple, ensuring proper balance and implementing self-balancing mechanisms can add complexity to the code.
- The performance of a binary search tree can be sensitive to the order in which elements are inserted.

Applications/Uses of Binary Trees in real life

- File systems in computers often use binary trees to represent the hierarchical structure of directories and file.
- Binary trees can represent organizational hierarchies in business structures
- Binary trees are used to represent mathematical expressions in compilers and evaluators

EXAMPLES

1. Write Inorder, Preorder, Post Order to below tree



2. Draw an Expression Tree to $A * B + C / D$.

SUMMARY

- Tree data structure is a collection of data (Node) which is organized in hierarchical structure recursively
- A tree in which every node can have a maximum of two children is called Binary Tree.
- Several terminologies are used in tree like follows..
root, parent, child, height, level, depth and subtree etc....
- In a Binary tree, we perform the following traversals
 - i) Inorder ii) Preorder iii) post order
- An expression tree is a special type of binary tree that is used to store algebraic expressions. In an expression tree, each internal node corresponds to the operator and each leaf node corresponds to the operand.
- Advantages, Disadvantages and Applications/Uses of Binary Trees in real life.

SELF-ASSESSMENT QUESTIONS

What is a tree in data structures?

- a. A hierarchical data structure
- b. A linear data structure
- c. A non-linear data structure
- d. A sequential data structure

In a binary tree, each node has at most how many children?

- a. 1
- b. 2
- c. 3
- d. 4

TERMINAL QUESTIONS

- 1. Define Binary Tree?**
- 2. Define the height of a binary tree?**
- 3. What is an in-order traversal of a binary tree?**
- 3. Define a leaf node in the context of a binary tree?**
- 4. Define the term "binary tree traversal."?**

Reference Books:

1. "Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2010 , Second Edition, Pearson Education.
2. 2. Ellis Horowitz, Fundamentals of Data Structures in C: Second Edition, 2015.
3. A.V.Aho, J. E. Hopcroft, and J. D. Ullman, “Data Structures And Algorithms”, Pearson Education, First Edition Reprint 2003.

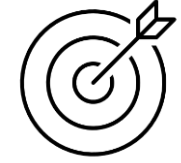
4. Sites and Web links:

1. <https://nptel.ac.in/courses/106102064>
2. <https://in.udacity.com/course/intro-to-algorithms--cs215>
3. <https://www.coursera.org/learn/data-structures?action=enroll>

THANK YOU

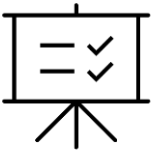
Advanced Algorithms & Data Structures

AIM OF THE SESSION



To familiarize students with the basic concept of Hashing

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate Hashing. Separate chaining
2. Describe the types of hashing
3. Describe each method

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Understand the fundamental concepts of hashing, including hash functions, hash tables
2. Learns different hashing techniques (e.g., Division Method, Mid Square Method, Digit Folding Method, Multiplication Method) and their suitability for different scenarios.
3. Apply hashing to solve practical problems involving efficient data retrieval and storage.

Introduction to Hashing

- Hashing is a technique or process of mapping keys, and values into the hash table by using a hash function.
- It is done for faster access to elements and minimizes the number of comparisons while performing the search.
- The efficiency of mapping depends on the efficiency of the hash function used.

Hashing Mechanism

- It's a technique for efficiently storing and retrieving data using a hash function.
- A hash function converts a key (like a name or a number) into an index, which is used to locate the corresponding value in a data structure called a hash table.
- Ideally, each key maps to a unique index, but collisions (multiple keys mapping to the same index) can occur and need to be resolved.

Introduction to Hashing

Hash Table

- It's the primary data structure used for hashing.
- It's typically an array of slots, where each slot can hold a key-value pair.
- The hash function determines which slot a key-value pair is placed into.

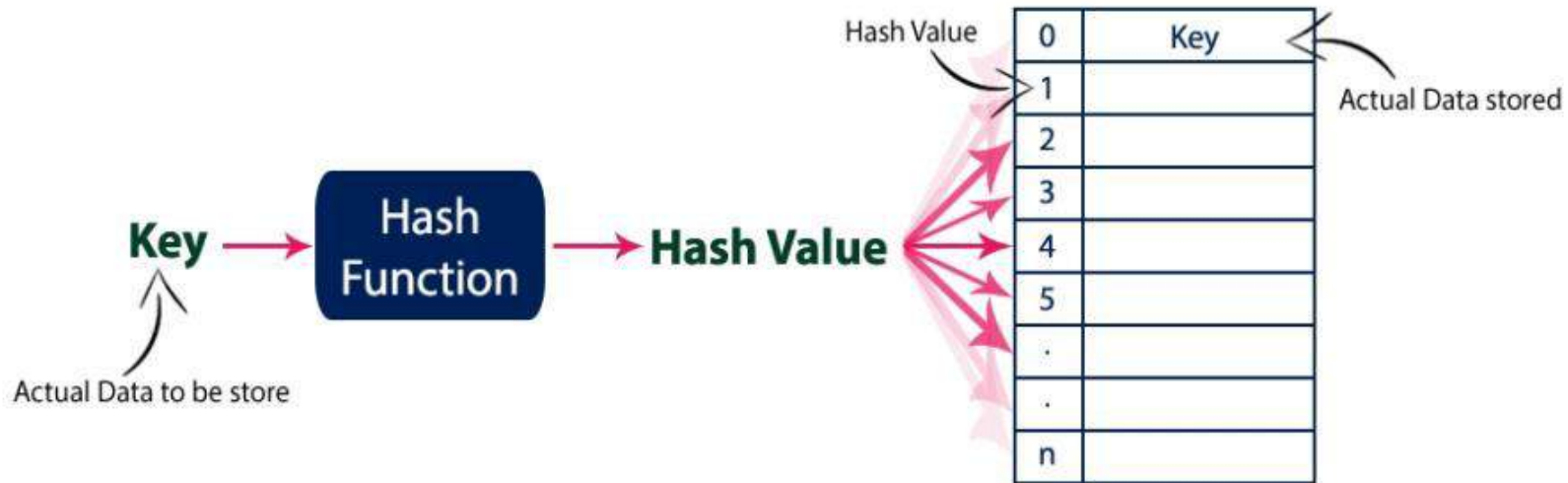
Hash Function

- It's the core component of hashing.
- It takes a key as input and produces a unique integer value (the hash value) that serves as the index in the hash table.
- Common hash functions include division-based, multiplication-based, and universal hashing.

Hash Key Value

- Hash key value is a special value that serves as an index for a data item.
- It indicates where the data item should be stored in the hash table.
- Hash key value is generated using a hash function.

Types of Hashing



Types of Hashing

1. Division Method
2. Mid Square Method
3. Digit Folding Method
4. Multiplicative Method

Division method

1. Division method

- It's a simple and common technique for creating hash functions.
- It involves dividing the key by a fixed integer value (usually the size of the hash table) and using the remainder as the hash value.
- The hash function for the Division method is:
$$h(\text{key}) = \text{Key} \% \text{Table size}.$$

Example: If the record 52,68,99,84 is to be placed in a hash table and the table size is 10.

- The hash function for the Division method is:
$$h(\text{key}) = \text{Key} \% \text{table size}.$$
$$h(52) = 52 \% 10 = 2$$
$$h(68) = 68 \% 10 = 8$$
$$h(99) = 99 \% 10 = 9$$
$$h(84) = 84 \% 10 = 4$$

0	
1	
2	52
3	
4	84
5	
6	
7	
8	68
9	99

Mid square method

2. Mid square method

- It's a technique for generating hash values.
- It involves squaring the key value and extracting a fixed number of digits from the middle of the squared result as the hash value.
- The hash function for the Mid Square method is:

$$h(\text{key}) = \text{Key} * \text{Key}$$

Example: Consider a Key = 12

The hash function for the Mid Square method is:

$$h(\text{key}) = \text{Key} * \text{Key}$$

$$H(12) = 12 * 12 = 144$$

Extract middle digit (hash table size = 10)

Hash value = 4

Store or retrieve data at index 4 in the hash table

Digit folding method

3. Digit folding method

- It's a hash function method that involves folding (dividing) a key into multiple parts, adding those parts, and using the result as the hash value.

Example: Consider a Key = 123456789 with a hash table size of 1000

Now, break the key into segments of equal length i.e., 123 | 456 | 789.

Sum these digits within each segment.

$$\begin{aligned} H(\text{key}) &= 123 + 456 + 789 \\ &= 1368 \end{aligned}$$

If the sum exceeds the hash table size, use modulo operation to fit it within the table's range.

In the example: $1368 \% 1000 = 368$ (the final hash value).

Multiplicative Method

4. Multiplicative Method

- Multiplicative hashing is a technique for mapping keys to indices in a hash table that uses multiplication and the fractional part of a number to achieve a good distribution of keys.
- The steps involved in the multiplication method are as follows:
 - Choose a constant A ($0 < A < 1$)
 - A common choice is the golden ratio ($A \approx 0.6180339887$)
 - Multiply the key k by A
 - Extract the fractional part of kA
 - Multiply the fractional part by the size of the hash table M
 - Take the floor of the result

Multiplicative Method

- The hash function for the Multiplicative method is:

$$h(k) = \text{floor} [m(kA \text{ mod size})]$$

Example: Consider a Key k: 23

Donald knuth suggested to use Constant A: 0.6180339887

Hash table size m: 10

key is 23

$$\begin{aligned} h(23) &= \text{floor}(10 * (23 * 0.618033)) \% 10 \\ &= \text{floor}(142.14) \% 10 \\ &= 142 \% 10 \\ &= 2 \end{aligned}$$

So key 23 is stored at location 2 of hash table.

Advantages of Hashing

- Hashing is extremely efficient.
- It completes the search with constant time complexity $O(1)$.

Disadvantages of Hashing

- Hash tables have a limited capacity and can fill up over time.
- Hash functions may not have the ability to navigate to the next or previous data set.

Applications/Uses of Hashing in real life

- Password storage: Hashing is used to store passwords securely by applying a hash function to the password and storing the hashed result, rather than the plain text password.
- Data compression: Hashing is used in data compression algorithms, such as the Huffman coding algorithm, to encode data efficiently.

EXAMPLES

1. Let us consider a simple hash function as “key mod 10” and sequence of keys as 40, 70, 76, 85, 92, 73, 100. Draw hash table and use separate chaining when collision occurs.
2. Construct a hash table for the given input sequence: 461, 137, 675, 197, 294, 965, 131 and hash function $H(k) = k \bmod 10$. Use separate chaining if collision occurs.

SELF-ASSESSMENT QUESTIONS

What is the hash function used in the division method?

- a) $h(k) = k/m$
- b) $h(k) = k \bmod m$
- c) $h(k) = m/k$
- d) $h(k) = m \bmod k$

2. Using division method, in a given hash table of size 157, the key of value 172 be placed at position

- a) 19
- b) 72
- c) 15
- d) 17

- Hashing is a powerful technique for fast data access and storage, but understanding its limitations and careful implementation are essential for optimal performance.
- It minimizes the number of comparisons while performing the search.
- It depends on the user, which hash function we want to use.

TERMINAL QUESTIONS

1. Define Hashing?
2. List out different types of Hashing Techniques?
3. Write a C program to implement hashing using separate chaining.

Assume that the has table has a size of 10

4. Construct hash table using separate chaining for the following elements of table size 15, design your own hash function.

23,27,98,6,17,51,77,72,36,53.

Reference Books

1. Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2010 , Second Edition, Pearson Education.
2. Ellis Horowitz, Fundamentals of Data Structures in C: Second Edition, 2015
3. A.V.Aho, J. E. Hopcroft, and J. D. Ullman, “Data Structures And Algorithms”, Pearson Education, First Edition Reprint 2003.

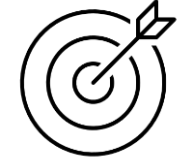
Sites and Web links:

1. <https://nptel.ac.in/courses/106102064>
2. <https://in.udacity.com/course/intro-to-algorithms--cs215>
3. <https://www.coursera.org/learn/data-structures?action=enroll>

THANK YOU

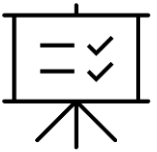
Advanced Algorithms & Data Structures

AIM OF THE SESSION



To familiarize students with the basic concept of Collision Resolution Techniques

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate Collision resolution techniques
2. Describe the types of collision resolution techniques
3. Describe each method

.

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define Collision resolution techniques
2. Describe types of collision resolution techniques.
3. Summarize definition, types of resolution techniques and its applications

Introduction To Collision Resolution Techniques

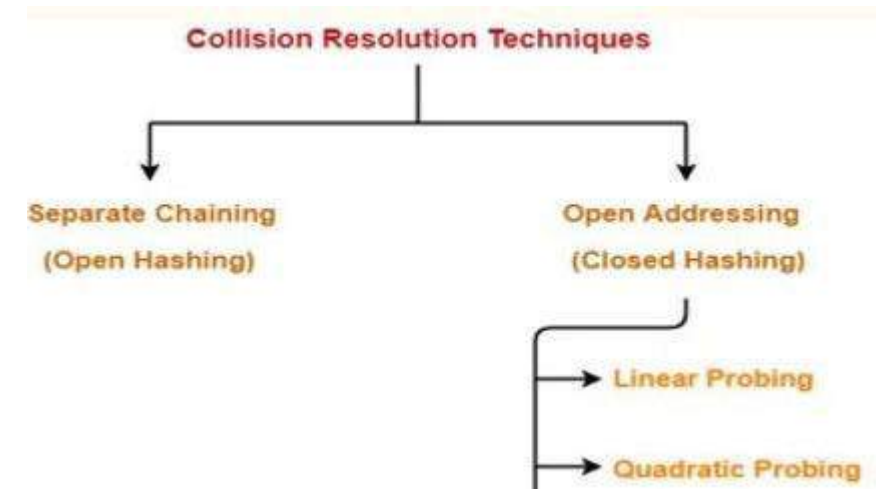
- Collision occurs when multiple keys are mapped to the same index (slot) in a hash table by the hash function.
- For example, several students having the same locker number in a school, leading to potential conflicts for storage and retrieval.
- The collision creates a problem because each index in a hash table is supposed to store only one value.
- There are several collision resolution techniques to manage the performance of a hash table. The two most popular techniques are:

1. Open Hashing

- Separate Chaining

2. Closed Hashing or Open Addressing

- Linear Probing
- Quadratic Probing



Introduction To Collision Resolution Techniques

Open Hashing

- Each Bucket in the Hash table is the head of a Linked List.
- All Elements that hash to a Particular Bucket are Placed on the Buckets Linked List .

Closed Hashing

- Ensures that all elements are stored directly into the Hash Table.

Separate Chaining

- It is a collision resolution technique that handles collisions in hash tables by storing colliding keys in linked lists.
- Instead of having a single value at each slot, each slot holds a linked list (or chain) of keys that have hashed to the same index.
- The hash function for the Separate Chaining is:
 $h(\text{key}) = \text{key} \% \text{table_size}$.

Example: Consider a sequence of keys as 50, 700, 76, 85, 92, 73, 101 with table_size 7.

$$h(\text{key}) = \text{key} \% \text{table_size}$$

$$h(50) = 50 \% 7 = 1$$

$$h(92) = 92 \% 7 = 1$$

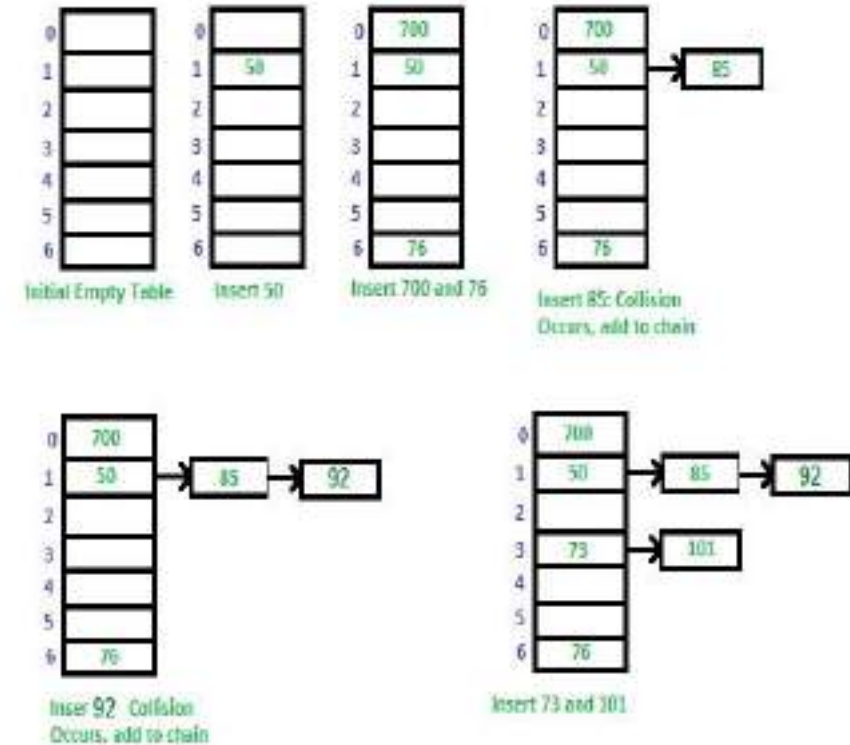
$$h(700) = 700 \% 7 = 0$$

$$h(73) = 73 \% 7 = 3$$

$$h(76) = 76 \% 7 = 6$$

$$h(101) = 101 \% 7 = 3$$

$$h(85) = 85 \% 7 = 1$$



Linear Probing

- It's an open addressing collision resolution technique.
- When a collision occurs, it probes sequentially for the next available slot in the hash table.
- The hash function for the Linear Probing to an initial index:
$$h(\text{key}) = \text{Key} \% \text{Table_size}.$$
- If a collision occurs at the any index, linearly probe for the next available slot:
$$\text{probe_index}(\text{key}) = (\text{key} + i) \% \text{Table_size}$$

Where, i is the probe count, starting from 1 and incrementing for each failed attempt.

Working Mechanism

- **Calculate the hash value:** Use a hash function to determine the initial slot for a key.
- **Check for collision:** If the slot is occupied, start linear probing:
 - Sequentially check the next available slot in the table.
 - Wrap around to the beginning if you reach the end.
- **Insert or search:** Once an empty slot is found, insert the key or search for it.

Linear Probing

Example: Consider a sequence of keys as 71, 64, 56, 36 with table size 10

- The hash function for the Division method is:

$$h(\text{key}) = \text{Key} \% \text{table size.}$$

$$h(71) = 71 \% 10 = 1$$

$$h(64) = 64 \% 10 = 4$$

$$h(56) = 56 \% 10 = 6$$

$$h(36) = 36 \% 10 = 6$$

Now, collision occurs at the index 6

$$\text{probe_index}(\text{key}) = (\text{key} + i) \% \text{Table_size}$$

Initially

$$\text{probe_index}(36) = (36 + 1) \% 10 = 7$$

Now, key 36 is placed at index 7

0	NULL
1	71
2	NULL
3	NULL
4	64
5	NULL
6	56
7	36
8	NULL
9	NULL

$i=1$

Quadratic Probing

- It's a collision resolution technique used in hash tables that employ open addressing.
- When a collision occurs (two keys hash to the same index), quadratic probing probes a sequence of alternate hash table indices using a quadratic function.
- The hash function for the Quadratic Probing to an initial index:

$$h(\text{key}) = \text{Key} \% \text{Table_size}.$$

- If a collision occurs at the any index, linearly probe for the next available slot:

$$\text{Quaprobe_index}(\text{key}) = (\text{key} + i*i) \% \text{Table_size}$$

Where, i is the probe count, starting from 1 and incrementing for each failed attempt.

Quadratic Probing

Example: Consider a sequence of keys as 67, 90, 55, 49, 17 with table size 10

The hash function for the Division method is:

$$h(\text{key}) = \text{Key} \% \text{table size.}$$

$$h(67) = 67 \% 10 = 7$$

$$h(90) = 90 \% 10 = 0$$

$$h(55) = 55 \% 10 = 5$$

$$h(49) = 49 \% 10 = 9$$

$$h(17) = 17 \% 10 = 7$$

Now, collision occurs at the index 7

$$\text{Quaprobe_index}(\text{key}) = (\text{key} + i*i) \% \text{Table_size}$$

Initially

$$\text{probe_index}(17) = (17 + 1*1) \% 10 = 8$$

Now, key 17 is placed at index 8

0	90
1	
2	
3	
4	
5	55
6	
7	67
8	17
9	49

i=1

Advantages of a Collision Resolution Techniques

- Enable efficient hash table operations (insertion, deletion, search) in average cases, often achieving $O(1)$ time complexity.
- Allow for flexible hash table usage, even when collisions occur.

Disadvantages

- Can degrade performance if the hash table becomes too full, potentially leading to worst-case $O(n)$ time complexity.
- Require careful selection of a good hash function to minimize collisions and ensure even key distribution.

Applications

- Symbol tables
- Caching

EXAMPLES

1. Let us consider a simple hash function as “key mod 5” and a sequence of keys that are to be inserted are 50, 70, 76, 93. Draw hash table using Linear probing technique.
2. Let us consider table Size = 7, hash function as $\text{Hash}(x) = x \% 7$ and collision resolution strategy to be $f(i) = i^2$. Insert = 22, 30, and 50.

SELF-ASSESSMENT QUESTIONS

1. A hash table of length 10 uses open addressing with hash function $h(k) = k \bmod 10$, and linear probing. After inserting 6 values into an empty hash table, the table is as shown below.

Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

- (A) 46, 42, 34, 52, 23, 33
- (B) 34, 42, 23, 52, 33, 46
- (C) 46, 34, 42, 23, 52, 33
- (D) 42, 46, 33, 23, 34, 52

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

2

2. Which of the following schemes does quadratic probing come under?

- a) rehashing
- b) extended hashing
- c) separate chaining
- d) open addressing

- Collision resolution techniques are crucial in hash tables to effectively deal with collisions occurring when different keys map to the same index.
- Each technique has its own advantages and disadvantages, making the choice depend on specific needs and priorities.

1. List out different types of Collision Resolution Techniques?
2. The keys 12,18,13,2,3,23,5 and 15 are inserted into an initially empty hash table of length 10 using open addressing with hash function $h(k)=k \bmod 10$ and linear probing show the resultant hash table?
3. Define collision

Reference Books:

1. Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2010 , Second Edition, Pearson Education.
2. 2. Ellis Horowitz, Fundamentals of Data Structures in C: Second Edition, 2015
3. A.V.Aho, J. E. Hopcroft, and J. D. Ullman, “Data Structures And Algorithms”, Pearson Education, First Edition Reprint 2003.

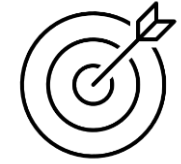
Sites and Web links:

1. <https://nptel.ac.in/courses/106102064>
2. <https://in.udacity.com/course/intro-to-algorithms--cs215>
3. <https://www.coursera.org/learn/data-structures?action=enroll>

THANK YOU

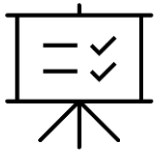
Advanced Algorithms & Data Structures

AIM OF THE SESSION



To familiarize students with the basic concept of Double Hashing, Extendible hashing, Rehashing

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate Double Hashing, Extendible hashing, Rehashing
2. List out the advantages and applications of hashing techniques.

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define Double Hashing, Extendible hashing, Rehashing.
2. Describe each method.
3. Summarize definition, types of hashing and its applications

Double hashing

- Double hashing is a collision resolution technique that utilizes two hash functions to handle collisions when multiple keys map to the same index in a hash table.
- First hash function is typically **$\text{hash1}(\text{key}) = \text{key} \% \text{TABLE_SIZE}$**
- Whenever collision occurs, second hash function is
 $\text{hash2}(\text{key}) = \text{PRIME} - (\text{key} \% \text{PRIME})$
where PRIME is a prime smaller than the TABLE_SIZE.
- Then calculate **$h(\text{key}, i) = (\text{hash1}(\text{key}) + i * \text{hash2}(\text{key})) \% \text{tableSize}$**
Where, i is the probe count, starting from 1 and incrementing for each failed attempt.

Double Hashing

Example: Consider a sequence of keys as 20, 34, 45, 70 with table_size 11

- $h_1(20) = 20 \bmod 11 = 9$
- $h_1(34) = 34 \bmod 11 = 1$
- $h_1(45) = 45 \bmod 11 = 1$

Collision Occurs at index 1, so calculate second hash function

$$h_2(45) = 7 - (45 \bmod 7) = 3$$

Now, calculate $h(45, 1) = (1 + 1 * 3) \bmod 11 = 4$, where $i=1$

- $h_1(70) = 70 \bmod 11 = 4$

Collision Occurs at index 4, so calculate second hash function

$$h_2(70) = 7 - (70 \bmod 7) = 7$$

$$h(70, 1) = (4 + 1 * 7) \bmod 11 = 1$$

Again, Collision Occurs, increment i , $h(70, 2) = (1 + 2 * 7) \bmod 11 = 4$, where $i=2$

Again, Collision Occurs, increment i , $h(70, 3) = (1 + 3 * 7) \bmod 11 = 0$, where $i=3$

0	
1	34
2	
3	56
4	45
5	
6	70
7	
8	
9	20
10	

Rehashing:

- Rehashing method is used to prevent collisions by increasing the size of a hashmap and redistributing the elements into new buckets based on their new hash values.

How Rehashing is done?

- For each new entry to the map, check the load factor.
- If it's greater than its pre-defined value (or default value of 0.75 if not given), then Rehash.
- For Rehash, make a new array of double the previous size and make it the new bucket array.
- Then traverse to each element in the old bucket Array and call the insert() for each so, as to insert it into the new larger bucket array.

Load factor can be calculated by using the following formula:

- The initial capacity of the HashMap * Load factor of the HashMap.
- Good load factor value is 75, while HashMap's default beginning capacity is 16, when the number of elements hits or exceeds 0.75 times the capacity, the complexity rises.
- To combat this, the array's size is doubled, and all the values are hashed again and saved in the new double-sized array.
- After Rehashing items can fall into same bucket or different bucket into new array.

Ex: when the number of items reaches 12, rehashing begins. (as $12 = 0.75 * 16$).

- Extendible Hashing is a dynamic hashing method wherein directories, and buckets are used to hash data. It is an aggressively flexible method in which the hash function also experiences dynamic changes.

Frequently used terms in Extendible Hashing

- Directories:** Instead of using a single hash table, extendible hashing employs a multi-level directory structure.
- Buckets:** Each directory entry points to a bucket, which stores a subset of key-value pairs.
- Local Depth:** Each bucket has a local depth, indicating the number of bits from the hash value used for addressing within that bucket.
- Global Depth:** The directory's global depth determines the maximum number of bits used for addressing.
- Bucket Splitting:** When the number of elements in a bucket exceeds a particular size, then the bucket is split into two parts.
- Directory Expansion:** Directory Expansion Takes place when a bucket overflows. Directory Expansion is performed when the local depth of the overflowing bucket is equal to the global depth.

Example: Consider a prominent example of hashing the following elements: 16,4,6,22,24,10,31,7,9,20,26 with bucket Size 3.

Hash Function: Suppose the global depth is X . Then the Hash Function returns X LSBs.

First, calculate the binary forms of each of the given numbers.

- 16- 10000
- 4- 00100
- 6- 00110
- 22- 10110
- 24- 11000
- 10- 01010

31- 11111

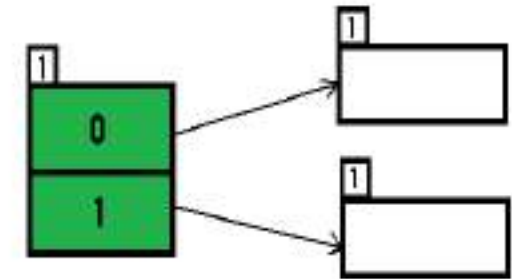
7- 00111

9- 01001

20- 10100

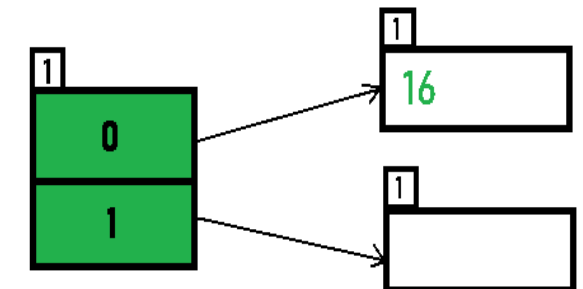
26- 11010

- Initially, the global-depth and local-depth is always 1. Thus, the hashing frame looks like this:



Inserting 16:

- The binary format of 16 is 10000 and global-depth is 1. The hash function returns 1 LSB of 10000 which is 0. Hence, 16 is mapped to the directory with id=0.

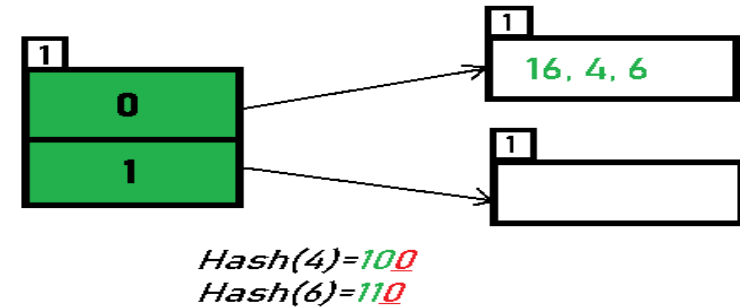


$Hash(16) = 1000\underline{0}$

Extendible Hashing

- Inserting 4 and 6:

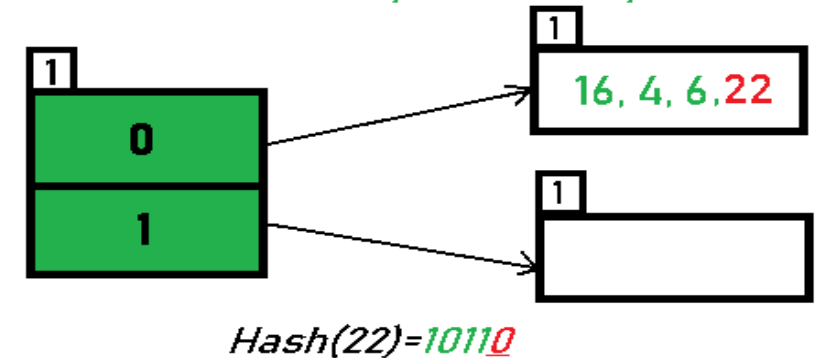
Both 4(100) and 6(110) have 0 in their LSB. Hence, they are hashed as follows:



- Inserting 22: The binary form of 22 is 10110. Its LSB is 0. The bucket pointed by directory 0 is already full. Hence, overflow occurs.

Overflow Condition

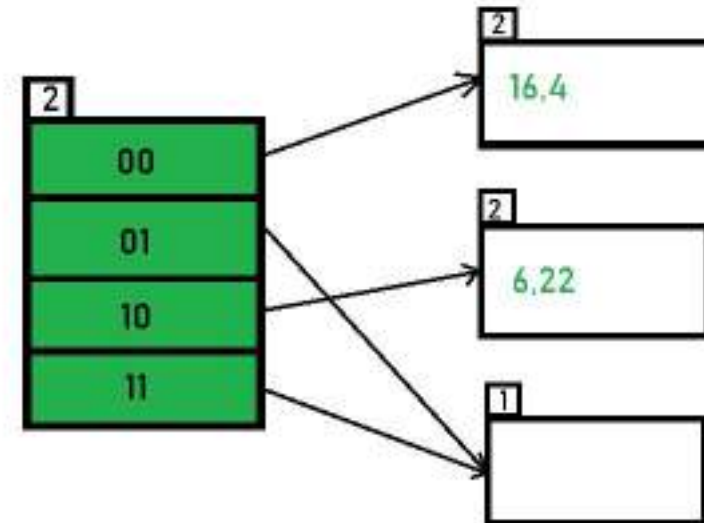
Here, Local Depth=Global Depth



Extendible Hashing

Since Local Depth = Global Depth, the bucket splits and directory expansion takes place. Also, rehashing of numbers present in the overflowing bucket takes place after the split. And, since the global depth is incremented by 1, now, the global depth is 2. Hence, 16,4,6,22 are now rehashed w.r.t 2 LSBs.
 [16(10000),4(100),6(110),22(10110)]

After Bucket Split and Directory Expansion



Advantages of Double hashing

- It can reduce clustering by using a second hash function to determine the next probe location.
- It can handle a larger number of collisions than linear probing.

Disadvantages of Double hashing

- It can be more complex to implement and understand than linear or quadratic probing

Advantages of Rehashing

- Handling collisions: As mentioned above, rehashing is primarily used to handle collisions in hash tables. ...
- Improving performance: Rehashing can also be used to improve the performance of hash tables

Advantages of Extendible Hashing

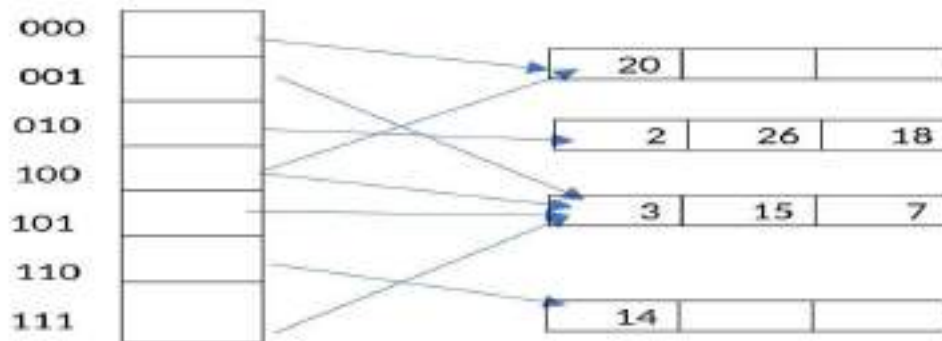
- Data retrieval is less expensive (in terms of computing).
- Very flexible in choosing size of bucket and allows storage on disks/remote memory access.

Disadvantages of Extendible Hashing

- Increased algorithm complexity
- Extra memory overhead to store index inside the bucket.

EXAMPLES

- Consider an extendible hashing structure that, each bucket can hold up to 2 records. Is initially empty (only one empty bucket). Consider the result after inserting keys 8, 16, 4, 3, 11, 12 in order, using the lowest bits for the hash function. That is records in a bucket of local depth d agree on their rightmost d bits. For example, key 4 (0100) and key 12(1100) agree on their rightmost 3 bits (100). What is the global depth of the resulting directory? Now insert key 18. What is the local depth of the bucket that contains key 18?



Applications of rehashing in real life

- Password Verification
- Data Structures
- File System.

Applications of Extendible Hashing

- All modern file systems use extendible hashing.

Applications of Double Hashing

- To prevent the collision of two keys ,the idea of Double Hashing is used. In case any collision occurs when we just use traditional hash code evaluating function, another hash code is generated by some other function and both the hash codes are used to find a free space by probing through array elements.

- **Double hashing:** It is a technique in which two hash function are used when there is an occurrence of collision.
- The hash functions for this technique are:
$$H1(\text{key}) = (\text{key} \% \text{table size} + i * H2(\text{key})) \% \text{table size}$$

Whereas $H2(\text{key}) = P - (\text{key} \bmod P); i = 0, 1, \dots, \text{size}$
- **Rehashing** is the process of increasing the size of a hashmap and redistributing the elements to new buckets based on their new hash values. It is done to improve the performance of the hashmap and to prevent collisions caused by a high load factor.
- **Extendible Hashing** is a dynamic hashing method wherein directories, and buckets are used to hash data. It is an aggressively flexible method in which the hash function also experiences dynamic changes.
- Number of Directories = $2^{\text{Global Depth}}$.

SELF-ASSESSMENT QUESTIONS

1. What is the hash function used in Double Hashing?

- a) $(h1(k) - i * h2(k)) \bmod m$
- b) $h1(k) + h2(k)$
- c) $(h1(k) + i * h2(k)) \bmod m$
- d) $(h1(k) + h2(k)) \bmod m$

2. Double hashing is one of the best methods available for open addressing

- a) True
- b) False

1. What is Load factor?
2. When is rehashing done?
3. What is local depth, global depth?

Reference Books:

1. Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2010 , Second Edition, Pearson Education.
2. 2. Ellis Horowitz, Fundamentals of Data Structures in C: Second Edition, 2015
3. A.V.Aho, J. E. Hopcroft, and J. D. Ullman, “Data Structures And Algorithms”, Pearson Education, First Edition Reprint 2003.

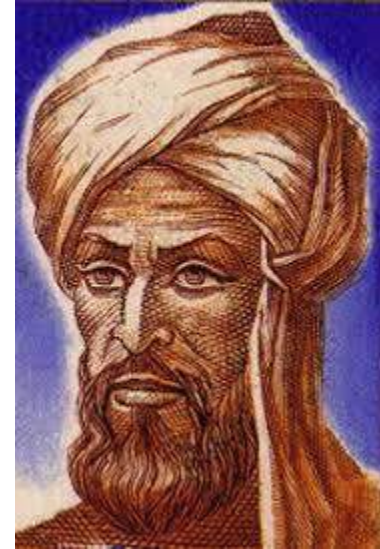
Sites and Web links:

1. <https://nptel.ac.in/courses/106102064>
2. <https://in.udacity.com/course/intro-to-algorithms--cs215>
3. <https://www.coursera.org/learn/data-structures?action=enroll>

THANK YOU

Advanced Algorithms & Data Structures

Muhammad ibn Musa - Khwarizmi Father of Algorithm



Definition:

An *algorithm* is a finite set of instructions that accomplishes a particular task.

All algorithms must satisfy the following criteria.

Characteristics:

1. Input : Zero or more quantities are externally supplied.
2. Output : At least one quantity is produced.
3. Definiteness : Each instruction is clear and unambiguous.

Statements such as “add 6 or 7 to x” is not permitted”

4. Finiteness : The algorithm should terminate after a finite number of steps.
5. Effectiveness : Instruction is basic enough to be carried out.

- Algorithms that are effective and definite are called computational procedures.

Example : The operating system of a digital computer.

- To achieve the criterion of definiteness algorithm are written in a programming language.
- A program is the expression of an algorithm in a programming language.

Areas for the study of algorithms:

1. How to devise an algorithm
2. How to validate an algorithm
3. How to analyze an algorithm
4. How to test a program
 - Debugging
 - Profiling

1. How to devise an algorithm

To design new algorithm, we apply a existing design strategy.

Some of the existing strategies are

Divide and Conquer

Greedy Method

Dynamic Programming

Back Tracking

Branch and Bound etc.

2. How to validate an algorithm

To check whether the algorithm giving correct output or not for all possible inputs.

3. How to analyze an algorithm

Based on performance measures such as space and time

- i) How much time required
- ii) How much space required

4. How to test a program

- Debugging
- Profiling

Debugging is a process of executing program on sample data sets, to determine whether any fault results occur. If so correct them.

Profiling is a process of executing a correct program on sample data sets and determine how much time/space required by program

Representation and Structure of an Algorithm:

We can describe an algorithm in many ways.

- ✓ We can use a natural language like English, but if we select this option, we must make sure that the resulting instructions are definite.
- ✓ Graphic representations called flow charts are another possibility, but they work well only if the algorithm is small and simple.
- ✓ We can also represent algorithms using pseudo code that resemble C and Pascal.

We present most of our algorithms using pseudo code that resembles C and Pascal.

1. Comments begin with // and continue until end of the line.
2. Blocks are indicated with matching braces: { and }.
 - i. A compound statement
 - ii. Body of a procedure.
3.
 - i. An identifier begins with a letter.
 - ii. The data types of variables are not explicitly declared.
 - iii. The types will be clear from the context.
 - iv. Whether a variable is global or local to a procedure will also be evident from the context.
 - v. We assume simple data types such as integer, float, char, boolean, and so

4. Assignment of values to variables is done using the assignment statement.

$\langle \text{variable} \rangle := \langle \text{expression} \rangle$

5. There are two boolean values **true** and **false**. To produce these values, logical operators **and**, **or** and **not** and the relational operators $\langle, \leq, =, \neq, \geq$ and $\rangle$ are provided.

6. Elements of multidimensional arrays are accessed using $[$ and $]$. For example the (i,j) th element of the array A is denoted as $A[i,j]$.

7. The following looping statements are used: while, for and repeat until.

i) The general form of a **while** loop:

while(condition) **do**

```
{  
    statement_1;  
    :  
    statement_n;  
}
```

ii) The general form of a **for** loop:

for *variable* := *value1* **to** *value2* **step** *step* **do**

iii) The general form of a repeat-until loop:

repeat

<statement 1>

:

<statement n>

until (condition)

The statements are executed as long as condition is false

8. A conditional statement has the following forms:

if < condition > then < statement >

if < condition > then

< statement 1>

else

< statement 2>

9. Input and output are done using the instructions read and write.

Ex: read n;

write n;

10. Procedure or function starts with the word **Algorithm.**

General form :

```
Algorithm Name( <parameter list> )  
{  
    body  
}
```

where *Name* is the name of the procedure.

Simple variables to procedures are passed by value.

Arrays and records are passed by reference

Write an algorithm to find sum of n numbers in the given list

Algorithm sum(a,n)

{

 s:=0;

 for i:=1 to n do

 s:=s+a[i];

 return s;

}

Write an algorithm to find factorial of a given number.

Algorithm factorial(n)

```
{  
    if n < 0 then  
        return "Invalid input";  
    if n = 0 or n = 1 then  
        return 1;  
    else  
        return n * factorial(n-1);  
}
```

SAMPLE QUESTIONS

- Explain the Importance of Algorithm to Solve Real World Problems
- Summarize the Characteristics of Algorithm
- What is the difference between an algorithm and a program
- Describe the concept of recursion in algorithms. Provide an example algorithm that uses recursion
- What is an algorithm? Provide a brief definition and explain its importance in computer science.
- What is effectiveness in algorithms
- What is finiteness in algorithms
- In how many ways can we describe an algorithm
- What are the steps included in writing an algorithm
- How do we validate an algorithm
- How to Analyse an algorithm

Advanced Algorithms & Data Structures

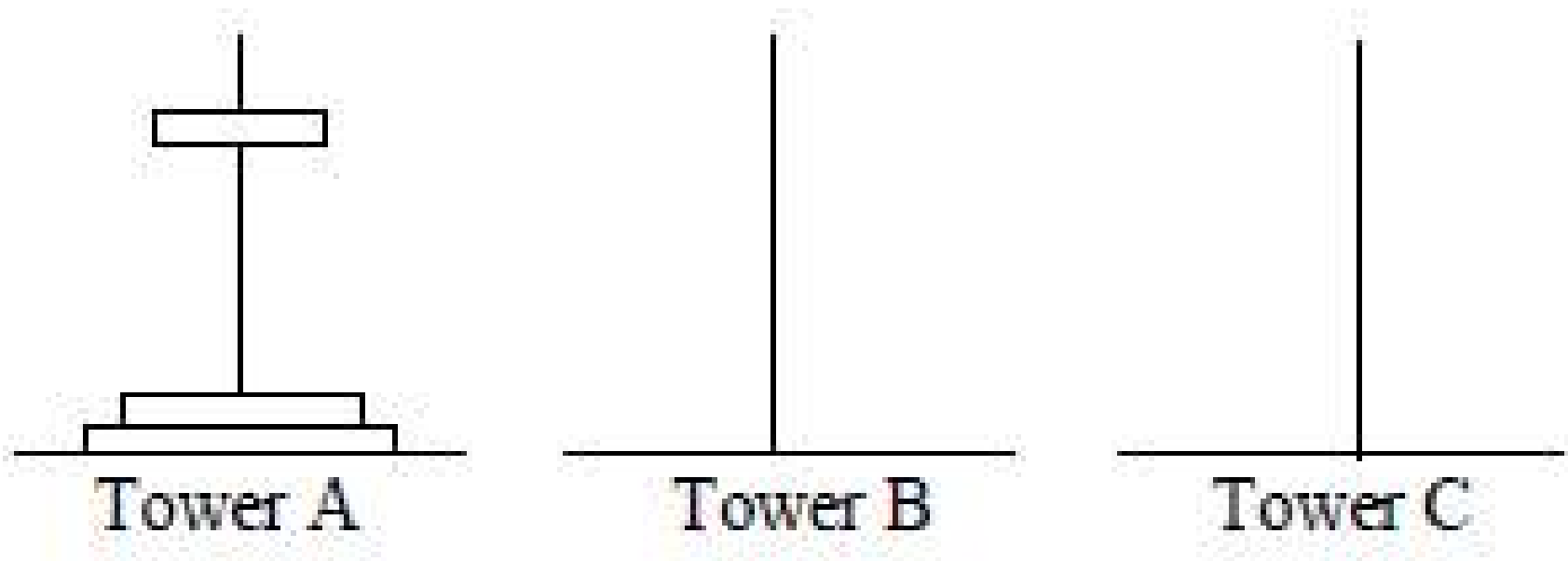
Session -12

Performance Analysis of recursive and non-recursive algorithms

Recursive Algorithms:

- A recursive function is a function that is defined in terms of itself. Similarly an algorithm is said to be recursive if the same algorithm is invoked in the body. An algorithm that calls itself is **direct recursive**.
- Algorithm A is said to be **indirect recursive** if it calls another algorithm which in turn calls A. These recursive mechanisms are extremely powerful, and also it can express a complex process very clearly.

Example: Towers of Hanoi problem



Algorithm **TowersOfHanoi**(n, x, y, z)

{

 If ($n \geq 1$) then

 {

 TowersOfHanoi(n-1, x, z, y);

 Write("move top disk from tower" , x, "to
top of tower", y);

 TowersOfHanoi(n-1, z, y, x);

 }

}

Performance Analysis:

There are two major criteria for judging algorithms that have a more direct relationship to performance.

computing time(Time complexity)

storage requirement (Space complexity)

Time Complexity :

Is the amount of time it needs to run to completion

- The time, $T(P)$, taken by a program P , is the sum of its compile time C and its run (or execution) time, $T_p(I)$.

$$T(P) = C + T_p(I)$$

- The compile time does not depend on the instance characteristics.
- We will concentrate on estimating run time $T_p(I)$.

Methods to compute Time Complexity

1) Step Count Method

2) Tabular Method

- **Step Count Method**
 - Introduce global variable **count** into programs with initial value zero.
 - Statements to increment **count** by the appropriate amount are introduced into the program.
 - The value of the **count** by the time program terminates is the number steps taken by the program.

Method-I: Introduce variable count

EX:- 1) Iterative sum of n numbers

Algorithm sum(a, n)

```
{  
  s:=0;  
  
  for i:=1 to n do  
  {  
    s:=s+a[i];  
  
  }  
  
  return s;  
}
```

EX:- 2) Recursive sum of n numbers

```
Algorithm RSum(a,n)
{
    // for the if conditional
    if( $n \leq 0$ ) then
    {
        return 0;
    }
    else
    {
        return RSum(a,n-1)+a[n];
    }
}
```

- When analyzing a recursive program for its step count, we often obtain a recursive formula for the step count.
- We obtain the following recursive formula for above (RSum) algorithm.

$$t_{\text{RSum}}(n) = \begin{cases} 2 & \text{If } n=0 \\ 2 + t_{\text{RSum}}(n-1) & \text{If } n>0 \end{cases}$$

One way of solving such recursive formula is by using **substitution** method.

$$\begin{aligned}t_{\text{RSum}}(n) &= 2 + t_{\text{RSum}}(n-1) \\&= 2 + 2 + t_{\text{RSum}}(n-2) \\&= 2(2) + t_{\text{RSum}}(n-2) \\&= 2 + 2 + 2 + t_{\text{RSum}}(n-3) \\&= \mathbf{3(2) + t_{\text{RSum}}(n-3)} \\&\vdots \\&\vdots \\&= n(2) + t_{\text{RSum}}(n-n) \\&= 2n + t_{\text{RSum}}(0) \\&= 2n + 2\end{aligned}$$

The step count for Rsum is **2n+**

Tabular method

- Determine the total number of steps contributed by each statement per **execution × frequency**
- Add up the contribution of all statements

Method-II: Tabular method

- EX:- 1) Iterative sum of n numbers

Statement	s/e	frequency	Total steps
Algorithm sum(a, n)	0	--	0
{	0	--	0
s:=0 ;	1	1	1
for i:=1 to n do	1	n+1	n+1
s:=s+a[i];	1	n	n
return s;	1	1	1
}	0	--	0
Total			2n+3

EX:- 2) Recursive sum of n numbers

Presentation last saved: Just now

- EX:- 2) Recursive sum of n numbers

Statement	s/e	Frequency		Total steps	
		n=0	n>0	n=0	n>0
Algorithm RSum(a,n)	0	--	--	0	0
{	0	--	--	0	0
if(n ≤ 0) then	1	1	1	1	1
return 0;	1	1	0	1	0
else					
return Rsum(a,n-1)+a[n] ;	1+x	0	1	0	1+x
}	0	--	--	0	0
Total				2	2+x

$$x = t_{\text{RSum}}(n-1)$$

Space Complexity

- Amount of computer memory that is required during program execution as a function of input size.

OR

- Space complexity is the amount of memory it needs to run to completion

Space=Fixed part +Variable part

Fixed: varies from problem to problem. Includes space needed for storing instructions variables , constants and structured variables.[arrays , struct].

Variable: varies from program to program. Includes space needed for stack and for structured variables that are dynamically allocated during run time.

$$S(P)=C+SP(I)$$

Fixed Space Requirements (C)

Independent of the characteristics of the inputs and outputs

instruction space(space for code)

space for simple variables, fixed-size structured variable, constants

Variable Space Requirements (SP (I))

depend on the instance characteristic I

number, size, values of inputs and outputs associated with I

recursive stack space, formal parameters, local variables, return address

Example1

Algorithm abc(a,b,c)

```
{  
    return a+b+b*c+(a+b-c)/(a+b)+4.0;  
}
```

Problem instance characterized by the specific values of a,b and c.

If we assume one word (4 bytes) is adequate to store the values of each a, b, and c , then the space needed by abc is independent of the instance characteristics.

Therefore, $S_{abc}(\text{instance characteristics})=0$

Example2

Algorithm sum(a,n)

```
{  
    s:=0;  
    for i:=1 to n do  
        s:=s+a[i];  
    return s;  
}
```

The amount of space needed depends on the value of n.

Therefore, $S_{\text{sum}}(n) \geq (n+3)$

Example3

Algorithm RSum(a,n)

```
{  
    if(n ≤ 0) then return 0;  
    else return RSum(a,n-1)+a[n];  
}
```

Total no.of recursive calls n, therefore $S_{\text{RSum}}(n) \geq 3(n+1)$

Find time and space complexity for the following algorithm

```
Algorithm Add(a, b, c, m, n)
{
  for i:=1 to m do
  {
    for j:=1 to n do
    {
      c[i,j]:=a[i,j]+b[i,j];
    }
  }
}
```


SAMPLE QUESTIONS

- Define Performance Analysis
- Differentiate recursive and non-recursive algorithms
- Provide an example algorithm that uses recursion.
- Explain in detail about Towers of Hanoi problem
- Solve Towers of Hanoi problem using 3 disk
- Solve Towers of Hanoi problem using 5 disk
- Write an algorithm for Towers of Hanoi problem using recursion
- Write an algorithm for Towers of Hanoi problem using iterative method
- Explain in detail about step-count method

SAMPLE QUESTIONS Cont..

- Explain in detail about tabular method
- Why is it important to consider the step count of an algorithm when analyzing its efficiency
- Discuss the role of data structures in determining the step count of an algorithm.
- Explain the concept of nested loops and their impact on the step count of an algorithm
- Explain the process of creating a table using the tabular method to analyze an algorithm.
- Discuss the relationship between the tabular method and the Big O notation.

Advanced Algorithms & Data Structures

Asymptotic efficiency

- Asymptotic efficiency means study of algorithms efficiency for large inputs.
- To compare two algorithms with running times $f(n)$ and $g(n)$, we need a **rough measure** that characterizes **how fast each function grows** as n grows.
- **Hint:** use *rate of growth*
- Compare functions **asymptotically!**
(i.e., **for large values of n**)

Functions ordered by growth rate

Function	Name
1	Growth is constant
$\log n$	Growth is logarithmic
n	Growth is linear
$n \log n$	Growth is $n \cdot \log n$
n^2	Growth is quadratic
n^3	Growth is cubic
2^n	Growth is exponential
$n!$	Growth is factorial

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

- To get a feel for how the various functions grow with n , you are advised to study the following figs:

		Instance characteristic n						
Time	Name	1	2	4	8	16	32	
1	Constant	1	1	1	1	1	1	1
$\log n$	Logarithmic	0	1	2	3	4	5	
n	Linear	1	2	4	8	16	32	
$n \log n$	Log linear	0	2	8	24	64	160	
n^2	Quadratic	1	4	16	64	256	1024	
n^3	Cubic	1	8	64	512	4096	32768	
2^n	Exponential	2	4	16	256	65536	4294967296	
$n!$	Factorial	1	2	24	40326	20922789888000	26313 x 10 ³³	

Figure 1.7 Function values

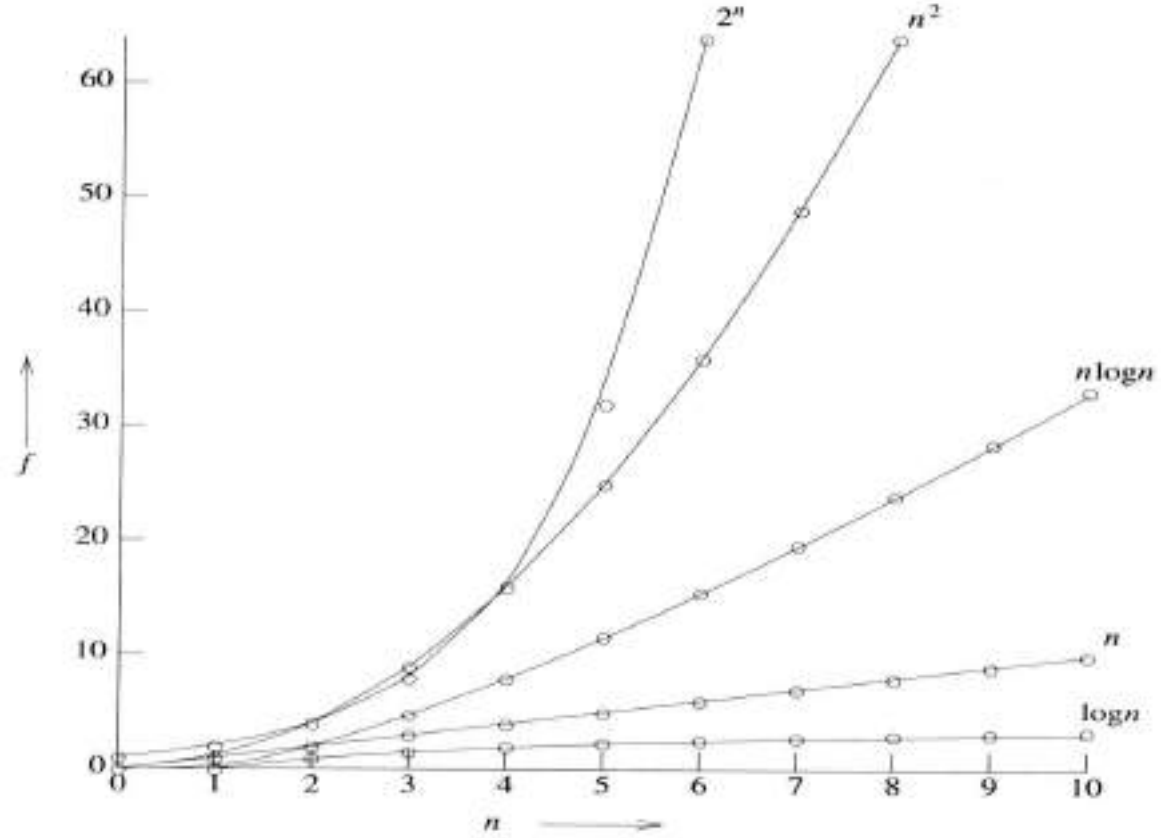


Figure 1.8 Plot of function values

- The low order terms and constants in a function are relatively insignificant for **large n**

$$n^2 + 100n + \log_{10}n + 1000 \sim n^2$$

i.e., we say that $n^2 + 100n + \log_{10}n + 1000$ and n^2 have the same **rate of growth**

Some more examples

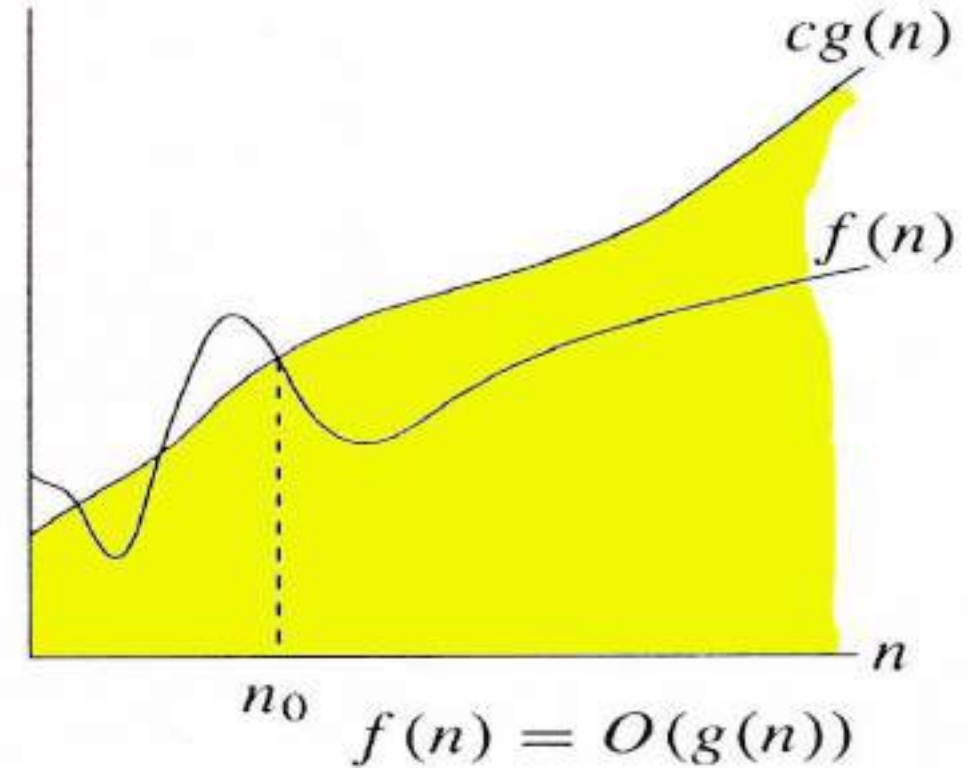
- $n^4 + 100n^2 + 10n + 50$ is $\sim n^4$
- $10n^3 + 2n^2$ is $\sim n^3$
- $n^3 - n^2$ is $\sim n^3$
- constants
 - 10 is ~ 1
 - 1273 is ~ 1

Asymptotic/order Notations

- Asymptotic/order notation describes the behavior of functions for the large inputs.
- Big Oh(O) notation:**
 - The big oh notation describes an **upper bound** on the **asymptotic growth rate** of the function f .

Definition: [Big “oh”]

- $f(n) = O(g(n))$ (read as “ f of n is big oh of g of n ”) iff there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n, n \geq n_0$.



- The definition states that the function $f(n)$ is at **most** c times the function $g(n)$ except when n is smaller than n_0 .
- *In other words, $f(n)$ grows slower than or same rate as” $g(n)$.*
- When providing an upper –bound function g for f , we normally use a single term in n .
- **Examples**
 - $f(n) = 3n+2$
 - $3n + 2 \leq 4n$, for all $n \geq 2$, $\therefore 3n + 2 = O(n)$
 - $f(n) = 10n^2+4n+2$
 - $10n^2+4n+2 \leq 11n^2$, for all $n \geq 5$, $\therefore 10n^2+4n+2 = O(n^2)$
 - $f(n)=6*2^n+n^2=O(2^n)$ /* $6*2^n+n^2 \leq 7*2^n$ for $n \geq 4$ */

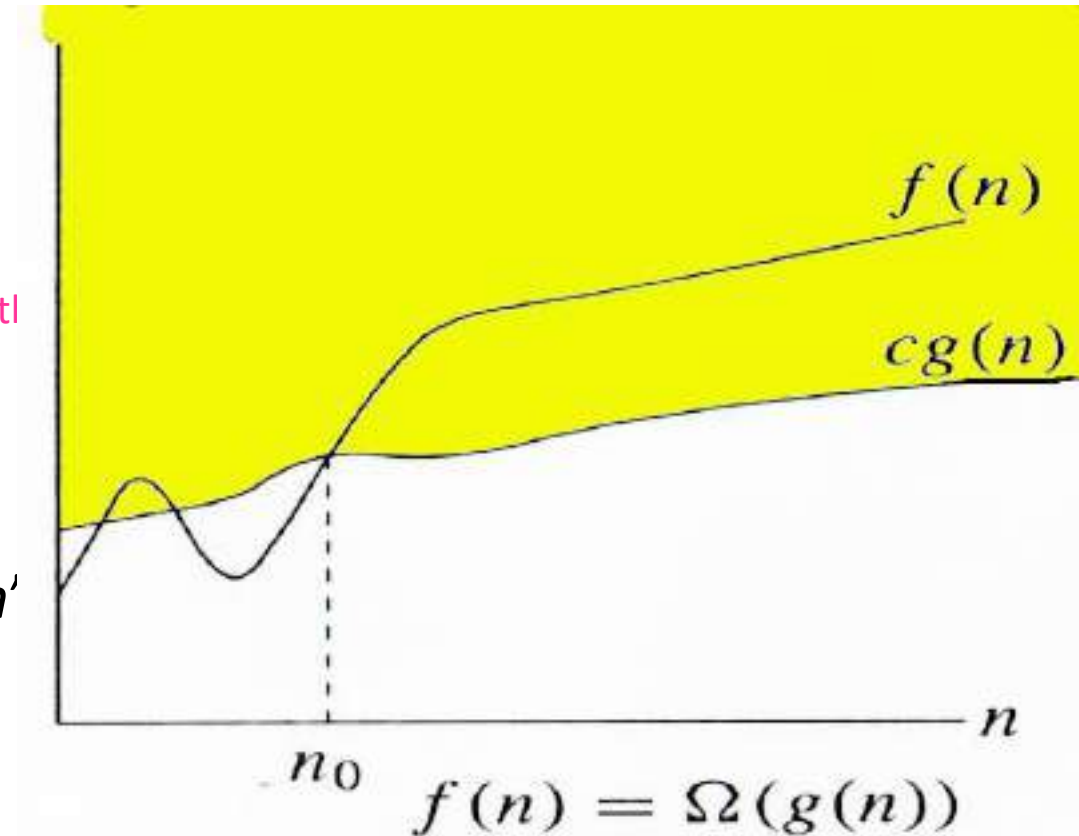
- It also possible to write $10n^2+4n+2 = O(n^3)$ since $10n^2+4n+2 \leq 7n^3$ for $n \geq 2$
- Although n^3 is an upper bound for $10n^2+4n+2$, it is not a tight upper bound; we can find a smaller function (n^2) that satisfies big oh relation.
- But, we can not write $10n^2+4n+2 = O(n)$, since it does not satisfy the big oh relation for sufficiently large input.

- **Omega (Ω) notation:**

- The omega notation describes a **lower bound** on the **asymptotic growth rate** of the function **f**.

Definition: [Omega]

- $f(n) = \Omega(g(n))$ (read as “ f of n is omega of g of n ” iff there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all n , $n \geq n_0$.



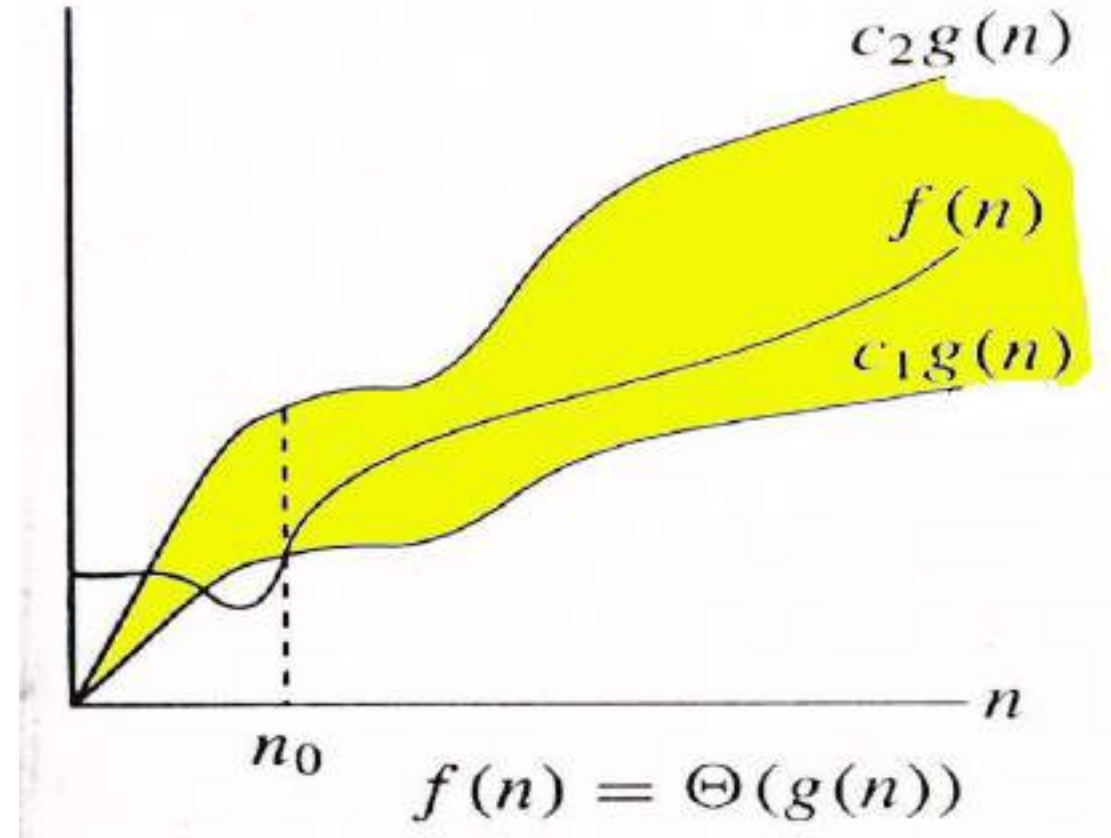
- The definition states that the function $f(n)$ is at **least** c times the function $g(n)$ except when n is smaller than n_0 .
- *In other words, $f(n)$ grows faster than or same rate as" $g(n)$.*
- **Examples**
 - $f(n) = 3n+2$
 - $3n + 2 \geq 3n$, for all $n \geq 1$, $\therefore 3n + 2 = \Omega(n)$
 - $f(n) = 10n^2+4n+2$
 - $10n^2+4n+2 \geq n^2$, for all $n \geq 1$, $\therefore 10n^2+4n+2 = \Omega(n^2)$
- It also possible to write $10n^2+4n+2 = \Omega(n)$ since $10n^2+4n+2 \geq n$ for $n \geq 0$
- Although n is a lower bound for $10n^2+4n+2$, it is not a tight lower bound; we can find a larger function (n^2) that satisfies omega relation.
- But, we can not write $10n^2+4n+2 = \Omega(n^3)$, since it does not satisfy the omega relation for sufficiently large input.

- **Theta (Θ) notation:**

- The Theta notation describes a **tight bound** on the **asymptotic growth rate** of the function **f**.

Definition: [Theta]

- $f(n) = \Theta(g(n))$ (read as “ f of n is theta of g of n ”) iff there exist positive constants c_1 , c_2 , and n_0 such that $c_1g(n) \leq f(n) \leq c_2g(n)$ for all $n, n \geq n_0$.



- The definition states that the function $f(n)$ lies between c_1 times the function $g(n)$ and c_2 times the function $g(n)$ except when n is smaller than n_0 .
- In other words, $f(n)$ grows same rate as $g(n)$.
- **Examples:-**
 - $f(n) = 3n+2$
 - $3n \leq 3n + 2 \leq 4n$, for all $n \geq 2$, $\therefore 3n + 2 = \Theta(n)$
 - $f(n) = 10n^2+4n+2$
 - $n^2 \leq 10n^2+4n+2 \leq 11n^2$, for all $n \geq 5$, $\therefore 10n^2+4n+2 = \Theta(n^2)$
- But, we can not write either $10n^2+4n+2 = \Theta(n)$ or $10n^2+4n+2 = \Theta(n^3)$, since neither of these will satisfy the theta relation.

Big-Oh, Theta, Omega

Tips :

- Think of $O(g(n))$ as “less than or equal to” $g(n)$
 - **Upper bound:** “grows slower than or same rate as” $g(n)$
- Think of $\Omega(g(n))$ as “greater than or equal to” $g(n)$
 - **Lower bound:** “grows faster than or same rate as” $g(n)$
- Think of $\Theta(g(n))$ as “equal to” $g(n)$
 - **“Tight” bound:** same growth rate
- (*True for large N*)

Example 1 - Iterative sum of n numbers

Statement	s/e	frequency	Total steps
Algorithm sum(a, n)	0	--	0
{	0	--	0
s:=0 ;	1	1	O(1)
for i:=1 to n do	1	n+1	O(n+1)
s:=s+a[i];	1	n	O(n)
return s;	1	1	O(1)
}	0	--	0
Total			O(n)

Example 2 - Addition of two $m \times n$ matrices

Statement	s/e	frequency	Total steps
Algorithm Add(a,b,c,m, n)	0	--	0
{	0	--	0
for i:=1 to m do	1	$m+1$	$O(m)$
for j:=1 to n do	1	$m(n+1)$	$O(mn)$
$c[i,j]:=a[i,j]+b[i,j]$;	1	mn	$O(mn)$
}	0	--	0
Total			$O(mn)$

Time complexity of Towers of Hanoi

$$\begin{aligned} - T(n) &= T(n-1) + 1 + T(n-1) = 2T(n-1) + 1 \\ &= 2 * (2 * T(n-2) + 1) + 1 \\ &= (2^2) * T(n-2) + 2^1 + 2^0 \\ &\vdots \\ &= (2^k) * T(n-k) + 2^{(k-1)} + 2^{(k-2)} + \dots + 2^0 \end{aligned}$$

Base condition $T(0) == 1$

$n - k = 0 \Rightarrow n = k;$

put, $k = n$

$$T(n) = 2^n T(0) + 2^{(n-1)} + \dots + 2^1 + 2^0$$

It is GP series, and sum is $2^{(n+1)} - 1$

$T(n) = O(2^n)$ which is exponential.

SAMPLE QUESTIONS

- What are asymptotic notations in algorithm analysis
- Why are they important in evaluating the efficiency of algorithms
- What is the difference between the best-case, worst-case, and average-case time complexities represented by Big O notation
- What is rate of growth
- Provide examples of algorithms and their corresponding time complexity expressed in Big O notation.
- Discuss the properties of Big O notation
- What are the rules for comparing and combining functions represented by Big O notation?
- Explain the concept of Omega notation.
- How does it differ from Big O notation, and what does it represent in terms of lower bounds of time complexity
- What is function value

SAMPLE QUESTIONS CONT..

- What is the purpose of Theta notation
- How does it provide a tighter bound on the time complexity of an algorithm compared to Big O notation?
- Explain the concept of space complexity in asymptotic notations. How can it be represented using Big O notation?
- Discuss the concept of logarithmic time complexity and how it is represented using asymptotic notations.
- Provide examples of exponential time complexity and how it is expressed in asymptotic notations.

